

6CCS3PRJ Final Year

Choosing good compiler optimisation flags using AI optimisation techniques

Final Project Report

Author: Thomas Holden

Supervisor: [redacted]

Student ID: 23052314

October 27, 2026

Abstract

Compilers are central to software development, but generic optimisation presets like -O3 fail to extract maximum possible performance for specific applications due to the infinite search space of optimisation passes; potentially wasting power and time. This project addresses the undecidable phase ordering problem by developing an autotuning system that utilises Artificial Intelligence to discover bespoke optimised compiler pass sequences. This is implemented as a Genetic Algorithm (GA) designed and implemented in Go to automatically navigate the vast LLVM pass search space. The tool operates from a generic build command, iteratively evaluating sequences using robust benchmarking practices to mitigate system environmental noise. Upon evaluation using the PolyBench suite, the GA-driven autotuner consistently outperformed standard -O3 optimisations by approximately 3% across all tested benchmarks. These results demonstrate the viability of evolutionary algorithms in accessible compiler optimisation, with potential to expand benchmark diversity and integrate search guided by heuristics.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Thomas Holden

October 27, 2026

Acknowledgements

I would like to express my sincere thanks to my supervisor, [redacted], for his patience and assistance throughout all stages of the project. His experience and advice was invaluable to the betterment of my academic writing skills and essential knowledge needed for my implementation.

Further I would like to thank my partner for her support and patience throughout the many late nights and days spent ensuring my dissertation was of quality.

Finally many thanks to the researchers whose work my dissertation has been built off of. Without the trial and error and various approaches they conducted, the research following would have been far more limited.

Contents

1 Introduction **5**

1.1 Motivation . . . 5

1.2 Scope . . . 6

1.3 Objectives . . . 6

2 Background **7**

2.1 Compiler . . . 7

2.1.1 History . . . 7

2.1.2 Optimisations . . . 7

2.1.3 Architecture . . . 8

2.2 Phase Ordering Problem . . . 9

2.3 AI optimisation . . . 10

2.3.1 Reinforcement Learning . . . 10

2.3.2 Supervised Learning . . . 12

2.3.3 Genetic Algorithm . . . 14

2.3.4 Other Metaheuristics . . . 17

2.4 Benchmarking . . . 18

2.4.1 Adaptive capping . . . 19

3 Design & Specification **21**

3.1 Requirement Specification . . . 21

3.1.1 User Requirement Specification . . . 21

3.1.2 Functional Requirement Specification . . . 22

3.1.3 Non-functional Requirement Specification . . . 23

3.2 Software Development Methodology . . . 24

3.2.1 Incremental delivery . . . 25

3.2.2 Version control . . . 25

3.3 Design . . . 25

3.3.1 Genetic Algorithm . . . 26

3.3.2 Evaluate Convergence . . . 28

3.3.3 Iteratively build population . . . 29

3.3.4 Iteratively compile and execute population . . . 29

4 Implementation **31**
4.1 Language . . . 31
4.2 Package structure . . . 32
4.2.1 Genetic Algorithm . . . 32
4.2.2 Test Harness . . . 38
4.2.3 Pass Manager . . . 43
4.2.4 Worker Pool . . . 44
4.2.5 Utilities . . . 46
4.3 Third-party libraries . . . 47
4.3.1 Shlex . . . 47
4.3.2 Progress Bar . . . 47
4.3.3 XXH3 . . . 47
4.4 Testing . . . 47
4.4.1 Test methodology . . . 48
4.4.2 Comprehensive tests . . . 49
4.4.3 Uncovered Components . . . 49

5 Results & Evaluation **51**
5.1 Performance Testing . . . 51
5.1.1 Benchmark choices . . . 51
5.1.2 Results . . . 53
5.2 Significance . . . 55
5.3 Comparisons to past results . . . 56
5.3.1 LLVM version . . . 57
5.3.2 Architecture . . . 57
5.3.3 Environment . . . 57

6 Conclusion and Future Work **59**
6.1 Conclusion . . . 59
6.2 Limitations . . . 60
6.2.1 Sample issues . . . 60
6.2.2 Heuristic guidance . . . 61
6.2.3 Version locked software . . . 61
6.3 Future Work . . . 61

7 Legal, Social, Ethical and Professional Issues **63**
7.1 Code of Conduct . . . 63
7.1.1 Legal . . . 63
7.1.2 Social . . . 63
7.1.3 Ethical . . . 64
7.1.4 Professional . . . 64
7.2 Environmental . . . 65
Bibliography . . . 66

A Extra Information **72**
A.1 Sensitivity analysis of executable repetitions . . . 72
A.2 Proof for median relative standard error . . . 73
A.3 Welch's t-test . . . 75
A.4 LLVM pass sequences . . . 75

B User Guide **80**
B.1 Instructions . . . 80

C Source Code **82**
C.1 Dockerfile . . . 82
C.2 go.mod . . . 82
C.3 go.sum . . . 83
C.4 main.go . . . 85
C.5 README.md . . . 87
C.6 example . . . 89
C.6.1 hello_world . . . 89
C.6.2 string_concat . . . 89
C.7 geneticalgorithm . . . 98
C.7.1 crossover.go . . . 98
C.7.2 crossover_test.go . . . 104
C.7.3 genetic.go . . . 110
C.7.4 genetic_test.go . . . 118
C.7.5 mutation.go . . . 129
C.7.6 mutation_test.go . . . 130
C.8 passmanager . . . 132
C.8.1 passes.go . . . 132
C.8.2 passes_test.go . . . 151
C.9 testharness . . . 158
C.9.1 median_confidence.go . . . 158
C.9.2 median_confidence_test.go . . . 159
C.9.3 testharness.go . . . 163
C.9.4 testharness_test.go . . . 165
C.9.5 testharnessutils.go . . . 172
C.9.6 testharnessutils_test.go . . . 184
C.10 util . . . 205
C.10.1 bidirectionalmap.go . . . 205
C.10.2 bidirectionalmap_test.go . . . 209
C.10.3 cache.go . . . 215
C.10.4 cache_test.go . . . 217
C.10.5 util.go . . . 224
C.10.6 util_test.go . . . 228
C.10.7 fileio.go . . . 240
C.10.8 fileio_test.go . . . 241
C.11 workerpool . . . 246

C.11.1 ga_worker_pool.go 246
C.11.2 ga_worker_pool_test.go 252

Chapter 1

Introduction

Compilers are the core of the majority of software development workflows; they act to translate human readable source code into executable binaries. For maximum efficiency, modern compilers like LLVM apply various transformations throughout the compilation process known as passes. Through careful combination of these passes source code can be lowered into a more optimised state. The vast majority of software engineers do not possess the deep, specialised knowledge required for manually tuning these passes, they typically rely on hardcoded performance presets like -O3. However, these static presets are designed to be as general as possible. As a result they can fail to achieve the maximum possible performance required for heavily memory or processing bounded applications.

1.1 Motivation

Efficiency, energy usage, and runtime are major factors that affect the work of software engineers making critical design decisions in time or energy constrained environments. By reducing these, an engineer can improve profits by saving power usage and reducing latency in high stakes environments like a stock exchange.

The options for optimising without interacting with the compiler are limited. A developer can only rewrite with more efficient implementations or transition to languages that are more performant, such as C, C++ or Rust.

Once these options have been exhausted, how can a general developer iterate on the efficient optimisation presets without having deep, intimate understandings of the inner workings of the compiler, the passes, and the countless ways in which they may interact with one another? It

would not be feasible to expect all software engineers to understand each compiler deeply enough to consistently interact positively and make any substantial difference over the performance presets.
Furthermore, the expectation for developers to learn each compiler's in-depth optimisation system, when a single project may require juggling multiple, is impractical.

As a solution this project proposes an autotuner, an algorithm that attempts to find improvements over the generic performance presets given a particular project and set of test cases or benchmarks.

1.2 Scope

Finding a *perfect* sequence of compiler optimisation passes for a specific piece of software is an inherently impossible task. This is due to the infinite nature of the search space, as a result of having no maximum length. Algorithms known as autotuners have been proposed as solutions, searching for bespoke improvements over generic presets.

This project sets out to tackle the phase ordering problem by designing and implementing an autotuner that utilises Artificial Intelligence, specifically a Genetic Algorithm (GA), to automatically discover optimal compiler flag sequences. The core aim of the project is to provide users with a tool that requires only a generic build command to automatically navigate this infinite search space and output an executable that successfully outperforms the standard -O3 optimisation level. Built using the Go programming language to leverage its efficient native parallelisation, the system's performance is ultimately benchmarked against the PolyBench suite.

1.3 Objectives

The proposed project sets out to build an autotuner leveraging a genetic algorithm, with several loosely defined objectives:

- Provide an easy to use command line application.
- Produce binaries whose performance surpasses that of compilation via generic presets.
- Acknowledging that a perfect sequence is impossible, construct a sequence that is, at worst, a local optimum.
- Ensure portability and a high level of compatibility.

Chapter 2

Background

The compiler (2.1) section details the background to compilers that provides the backdrop to optimisation. The sections on phase ordering (2.2) and benchmarking (2.4) explore the reasons for which compiler autotuning is an unsolved problem, making clear why most developers settle for less than optimal performance out of their code.
The section on Artificial Intelligence (AI) (2.3) optimisation and the techniques it employs showcases how past research has attempted to solve these problems.

2.1 Compiler

2.1.1 History

Compilers have been around for close to 70 years, depending on how they are defined. Optimising compilers emerged not long after, with the release of the IBM FORTRAN H compiler [43] written from 1963 to 1967. The researchers discuss how compiler optimisations relied heavily upon fixed optimisation levels even then, just as many do today. This is an important feature that allows the compiler's user base to abstract away from the inner workings of the compiler while still being guaranteed performant code for general use.

2.1.2 Optimisations

Hardcoded optimisation presets are heavily limited by their ambition to generalise well. It is not always the case that these performance presets, such as -O3 and -Oz, meet the desired speed necessary for performance critical, energy limited, or memory bounded software applications.
In cases like these, manually tuning the presets in hopes of generating more efficient machine

code for a use case is an option. But first, the greatest gains can be made by ensuring that the high-level source cannot be improved further.

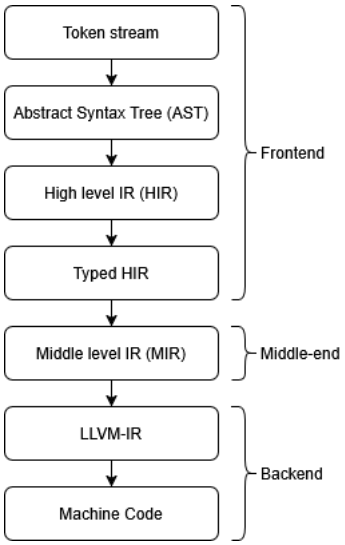
2.1.3 Architecture

The most performant languages mentioned previously, C, C++, and Rust, are by no coincidence all built atop the LLVM compiler infrastructure. This tool allows for transpiling source code into an intermediate representation (IR). Users who want to maximise performance will generally be using one of these languages. Therefore, LLVM can be considered as the primary contender for autotuning.

Frontend

Frontends of varying quality are available for a wide range of languages, from clang/clang++ for C/C++, to rustc for rust. These serve the purpose of taking a source project and outputting IR as their final step.
Along the way to producing this is usually a pipeline of increasingly lower level intermediate languages.

Figure 2.1: Showcases the numerous representation stages in the rustc pipeline. Representation specific optimisations can be applied to achieve better performance compared to a single step approach [45].



Intermediate Representation

The LLVM IR serves as the common denominator within any LLVM based compiler. It is a strongly typed virtual instruction set that abstracts away from the target machine's architec-

ture. A defining characteristic of LLVM IR is its use of Static Single Assignment (SSA) form, which dictates that every variable is assigned exactly once [42]. This vastly simplifies data-flow and, as a result, makes optimisations simpler to apply.

This stage is where the bulk of optimisations (passes) are executed. This makes it a critical point for autotuning. Manipulating the selection, sequence, and aggressiveness of these optimisation passes allows the autotuner to search for solutions to improve execution efficiency, benefiting any source language compiled through the LLVM toolchain.

Backend

The backend is responsible for generating targeted machine code from IR following its transformations [42]. This conversion involves several stages to exploit the specific features of the underlying hardware. The process begins with instruction selection, where the generic IR instructions are mapped to the instruction set of the native platform, such as x86-64, ARM, or RISC-V. This is followed by register allocation, which translates the virtual registers generated during the SSA phase to the CPU's limited physical registers.

Finally, instruction scheduling reorders the machine code to maximise hardware pipeline efficiency. The backend applies these target-dependent optimisations to ensure the final binary achieves peak performance on its designated execution environment.

2.2 Phase Ordering Problem

LLVM's opt has 200+ different transforming passes that can be applied to a given IR file, with -O3 itself containing ∼112 different passes, version dependent. Furthermore, with no hard limit on the length of the pass sequences, the search space to find an optimal pass sequence is truly infinite. Even limiting the search space to no more than 100 passes to compile in reasonable time, there are still more possible combinations than number of atoms in the universe. This makes navigating the search space to find a perfectly optimal pass sequence impossible. Contrastingly, it also gives us significant control and freedom in how to navigate this space.

This problem is commonly referred to as the phase ordering problem, which is considered to be undecidable as shown by Touati and Barthou [54]. In its general form there is no algorithm that can guarantee a perfectly optimal flag sequence for a given input. More specifically, you may have an algorithm that finds the optimal passes, e.g. one outputting only random passes may find the true optimal sequence on the first try, but this remains undecidable because this optimality cannot be verified in finite time. The authors further explain that by introducing

an increasing monotonic cost function to the problem, the search space is limited. This makes the problem decidable by reintroducing the opportunity to iterate each possible sequence. In practice, this is implemented as a hard cap on the length of pass sequences or time spent optimising to accomplish this goal. In this case, the problem remains intractable, but this can be tackled by utilising heuristic approaches or heavy parallelisation.

2.3 AI optimisation

AI parameter optimisation, known as Algorithm Configuration [34], is a search technique that takes advantage of intelligent approaches to navigate expansive search spaces where traditional means fail. When dealing with complex algorithms that have many configurable parameters, manual tuning or exhaustive search becomes impossible. Instead, AI optimisation leverages intelligent methods to systematically navigate massive configuration spaces and escape local optima. Given the vast undecidable nature of the search space of LLVM passes, finding an optimal pass sequence within a reasonable time-frame is unverifiable. As a consequence, recent research has increasingly turned to AI approaches.

Tackling the noisy problem of optimising compilations for execution speed, maximum memory usage, binary size and compilation speed has been the subject of extensive research, utilising various methods such as reinforcement learning [29], genetic algorithms (GAs) [52], decision forests [28], and many more.

2.3.1 Reinforcement Learning

Reinforcement learning is an approach to machine learning where the solutions for a problem space are unknown. This is achieved by approximating optimal mappings from environments onto actions based upon numerical rewards [51]. Such models often include hyperparameters that effect how they consider immediate versus future reward, and how they balance exploring the search space against exploiting the best past actions. Ultimately, they allow for modelling good actions to take in uncertain environments.

There have been attempts to tackle the compiler optimisation domain with reinforcement learning (RL) [29, 28]. The commonly cited reasoning being, "RL can learn with fewer samples [...] and can co-optimise for multiple objectives" [29]. Alongside this is the fact that the alternative supervised models require labelled input output pairs. These are difficult to gather for sufficiently large datasets without first building another model accomplishing this.

A noteworthy issue of using machines learning approaches to this issue lies in arranging the input source code or IR in a format that can be utilised for further computation. This conversion is often lossy, as shown by Haj-Ali et al who choose to extract key features from the IR, including but not limited to, "number of basic blocks, branches, and instructions of various types" [28]. This approach is reasonable given the problem, however, fails to encode perfectly the syntactic complexity of the source. Reconstructing the original program is impossible based on the key features alone and as such any model relying on them exclusively will be handicapped by the limitation of a partially observable environment.

Alternatives involve using vectorisation tools, "Code2vec is a [...] model [...that...] represents a code snippet as a single fixed-length code vector," [29]. This encodes significantly more data about the code snippets, especially since here they focus upon only loops. However still with 340 dimensions and an assumed 32 bits, there is only a finite space to encode an infinite possible collection of loops, making the conversion lossy by nature of the pigeonhole principle. In this case however, the authors do not need to encode every possible code block. They care more about distinguishing the features that are most relevant to the choice of optimisation passes; this is assumed to be a much smaller subset of features.

Graph Neural Networks

Graph Neural Networks (GNNs) offer a promising alternative to overcome the inherently lossy nature of fixed-length vectorisation and manual feature extraction. By natively representing programs as graphs, GNNs can accurately preserve the complex structural and semantic dependencies of the source code.

A framework that seeks to implement this is seen with ProGraML (Program Graphs for Machine Learning), modelling the compiler's Intermediate Representation (IR) as a directed, attributed multi-graph [10]. This comprehensive representation captures a program's control flow, data dependencies, and call relations, without discarding the details of specific instructions and operands. Information is then propagated through this architecture using Message Passing Neural Networks (MPNNs). This allows the model to reason over the program structure and dependencies by propagating information across the edges between vertices. By treating the code as an interconnected graph rather than a fixed size vector, GNNs bypass the pigeonhole principle. This provides a highly expressive, language-agnostic foundation that allows machine learning models to extract information out of an environment more observable than any alternatives. This allows for a focused navigation of the optimisation search space, predicting the

most performant passes with greater confidence.

This is a rich enough summarisation of a program's state that it was be used to achieve a 94.0% F1 score on compiler analysis tasks over a broad range of 250k different LLVM-IRs [10]. Additionally an algorithm classification accuracy of 96.2% signifies how well the representation encapsulates data about the source code as a whole. This is key for applying GNNs for auto-tuning workloads where any pass' efficacy is dictated by the intricate relationships within the source code.

2.3.2 Supervised Learning

As aforementioned, supervised models require labelled training data. Further, they require a large volume of data points relative to their parameter count [30]. In this domain this is a significant problem due to the inherent complexity of the problem requiring a huge number of parameters to correctly model the interactions between the passes and source. This is incompatible with the data scarcity for perfectly optimal parameter code pairs. As the problem is intractable it is difficult to increase the volume of known high-quality solutions.

Support Vector Machines

Support vector machines (SVMs) are linear supervised learning models that can utilise the *kernel trick* to learn non-linear relationships within the training data they are given [9]. This is accomplished by finding the most optimal hyperplane that separates classes by a margin of largest separation [9].

Their usage in compiler autotuning is observed in the work integrating an SVM-based model into Testarossa, a commercial Just-in-Time (JiT) compiler employed in the IBM J9 Java Virtual Machine [48]. The researchers extracted 71 numerical features from source code and used this to train a multi-class linear SVM. The objective was to predict a compilation plan on a per-method basis that optimally balanced compilation effort with generated code quality.

Despite the robustness of this machine learning framework, the researchers struggled to observe comprehensive performance improvements, especially when evaluating long running program performance. In this domain, the SVM models fell short because they occasionally disabled code transformations that were beneficial to the program's overall execution speed. Although the SVM approach successfully reduced the overall compilation time by roughly half [48], its inability to outpace traditional, human-engineered heuristics for execution time of long-running applications underscores the difficulty of modelling the dependencies of compiler passes

using linear SVMs against a highly refined industry baseline that has undergone manual tuning over the course of years.

Bayesian optimisation

Bayesian optimisation is a method used to optimise a *black box* function that makes progress by iteratively maximising the probability of improvement [21]. These are especially effective when it comes to the evaluation of noisy or expensive to run functions; they can rely on a probabilistic surrogate model that is representative of beliefs about the optimisable function instead of wasting time evaluating the function directly. Using this model the method assigns scores to the available steps through the search space according to a chosen acquisition function, and chooses maximally from the options available.

Standard Bayesian optimisation struggles to scale to high-dimensional search spaces, such as the vast array of compiler optimisation flags. Traditional Bayesian optimisation models heavily rely on Gaussian Processes (GP) to approximate the objective function. However, GPs are computationally limited when faced with high-dimensional data; they are inefficient for compiler autotuning where tens to hundreds of interacting flags must be considered [7].

To circumvent these limitations within the compiler domain, recent research has demonstrated success by fundamentally altering the underlying surrogate model [7]. This was accomplished by replacing the traditional Gaussian Process with a Random Forest, better suited for high-dimensional data. This enabled effective learning of the complex, non-linear interactions and dependencies between various optimisation passes. The resulting model captures the mean and standard deviation of predictions across its multiple decision trees, allowing it to evaluate the Expected Improvement acquisition function, which requires an expectation and confidence, without facing the curse of dimensionality.

To manage the undecidable nature of the phase ordering problem, the research employs a targeted selection strategy that measures flag importance [7]. This allows the algorithm to isolate a small subset of optimisations deemed important. It continues to explore less important optimisations, gradually reducing the random exploration of these as the model's precision improves. This targeted approach yields significant results. The authors conducted tests on GCC and LLVM against baseline heuristics and competing approaches like GAs, where comparable performance speed-ups were found in significantly less time. The success of this method was largely tied to the underlying statistical models, explicitly tailored to handle both the dimensionality of the flags and the noisy nature of execution times.

2.3.3 Genetic Algorithm

Genetic algorithms (GAs) are a form of guided search that imitates biological processes to find an approximate solution within a search space. They have long been used to overcome the difficulty in finding a good solution to a problem in domains that are infeasible to enumerate. They work by mimicking natural selection: randomly generating a population and employing a repeated feedback loop of fitness evaluation to iteratively find the fittest among them to seed the next generation, imitating the "survival of the fittest" seen in nature [33]. Just as real speciation, they are made especially effective at avoiding local optima by maintaining a genetically diverse population [47]. This prevents premature convergence to the first promising candidate solution and can be achieved by measures like mutation, randomly flipping a gene.

As a result of their effective ability to find good enough solutions even given massive search spaces [52], GAs have been proposed numerous times [52, 20, 49] for compiler flag optimisation.

There are flaws to GAs. Their stochastic nature means that they are never guaranteed to find an optimal solution, even in a relatively small search space. Furthermore, in line with the building block hypothesis, performance of GAs is only better than completely random search when each, 'schema' is "short, low-order[ed] [...] [&] highly-fit," [18]. Here, a schema is an interacting set of genes. Short implies that the length of the schema is low enough to not cause issue when randomly split. Low-order implies that the schema is made up of few bits. Highly-fit implies that the performance of the schema is high with respect to the average schema.
In practice, this means that GAs struggle to make progress when good solutions include schema that may be made up of many interacting genes that are far apart. For the problem space, this is a significant issue; there is very high interaction between LLVM passes, they are not guaranteed to act only between two passes and may even be at opposite ends of the genome.

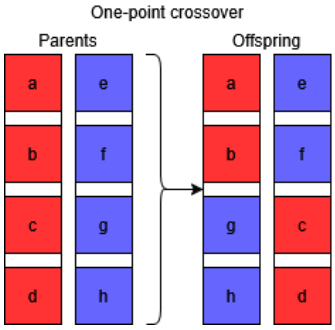
To combat the building block hypothesis, some approaches use a guided approach via heuristics that work to preserve the highest performing schemas.
It has been found that, "[Estimation of Distribution Algorithm (EDA)] strategy outperforms [...] GA," [20] due to their ability to learn the dependencies between different optimisation phases throughout execution.
Contrasting with other search approaches, genetic approaches have certain features that are well fitted for finding an optimal set of phases in reasonable time. They can halt via simple logic, as their solution can be considered optimal when they fail to find any improvement over a number of generations [52]. This number can be tweaked depending on how much time is available and how optimal a solution is necessary. Additionally, they have many adjustable

hyper-parameters that can help to optimise learning speed, including crossover, mutation and selection types [52].

Crossover

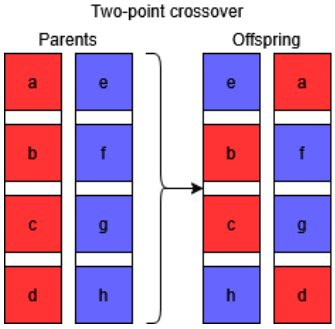
There are numerous crossover approaches for GAs. Unfortunately not all are compatible with genomes of differing lengths without modification. Outlined below are the main forms of crossover with static length genomes alongside the changes necessary to allow working with dynamic length genomes - if any.

Figure 2.2: A simple example of fixed length one-point crossover.



Often regarded as the simplest form of crossover, one-point crossover chooses an arbitrary index in the genome as the location to *split* the parents in two [24]. In Figure 2.2 this index is 1 (assuming 0 based count and split after the index). Then the children are formed by joining the former half of the genome from one parent with the latter half of the other.
This can be easily adapted to dynamic length genomes by selecting a point from within the first parent, and then a separate point within the second parent. By splitting the parents from here can proceed as for static one-point crossover.

Figure 2.3: A simple example of fixed length two-point crossover.



The next step in complexity is illustrated in Figure 2.3. Here two indices are chosen the *split* the parent's genomes on, as opposed to one. The indices chosen in 2.3 would be 0 and 2

(assuming 0 based count and split after the index). The children are formed by taking the start, middle, and end of each split genome and simply swapping the middle [24]. Two-point crossover is capable by definition of generating any sequence that can be generated by one-point.
Similarly to before this is adaptable to dynamic length genomes by selecting two points from the first parent, and two points from the second parent. Splitting on these points, both genomes are in three pieces, then proceed as with static.

Another common form of crossover is uniform crossover [24]. Here the genomes of the parents are iterated and a coin is flipped. When the coin is heads the first offspring gets their next gene from the first parent and the second offspring from the second parent. When tails, the opposite holds. This leads to completely shuffled genomes between the two offspring.
When adapting this for dynamic length genomes a pointer for each genome must be maintained. Instead of iterating through the genes of the two parents in parallel a coin is flipped to decide if the pointer should be increased for the first or second parent. Another coin is flipped to decide which child should the gene from the randomly chosen parent be appended to. If at any point one of the parent has length 0, the remaining genes are split randomly between the two children.

The issue with the above forms of crossover are their capability of being destructive [18]. Without any guidance or heuristic, the proportion of these forms of crossover's children that are worse than their parents is likely to be high. This is thanks to them breaking dependency chains, especially prevalent in sequences of optimisation passes where the tuned ordering of genes is what increases fitness.

Heuristic Guidance

Relying solely on unguided offspring generation is unlikely to generate superior offspring each generation given the volume of possible different solutions.
As with any search problem, heuristics are a key tool that allow for more efficient navigation within the space. They can be used to identify likely useful passes given a specific context. Haj-Ali et al, "use random forests to analyse the relationship between program features and optimisation passes [...] then leverage this relationship to reduce the search space by identifying the most likely optimisation phases to improve the performance," [28]. This takes what would be a blind search through every possible pass sequence and biases exploration toward passes that are statistically more likely to be performant for a given set of program features. By prioritising promising regions of the search space, the optimisation process can converge

rapidly by exploiting known good optimisation. The time saved in this process leaves time that can be allocated to finding a better solution locally.

Utilising these heuristics is a careful balancing act. To avoid replicating past experiences that may be suboptimal, there is a need to ensure exploration of the search space; simultaneously, relying on the heuristic model can narrow the set of transformations considered at each step, significantly reducing the candidate solutions compared to brute force or random search strategies.

2.3.4 Other Metaheuristics

Metaheuristics are a family of algorithms designed to govern the interaction between local and global improvement procedures [24]. They facilitate robust exploration of complex solution spaces, specifically by providing mechanisms to escape local optima where a search might otherwise terminate prematurely without reaching a global maximum or minimum.

Simulated Annealing

Simulated Annealing (SA) is an approach to function optimisation that can be used for discrete or continuous domains. This is accomplished by randomised local search that allows hill-climbing moves with a lower probability to escape local-optima [24]. This is modelled after the process of cooling seen by solids, starting at a *high temperature* (random state) and slowly cooling towards an approximation of the global optimum.

Unfortunately, unlike other methods, SA requires that sequential execution progresses through the search space one step at a time. Multiple instances can be run in parallel to allow for speed-up however these instances are isolated. This contrasts other methods whereby parallel execution can allow one instance of the algorithm to evaluate the search space more rapidly.

As a result of the aforementioned issues, SA has been applied to autotuning in a way unlike those seen by other optimisation techniques. Research proposes optimising tile size and unroll factor as opposed to optimising specific discrete passes [39]. The authors propose doing this with all other optimisations enabled, allowing their chosen parameters to further tune already optimised code. This approach was effective, and consistently the SA approach was seen to achieve speed-up ranging from 1.13-18.8x.

Particle Swarm Optimisation

Particle swarm optimisation (PSO) mimics social behaviour between particles, solutions within the search space, to find the optima within a problem space [60]. Each particle's solution is mapped to a position in higher-dimensional space. After evaluation of all particles, they are given a velocity value based on the best performer within the nearest `N` particle's positions as well as an internal best visited position. Via this velocity, new areas of the search space can be explored and incrementally better solutions can be found.

This makes a few assumptions about the search space that make it largely unusable for optimising LLVM:

- The number of dimensions is static from start to finish of the program. This is a problem due to the length of an LLVM pass sequence not being strictly fixed. A rough resolution for this could be starting with a hard limit on sequence length. Issues would remain however as changing positions must somehow accommodate for passes being added or removed.
- In PSO, the particles move around within a continuous domain. This must somehow be mapped correctly to a pass sequence, via rounding and integer mapping etc. Somehow mappings outside of the allowed range must also be discouraged.

Research circumvents the first issue by utilising GCC over LLVM, allowing for a fixed size feature space [60]. The researchers represent the active flags using a binary 1, and the inactive flags as 0. Furthermore, they solve the continuous domain issue by adopting a Binary Particle Swarm Optimisation (BPSO) approach. Instead of treating velocity as a displacement in physical space, it is interpreted as the probability of a bit flipping its state. This probabilistic mapping allows the algorithm to retain the global search dynamics of PSO while operating strictly within the discrete constraints of a compiler flag combination.

2.4 Benchmarking

Attempting to benchmark the execution time of a program binary is inherently an unstable process. This is largely due to the nature of operating system scheduling, the cache hit ratio, and other uncontrollable factors such as temperature [32].
In effort to combat this, multiple measurements must be taken in order to get an aggregate time taken. The best avenue to aggregate these measurements is a matter of debate:

- **Mean:** The arithmetic mean is a common measure of central tendency [15]. In the context of software benchmarking, uncontrollable systemic factors such as background operating system processes, sudden context switches, or unexpected network activity can cause spikes in execution time. Due to the sensitivity of the mean, these outliers can cause skew, producing an aggregate metric that is unrepresentative of the program's applied performance.

- **Minimum:** Taking the minimum execution time across all runs isolates the absolute best-case scenario. This approach operates on the assumption that the fastest run is the one that suffered the least interference from environmental noise, such as cache misses or OS pre-emptions. While it strips away the impact of outliers, the minimum represents an ideal execution run that does not reflect the expected performance of the binary under normal operating conditions; consider the unusual circumstance where the cache is perfectly primed for execution. This would achieve an impressive minimum execution time. Unfortunately execution time with a primed cache does not indicate how well it will run with a realistic cache hit ratio [22].

- **Median:** The median represents the middle value of a sorted dataset [58]. Unlike the mean, the median is highly robust to statistical outliers [13]. Occasional performance spikes caused by environmental noise will shift the bounds of the dataset but will not disproportionately pull the central value, providing a much more stable measurement of an average [41].

Upon evaluation, the median emerges as the most reliable metric to isolate the true performance of a compiled binary from the noise of the host hardware and operating system. By filtering out outliers while preserving the central tendency of the data, the median provides a representative baseline that accurately reflects the typical performance of the binary in practice.

2.4.1 Adaptive capping

Adaptive capping is an optimisation technique used to accelerate algorithmic search by avoiding unnecessary, time-consuming evaluations [34]. Originally introduced in frameworks like ParamILS, it tracks the performance of the best known candidate in a search, referred to as the incumbent. When evaluating a new candidate solution, the evaluation process is adaptively halted (capped) based on the incumbent's prior performance. If the cumulative runtime of the current evaluation exceeds the time taken by the incumbent solution, the run terminates early

because it is impossible for the candidate to outperform the current best. This strategy significantly reduces the computational budget wasted on suboptimal portions of the search space, permitting the algorithm to use the limited time available to explore more optimal solutions.

In the context of navigating an uncertain search space, like that of execution times, this approach is inherently flawed. If an algorithm testing for execution time halted the search immediately upon execution taking longer than time taken for the incumbent, it could miss out on optimal parts of the search space. This is due to the inherent noise of benchmarking discussed in section 2.4 which can lead to optimal solutions taking longer than expected on noisy runs. However, by limiting the execution time based upon a multiple of the incumbent's performance instead of the direct performance, the evaluation of whether a candidate solution is more optimal can account for noise.

Chapter 3

Design & Specification

This chapter will outline all of the user, functional, and non-functional requirements of the project that should be met for the project to be considered a success. Following this the methodology and approach to the development of the project will be discussed, before moving on to the design of the core components of the system. Each component's design is illustrated using a diagram illustrating the flow of data and processing throughout execution.

3.1 Requirement Specification

3.1.1 User Requirement Specification

ID	Description	Specification
U1	As a user, I *must* be able to autotune any project that can be built via clang/++ within the PolyBench suite.	The entry point of the program will give the user the opportunity to type in any clang command.
U2	As a user, I *must* only be required to provide a generic clang/++ command to initiate autotuning for my project.	By limiting user interaction to a single build command, any user capable of compiling through clang should be able to utilise the system.
U3	As a user, I *must* be able to view command-line usage instructions on demand.	When called with the -help command, use information will be provided to the end user. In normal execution basic guidance will be provided.

U4	As a user, I *must* find the resulting optimised executable in the same directory and with the same filename specified in my initial command.	An intuitive approach will be utilised whereby the executable will be output with a filename according to the argument given after the -o flag, as with any standard clang command.
U5	As a user, I *must* be able to run the autotuner on any system that can build a project via clang/++ and start a container via Docker.	Launching via the command line interface will be possible on any system.
U6	As a user, I *must* be provided with an estimated remaining execution time at the start of each generation.	An approximated execution time can be provided to the user for each generation. Predicted number of generations required can be used to estimate execution time.
U7	As a user, I *must* be able to utilise past results on future runs of the autotuner.	Starting passes will be seeded with sequences loaded from a file. The file will be made up of previously optimised pass sequences.

3.1.2 Functional Requirement Specification

ID	Description	Specification
S1	The system *should* parse a standard clang command and autotune without further user tweaking necessary.	Utilising libraries to parse a user's command into individual arguments we can abstract the argument handling from the user themselves.
S2	The system *should* guide the user with prompts and information at every step of execution.	Input prompts detailing what is needed from the user will be presented.

S3	The system *should* output the optimal pass sequence to a local file and build the executable with these applied.	The optimal flag set can be presented in clang compatible format. The executable will be produced by calling clang within the current directory.
S4	The system *should* default to tuned hyper-parameters.	By testing a diverse range of hyper parameters with a range of different software benchmarks we can identify what set generally performs best.
S5	The system *should* keep active core count consistent throughout the autotuning process.	Idle cores will be kept busy by dummy build commands until the end of each generation.
S6	The system *should* log elapsed time to the console upon the completion of each member in a generation.	The time from beginning of execution will be managed by external dependencies.
S7	The system *should* save/load past optimal sequences.	After execution the system will dump optimal flags and a unique comparable identifier to the current program. This allows the system to dynamically load relevant optimal flags to improve the system starting point.

3.1.3 Non-functional Requirement Specification

ID	Description	Specification
N1	The system *should* be documented according to the mainstream best practices for the language it is written in.	Each class, method and global variable within the system will have documentation comments.
N2	The system *should* require no additional arguments compared to a generic clang build command.	Every measure will be taken to ensure the system is only as cumbersome to use as a normal build process.

N3	The system *should* not exceed 4GB of active memory usage during its search execution.	Software size will be minimised to ensure cache utilisation and efficient algorithms will be used to keep memory usage as low as possible.
N4	The system *should* be deployable as a Docker container for high compatibility.	By running the system within a docker container we need only ensure compatibility for a singular OS while ensuring the artifacts are portable.
N5	The autotuner must complete its optimisation process within a user-defined time limit, defaulting to a maximum of 60 minutes.	The software will accept a -timeout <minutes> argument. The genetic algorithm will check the elapsed execution time at the end of each generation. If the time limit is reached, the search space traversal will halt and perform clean-up.
N6	The system *should* optimise an executable with a verifiable decrease in runtime compared to the standard clang -O3 optimisation level on the provided benchmark suite.	By traversing the search space of flag combinations and ordering via a genetic algorithm a high-quality solution can be found for any given build command.

3.2 Software Development Methodology

The project adopted an iterative agile software development life cycle. This was chosen due to the tight integration requirements of the system as a whole. It is composed of several subsystems, the command line interface (CLI), test harness, genetic algorithm, LLVM pass manager, and worker pool. This, as well as the delicate nature of running fuzzed pass sequences, deemed that alternative approaches such as waterfall were incompatible as a result of the high level of interaction components require.

3.2.1 Incremental delivery

Development of the project began with the objective of coding a minimum viable product (MVP) [50]. This focussed on producing a bare bones environment capable of running an end-to-end compilation with chosen passes and attaining a reliable benchmark measurement, the beginnings of the test harness. Following this, the system was build upon incrementally:

- Iteration 1: the pass manager was developed. The focus of this subsystem is to store a list of passes accessible via integer mappings that are indicative of their properties. From this it is possible to build and format required pass sequences.
- Iteration 2: the GA and basic operators were written. These allow for basic testing of the system as a whole.
- Iteration 3: from here the worker pool was developed to manage parallel processing of the genetic algorithm. This is interfaced with the GA via parallel go routines and channels.

This vertical slice of the system was incrementally fleshed out with new features to produce the final product through a process of testing and evaluation to tune the best genetic operators and parameters.

3.2.2 Version control

To support the incremental development, Git was used via GitHub for strict version control management and stability. This was vital as it requires a great deal of processing time to run the system, and as such occasional updates can introduce regressions before testing takes place. Any performance issues could be isolated via rolling back to an older commit and rectified from there.

3.3 Design

The system is a dockerized command line application (U5, N4). It is implemented in Golang (Go), a language suited to genetic algorithms thanks to its scalability, easy and performant parallelism, and simple communication between threads [53]. The application schedules the running of Clang and LLVM's optimiser with the intent of optimising for a C/++ project. User time measurements are taken using Go's OS package specifically excluding system time and reducing noise.

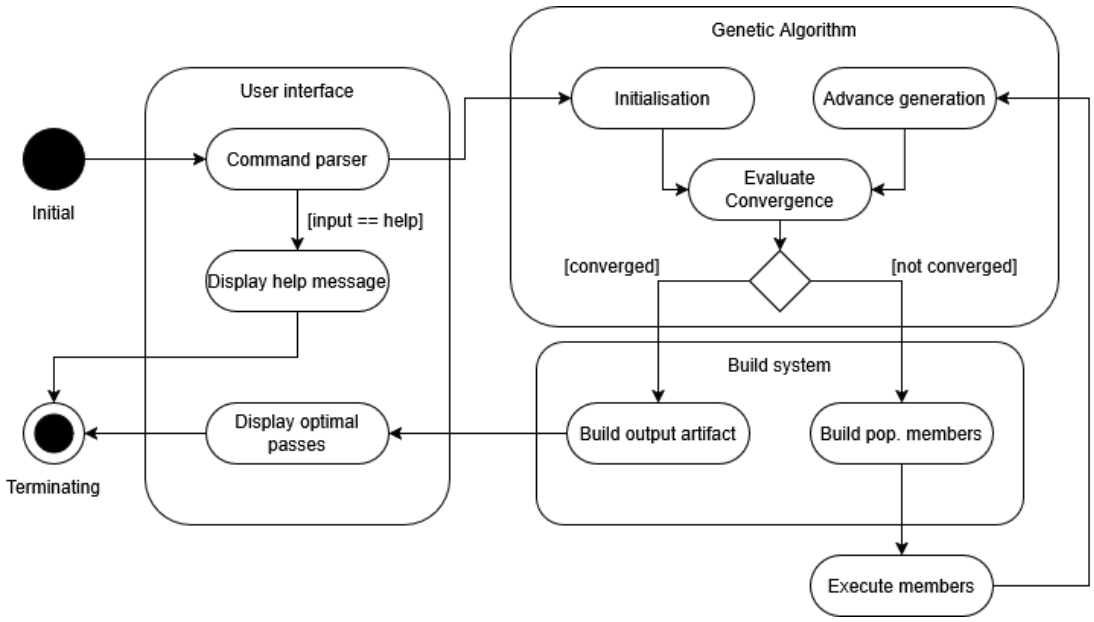


Figure 3.1: High-level overview of system functionality.

Figure 3.1 makes clear the program flow and main components of our system. The user interface utilises a command parser to abstract command handling away from the user (U2, S1). Additionally it gives user access to a help command that can list any optional parameters and provide basic guidance (U3). Each generation is characterised by the steps (assuming non-convergence):

- Advance Generation
- Evaluate Convergence
- Iteratively build population
- Iteratively execute population

The focus going forward will be on the design of the individual components necessary to implement this functionality, and the interaction between these components.

3.3.1 Genetic Algorithm

The general general flow of a genetic algorithm is shown in Figure 3.2, using truncation selection [37], single-point crossover, and random resetting mutation. It assumes that an initialisation generation has already run, generating a completely randomised population and executing to get a fitness value for sorting. Hyperparameters for this step are selection type, crossover type, mutation type, mutation chance, and proportion of elite members to persist between

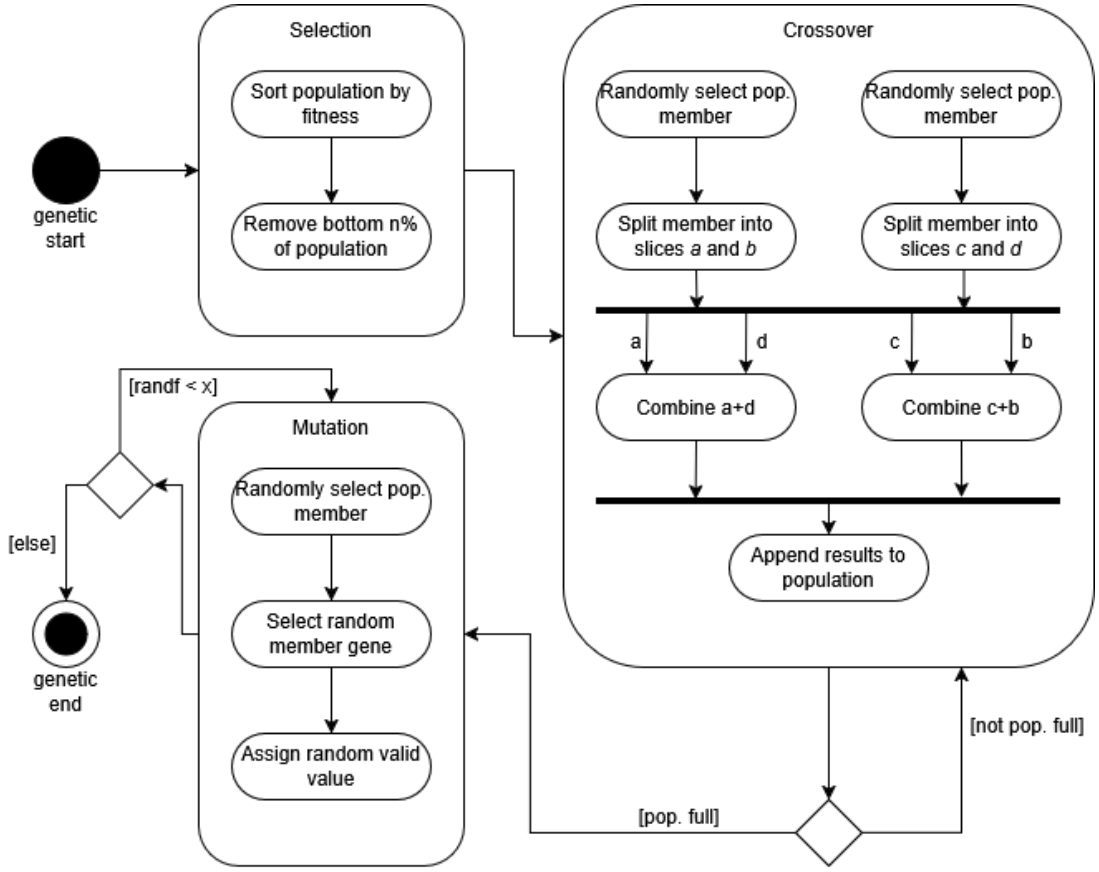


Figure 3.2: A basic functional genetic algorithm. Illustrates how we iterate the population to perform crossover and mutation.

generations. These will be assigned sensible default values while being customisable for power users (S4).

3.3.2 Evaluate Convergence

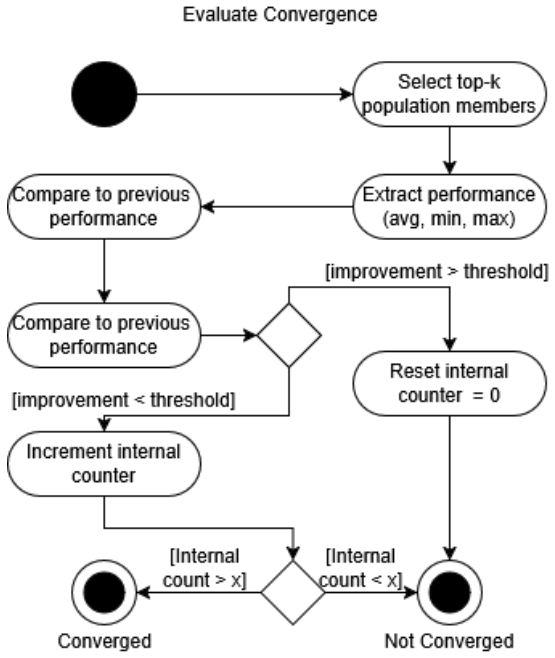


Figure 3.3: The system by which the GA verifies whether or not it has stopped finding better sequences.

Convergence is a key part of an efficient and effective genetic algorithm implementation. Without this a GA is likely to over or undershoot the optimal section of the search space. This either leads to us wasting energy, or losing out on performance. Figure 3.3 illustrates one way this check can be performed. It is important to balance the threshold by which we decide convergence with the other hyperparameters. If the population changes too much or too little it may cause premature convergence by failing to find small improvements when we get close to the optimum. Our convergence check must also account for time-elapsed since the start of execution. If this exceeds the user-defined or default value the system converges early (N5).

By ranking the population and taking the best member to build the final executable the user will have access to their optimised binary. This will be built in the same directory as the autotuner as to be easily locatable (U4, S3).

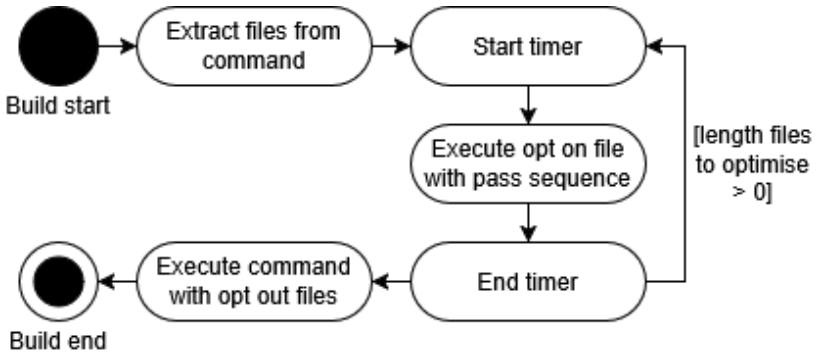


Figure 3.4: The build process. Utilises LLVM's OPT and its passes to act as our fitness function.

3.3.3 Iteratively build population

Figure 3.4 depicts the system's build process. By design the system should be as generic as possible and allow for compiling most projects by affecting the build process as minimally as possible (U1). This step yields information on the combined time taken to optimise all of the files alongside binary size, values commonly chosen to decide the fitness of a population member. In practice, we generally do not want to optimise for compile time; it is often inversely correlated to our other performance metrics and its inclusion introduces noise from time measurements. However, it remains as an accessible indicator for assessing build complexity.

3.3.4 Iteratively compile and execute population

Figure 3.5 showcases not just the compilation phase but also the loop used to calculate a reasonable time estimate for execution of the optimised executable. As mentioned, the median was chosen over any other averaging technique due to its enhanced ability to ignore outliers [13] compared to the mean or minimum. This is very useful in a time measurement context since the unpredictable nature of CPU and OS scheduling can lead to a log normal distribution for executable benchmark results [32].

Keeping core counts and workload as consistent as is computationally possible is necessary to minimise noise. To do this a consistent number of workers will be run at all times; this is easily achievable by producing dummy load on the machine whenever idle workers are available using fake build and execute processes (S5). This method is effective as it simulates the exact cache environment of autotuning in parallel for benchmark consistency throughout all stages of a generation.

In order to find an error for the median of the sample of benchmark data, a novel approach is used. This decision was made to avoid the inefficiency of bootstrap aggregation typically

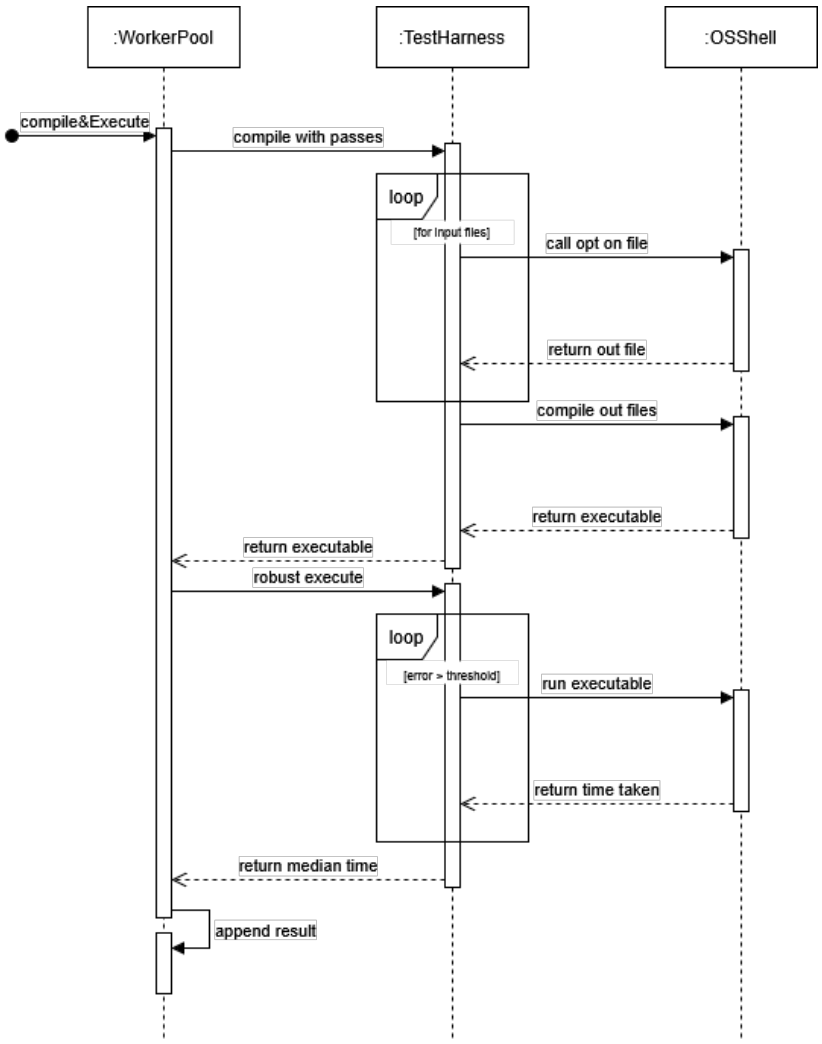


Figure 3.5: The three way interaction taking place for each worker job.

used to find the error for a metric when there isn't an effective alternative. This is done via a series of approximations that assume based upon our log-normally distributed data that the natural logarithm of our data obeys a normal distribution. From here a robust sample standard deviation is used, from which a standard error approximation is derived. See A.2 for the proof and further details.

Chapter 4

Implementation

This chapter provides the link between the design and specification of the project to its source code. It begins by outlining the structure of the implemented packages and key specifics of the source code, before laying out the third-party libraries used to simplify development. Further, the testing methodology alongside coverage details are summarised.

4.1 Language

When selecting a language for this implementation, Python and C++ were initially considered the primary candidates due to their widespread presence in the autotuning and compiler ecosystems respectively. Python is widely regarded as the industry standard for system interaction, heavily utilised by high-profile autotuning frameworks like CompilerGym [11]. Conversely, C++ is the native language of major compiler infrastructures, offering execution without any overhead and incredibly optimised performance. Further, both possess the key community support for the interaction necessary on a system level.

Despite their strengths, both languages presented critical flaws for the specific requirements of the project. Python's Global Interpreter Lock (GIL) [23] introduces significant roadblocks for achieving the noiseless, highly concurrent environment required for taking parallel benchmark measurements.

Similarly C++, while highly performant, does not use a native light threaded approach, instead needing to start a thread at the OS level to utilise multithreading. Further its agility is severely impacted by requirements for memory safety and error handling; compiling and executing fuzzed passes is inherently unpredictable, and a single unhandled error in C++ can

easily crash the entire autotuner and lose program state.

Consequently, Go (Golang) was selected as the optimal candidate. Go bridges the gap between Python's developer support and C++'s compiled advantages, specifically addressing the project's needs:

- **Efficient Parallelism**: The powerful and efficient approach Go takes to parallelisation. This provides the smallest possible overhead for the noiseless concurrency environment needed for taking benchmark measurements in parallel [53]. Natively there is support for *goroutines* within the language; these are called using the `go` keyword.
- **Safe Error Handling**: Go's explicit error handling guarantees program state at any time. Unlike C++, errors due to fuzzed passes and unexpected runtime behaviours are easily caught, preventing catastrophic panics from crashing the autotuner as a whole.
- **Strict enforcement of good programming practice**: The Go compiler enforces code hygiene by refusing to compile programs with unused variables or imports. On top of this, widely used tools like *gofmt* are available to format the code consistently across the codebase [19]. This ensures a maintainable and consistent codebase without requiring reading extensive language specification documents [19] (N1).

4.2 Package structure

Here we describe each package's structure and purpose within the project. Alongside this we make clear the interface exposed by each package and where they are needed.

4.2.1 Genetic Algorithm

The core GA package. This implements the core functionality you would expect from a GA. This includes population initialisation, a generation stepper and convergence evaluator, alongside mutation, crossover and selection operators.

Population initialisation

CreatePopulation resolves setup of the structure of the GA. It initialises the pass manager, worker pool, and parses the input command. Following this it generates the population members semi-randomly.

First the known good pass sequence, -O3, is added. This was found via the command

```
clang++ -mllvm -debug-pass=Arguments
```

Then, any past results available in a file are loaded in, up until a capacity scaled by `LOADED_POP` (U7). These are loaded randomly. Following this the rest of the population is generated. Uniquely these are generated by taking the O3 sequence and then performing mutations n times, where n is the length of O3. This results in a population that meets a baseline efficiency and accelerates surpassing O3.

```
func CreatePopulation(minPopSize, maxLength int, command string)
    workers := CPU.NumThreads() / 2
    population_size := (minPopSize / workers + 1) * workers

    pop := NewPopulation()
    pop.members = [][]int
    pop.pass_manager = GetPassManager()
    pop.maxLength, pop.lastSafePop = maxLength, []int{}
    pop.members[0] = pop.pass_manager.GetO3()
    pop.members[0] = clip(pop.members[0], maxLength)

    loadedMembers, _ := LoadSequences(size*LOADED_POP)
    for i := range len(pop.members) - 1 {
        if len(loadedMembers) > 0 {
            pop.members[i+1] = pop_front(loadedMembers)
        } else {
            pop.members[i+1] = generateRandomMember()
        }

    }

    commandSplit, err := shlex.Split(command)
    if err != nil {
        return nil, error("command lexing failed: " + err.Error())
    }

    pop.workers, err = workerpool.NewWorkerPool(
        commandSplit, workers, population_size, pop.members[0]
    )
    if err != nil {
        return nil, err
    }

    return pop, nil
```

Figure 4.1: Pseudo code for the population initialisation functionality.

The number of workers initialised in based on the thread count of the host CPU. As shown in Figure 4.1 specifically half of the threads are used. This decision was to balance the temperature of the CPU and cache availability with the number of generations the autotuner can process. Utilising all available CPU cores in fact tended towards providing no speed-up or even causing slowdown; as tends to be the case with heavy parallel computing [27].

```
func generateRandomMember()
    returnSlice := GetO3()
    returnSlice = returnSlice[:min(len(returnSlice), p.maxLength)]

    for range randint(len(returnSlice)) + 1 {
        returnSlice = mutate(returnSlice, p.passMan.MaxPass, p.maxLength)
    }

    return returnSlice
```

Figure 4.2: Pseudo code for creation of a semi-random individual.

Figure 4.1 also shows how population size is decided. This is by rounding the parameter passed in to the next multiple of the number of workers. The logic here is that assuming each workers job takes approximately the same time no extra time is necessary to perform a multiple of workers jobs.

Generation counter

```
func RunGenerationsForDuration(d time.Duration)
    startTime := time.Now()
    i := 0
    for time.Since(startTime) < d {
        minETA, maxETA := p.getMinMaxTimes(startTime, d, i)
        print("min. time: {minETA} : max. time: {maxETA}")

        converged := p.nextGeneration()

        if converged and i > 10 {
            return i + 1
        }
        i++
    }
    return i
```

Figure 4.3: Pseudo code for running the GA until it hits a maximum duration.

Figure 4.3 shows our core method for advancement of generations. This calls `nextGeneration` which handles fitness evaluation, selection, reproduction, and mutation. Following this `nextGeneration` returns a boolean indicating convergence. Throughout search, the minimum and maximum bounds for execution time are printed at the start of each generation; the bounds are calculated with the helper function `getMinMaxTimes` (U6).

There are two possible approaches to limiting runtime, a maximum duration or maximum generation count.

- Limiting by generations only has the advantage of being hardware independent, but pays for this by taking an unpredictable amount of time that may exceed real-world time constraints.

- Conversely, limiting by time is highly hardware dependent; it may converge on one system within the time limit, while ending early on another. However it has the benefit of being able to allocate a time budget based on how much of a performance improvement is desired.

The implementation goes with the approach of limiting by time, as this seems more convenient for a user who may not want to interact with low level details like generation count. Additionally this allows for reasonable default values. A default value for max. generations is not beneficial as it can not generalise well to sub second binaries and expensive binaries simultaneously.

Convergence evaluation

```
func testConvergence()
    safeMarker := SAFE_POP * len(p.members)
    safeMarker = max(safeMarker-1, 0)
    currentSafePop := p.members[safeMarker]

    if p.lastSafePop == currentSafePop {
        p.convergedCount += 1
    } else {
        p.convergedCount = 0
    }
    p.lastSafePop = currentSafePop

    return p.convergedCount >= 10
```

Figure 4.4: Pseudo code for evaluating the GA's stop condition.

Deciding convergence has many separate approaches. To attempt to generalise well to the population Figure 4.4 evaluates only the weakest elite member. If the weakest member remains the same over 10 generations we assume convergence. This has the benefit of ensuring if there is any improvement over elite individuals convergence is delayed. Improvement over elites is indicative that the local search space has not yet been fully exploited.

Alternative approaches include comparing the average of the whole population, the most elite member, or the average of the elites.

Comparing the whole population is a poor decision in the search space of LLVM pass sequences as not every pass sequence is valid. If the global optimum we are approaching is in an area of the search space with many invalid sequences, convergence could happen early as the population as a whole may get less healthy.

The most elite member causes issues with the specific approach we use of initialising our popu-

lation with -O3; the elites take a number of generations to find good competition to O3 which may cause early convergence.

The average of the elites is an alternative to the approach we used. It also tests whether the elites have improved as a whole, albeit less computationally efficiently.

Selection

```
func elitism(results []workerpool.Result, marker int)
    reproMarker := -1
    for i := range marker {
        if results[i].Score < MAX_FLOAT64 {
            p.members = append(p.members, results[i].PassSeq)
            reproMarker = i
        }
    }

    return reproMarker
```

Figure 4.5: Pseudo code implementing naive selection.

The GA implements a naive elitism approach to selection. It simply chooses the top performing members of the population, the size of which is dependent on population size and the hyperparameter *SAFE_POP*, set to a default of 0.25. This preserves 25% of our population from one generation to next.

Due to the unique difficulty of having a population composed of sequences of passes that may be invalid, we have to be flexible with what proportion of our population is saved. If some of the elites are invalid, then they shouldn't be involved in reproduction or carried over. This helps to prevent too much of the population becoming invalid, saving space for valid population members. The issue of invalid sequences is difficult to get around, as offspring of two valid sequences can still be invalid as a result of the complexities within the LLVM backend.

Crossover

Figure 4.6 shows our GA's abstracted reproduction function. It simply selects two, not necessarily different, pass sequences, and performs crossover between the two.

Contrastingly, Figure 4.7 implements the specifics of the reproduce function our reproduction function uses as an abstraction. It performs the safest form of reproduction between the two genomes, where safest is that which will not create offspring that are clones of the parents and will not cause errors.

The crossover implemented prioritises crossover that preserves the ordering of our pass

```
func reproduction(marker int)
    for len(p.members) < cap(p.members) {
        member1 := p.members[randint(marker + 1)]
        member2 := p.members[randint(marker + 1)]

        newSeq1, newSeq2 := reproduce(
            member1, member2,
            p.passMan.MaxPass,
            p.maxLength,
        )
        p.members = append(p.members, newSeq1)
        if len(p.members) < cap(p.members) {
            p.members = append(p.members, newSeq2)
        }
    }
```

Figure 4.6: Pseudo code implementing abstract reproduction.

```
func reproduce(a, b []int, maxVal, maxLen int)
    var newA, newB []int
    randNum := randint() % 5

    if len(a) == 0 or len(b) == 0 {
        newA, newB = pairMutate(a, b, maxVal, maxLen)
    } else if randNum < 3 and a != b {
        newA, newB = lcsCrossover(a, b)
    } else if randNum == 3 || a == b {
        newA, newB = twoPointCrossover(a, b)
    } else {
        newA, newB = pairMutate(a, b, maxVal, maxLen)
    }

    return newA, newB
```

Figure 4.7: Pseudo code managing precise safe reproduction.

sequences. Specifically this is implemented as longest common subsequence (LCS) crossover. This iterates the two sequences and locates the longest sequence of non-contiguous shared passes. Following this it performs uniform crossover of the genes between the LCS sequences, where the segments crossing over are the groups of genes differing, as shown in Figure 4.8.

As necessary for the scope of LLVM pass sequences, this allows for dynamic lengths on inputs and outputs without further modification necessary.

Mutation

Similarly to crossover, we implement an abstracted mutation function, shown in Figure 4.9. This allows us to change the implementation details of the called mutate function without breaking the functionality of our GA. This selects a random population member outside of our elites and alters its genome to locally navigate the search space.

Figure 4.10 illustrates our specific implementation for mutation. There are three ways in which we mutate our pass sequences: replacement, insertion and removal. In order to maintain

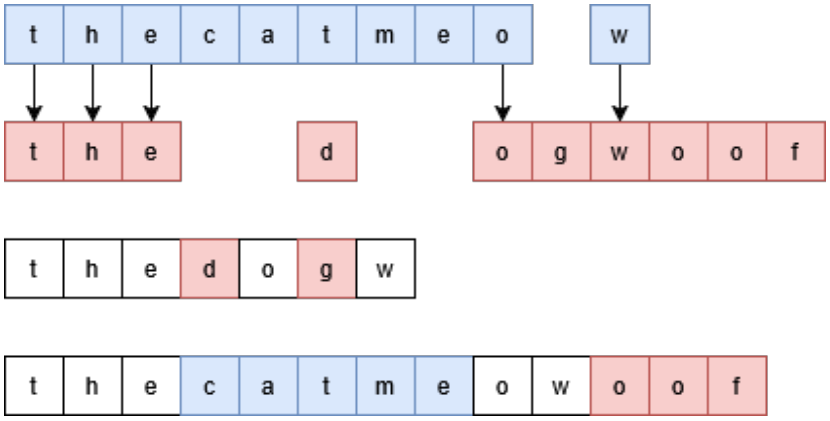


Figure 4.8: Example longest common subsequence crossover.

```
func mutation(marker int)
    steps := int(float64(len(p.members)) * MUTATION_STEPS)
    for range steps {
        popPtr := rand.Intn(len(p.members)-marker-1) + marker + 1
        p.members[popPtr] = mutate(p.members[popPtr], p.passMan.MaxPass, p.maxLength)
    }
```

Figure 4.9: Pseudo code implementing abstract mutation.

that our sequences stay in line with our requirements, the algorithm does not insert into a sequence that is already at maximum size; the algorithm does not remove from sequences that have zero length. This is the most standard implementation for mutation in a genetic algorithm, adapted to allow sequences to grow and shrink in length.

Our mutation implementation is critical to the GA's success, it allows it to navigate the search space locally from already good known solutions. This is the key component of fine-tuning our sequences once we converge towards an optimum.

Upon convergence, the main function will again call into the genetic algorithm to build the final product and return the well formed pass sequence. As a side-effect the genetic algorithm will save the discovered sequence to a file (U7, S3). These sequences are the reloaded in future runs as shown in figure 4.1. This allows the GA to exploit past results, thus saving time and power as well as improving the average result quality.

4.2.2 Test Harness

The test harness' main functionalities can all be contained within only two functions, as shown in 4.11 and 4.12.

The test harness can sometimes get into situations wherein loop structure within the source code is broken. This can cause infinite loops, or loops that take unreasonably long times.

```
func mutate(a []int, maxVal, maxLen int)
    point := rand.Int() % len(a)

    randf := randfloat()
    if randf < 0.33 {
        newPass := a[point]
        for newPass == a[point] {
            newPass = randint(maxVal)
        }
        a[point] = newPass
    } else if randf > 0.66 && len(a) < maxLen {
        newPass := randint(maxVal)
        a = a[:point] + newPass + a[point:]

    } else {
        a = a[:point] + a[point+1:]
    }

    return a
```

Figure 4.10: Pseudocode for precise safe mutation.

```
func GetSetBaselineStats(compilationCommand []string, passes string, noiseFunc func(), workers int)
    trueHash, mem, err := getExpectedHash(compilationCommand, passes)
    if err != nil {
        return -time.Second, 0, fmt.Errorf("failed basic execution:\n%v", err)
    }
    setTrueHash(trueHash)

    done := byte channel

    makeCompilerBackgroundNoise(workers-1, done, noiseFunc)

    o3Time, err := getExpectedExecutionStats(compilationCommand, passes)
    close(done)
    if err != nil {
        return -time.Second, 0, fmt.Errorf("failed basic execution:\n%v", err)
    }

    return o3Time, mem, nil
```

Figure 4.11: Pseudocode whose only functionality is to get the baseline stats for the baseline population member. In this case -O3.

To combat this the harness needs some estimate of execution time. Figure 4.11 accomplishes this by compiling and executing once to get a valid hash for the output of the file, which avoids excessive dead code elimination (DCE) that may break functionality; the function then approximates an execution time by running the compiled file until the execution time is stable. Having gained the hash and approximate execution time, the genetic algorithm can proceed with the hash to cross check binary output, and the baseline execution time with which to adaptively cap execution time.

Figure 4.12 takes the passes we are compiling with, an identifying integer, whether to output the new file locally, and the source command. It performs the main functionality of running the command, managing file deletion, and getting and setting the approximate statistics for the

```
func CompileAndExecute(passes string, diff int, outLocal bool, SourceCmd []string)
    compilation_t, binary_size, newFile, err := compileFileWithPasses(
        SourceCmd,
        passes,
        diff, outLocal,
    )
    if err != nil {
        return MAX_DURATION, MAX_DURATION, 0, 0, err
    }
    defer Remove(newFile)

    execution_t, maxrss, err := runProgramUntilStable(newFile)
    if err != nil {
        return MAX_DURATION, MAX_DURATION, 0, 0, err
    }
    newFile.Close()

    return compilation_t, time.Duration(execution_t), binary_size, maxrss, nil
```

Figure 4.12: Pseudocode for the driver of the test harness.

compiled binary. The stats are set globally within the test harness for use by other functions.

These two combined implement all external functionality required by the GA. Internally there are a number more key functions that support our external interface.

```
var hash uint64 = calculateHash(
    stderr.Bytes(), stdout.Bytes(),
)

func calculateHash(streams ...[]byte)
    hash := xxh3.Write(streams)

    return hash
```

Figure 4.13: Pseudocode for the hash calculation.

In Figure 4.13 is pseudo code for our simple hashing function alongside the use case that ties it into the project. When choosing our hash function there were a number to choose from. A cryptographic hash is not necessary for this use case; hashing outputs to check for consistency requires only a low collision rate and high throughput. Any hash function with these qualities would be a satisfactory fit.

XXH3 was chosen due both having one of the lowest collision rates and highest throughputs of any non-cryptographic hash [8], whilst simultaneously possessing a fast and universal Go implementation; discussed further in subsection 4.3.3.

Since working with the median as our average metric is intended to keep the measurement robust, a robust estimation for error is needed. This robust estimation (RSE) is outlined in

```
func estimateMedianRSE(list []any)
    if len(list) == 0 {
        return 0
    }
    ratio := 1 / 1.35
    fmMedianSqrt := Sqrt(Pi / 2)
    ratioDotFmMedianSqrt := ratio * fmMedianSqrt


    logList := ForEach(list, func(i any) {
        return math.Log(i)
    })

    iqr := IQR(logList)
    sqrtN := Sqrt(len(logList))

    return (ratioDotFmMedianSqrt * iqr) / sqrtN
```

Figure 4.14: Pseudocode for an approximate RSE.

Figure 4.14, the proof for which is outlined in A.2. Here Robustness to noise is maintained by utilising IQR approximation for the standard deviation.

```
var runProgramUntilStable = func(toRun *os.File)
    var times []int64
    var rsss []int64


    for begin := time.Now(); i := 0; (i < 5 ||
        !MedianConfidence(times, 0.03)) &&
        time.Since(begin) < 12*approxTime; {

        execution_t, rss, hash, err := runFile(toRun, approxTime*3)
        if err != nil {
            return MAX_FLOAT64, MAX_INT64, error("execution failed: %v", err)
        } else if hash != trueHash {
            return MAX_FLOAT64, MAX_INT64, error("incorrect hash")
        }

        times = append(times, int64(execution_t))
        rsss = append(rsss, rss)
        i++
    }

    median, confidence := Median(times), estimateMedianRSE(times)
    errorScaledMedian := median + (median * confidence)
    approxTime = min(time.Duration(errorScaledMedian), approxTime)

    return errorScaledMedian, max(rsss), nil
```

Figure 4.15: Main driver pseudocode to get an approximate measure for execution time of an individual executable.

Figure 4.15, the test harness' main function, is outlined in pseudocode. This uses or is used by much of the rest of the functionality the test harness provides. Specifically, this function is designed to take an executable file reference and return an upper confidence bound (UCB) for the median execution time and maximum resident set size of the program. In failure cases, where the program returns an error or the hash for the output does not match, an error will

be returned to be handled by the caller.

The number of times a program will be run depends on a variety of conditions.

1. A minimum of 5 executions allow for getting a reasonable approximation of how the program's runs are distributed.

2. Following these five runs the median time can be returned at any time provided that the relative squared error of the distribution is below 3%.

3. Execution of the loop is halted if the time spent running exceeds $12 \times optimised_time$. This is to avoid execution for any single sequence from disrupting program flow in the event that the error takes too long to fall, at the cost of allowing some error through. This is managed by returning an UCB for the median.

The UCB is calculated by utilising the RSE in conjunction with the median. This UCB is used to update the approxTime variable if it is seen to be lower. Then the UCB for the median is returned alongisde the maximum resident set size.

This practice of updating the time estimate is known as adaptive capping and is discussed in section 2.4.1.

Sensitivity Analysis

The parameter managing maximum execution time in Figure 4.15 was carefully chosen according to the average number of runs required to reach a low error of <3%. Sensitivity analysis was then performed to verify that this was a plausible approach to allow for efficient execution of an ample number of generations alongside real progress towards an optimal solution.

The resulting data is shown in Figures A.1 and A.2. These allow a maximum execution loop time of $3 \times optimised_time$ and $24 \times optimised_time$ respectively. As seen by the graphed improvements over time relative to our results obtained in Figure 5.1, there are issues with using a far higher or far lower parameter compared to the baseline.

Lower: the algorithm fails to successfully identify which solutions perform better without a huge margin of error. This allows poor solutions to enter the reproduction pool, resulting in repeated performance degradations and eventually in the algorithm regressing to a point in the solution space worse than O3.

Higher: the solutions steadily improve over the generations. This aligns with expectations, getting a more accurate measurement of execution time allows for improvements to be noted and exploited. The flaw however is the vastly poorer performance within the same time budget.

Due to increasing time taken by the GA to evaluate each population member, less of the search space is seen and exploited. This is inefficient as it reduces the improvement found per unit time/power and as such causes the GA to be less practical for use.

4.2.3 Pass Manager

All passes used within the pass manager are directly extracted from the LLVM pass registry or using the command outlined above in subsection 4.2.1. The pass manager implements the singleton design pattern. The global instance is initialised and accessed via the GetPassManager method.

```
func MakeValid(seq []int) string
    FUNCTEXT, CGSCCTEXT, LOOPTEXT := (
        "function<eager-inv>(","cgscc(devirt<4>(","function<eager-inv>(loop("
    )
    var buffer, next bytesBuffer, string
    currentScope := INMODULE
    if len(seq) == 0 { return ""}

    firstVal := seq[0]
    if firstVal in cgsccPasses {
        buffer.Write(CGSCCTEXT)
        currentScope = INCGSCC
    } else if firstVal in functionPasses {
        buffer.Write(FUNCTEXT)
        currentScope = INFUNC
    } else if firstVal in loopPasses {
        buffer.Write(LOOPTEXT)
        currentScope = INLOOP
    }
    buffer.Write(pm.IntToPass(firstVal))

    for val in seq[1:] {
        if val in cgsccPasses {
            next, currentScope = pm.nextAppend(val, currentScope, INMODULE, INCGSCC, CGSCCTEXT, getClosing)
        } else if val in functionPasses {
            next, currentScope = pm.nextAppend(val, currentScope, INMODULE, INFUNC, FUNCTEXT, getClosing)
        } else if val in loopPasses {
            next, currentScope = pm.nextAppend(val, currentScope, INMODULE, INLOOP, LOOPTEXT, getClosing)
        } else {
            next, currentScope = pm.nextAppend(val, currentScope, INMODULE, INMODULE, "", getClosing)
        }
        buffer.Write(next)
    }
    if currentScope != INMODULE { buffer.Write(getClosing(currentScope)) }
    return buffer.AsString()
```

Figure 4.16: Pseudocode to make an integer sequence a valid sequence of passes.

The pseudocode defined in Figure 4.16 relies on nextAppend and getClosing functions. Both of these behave as you would expect; they get the next valid piece of the sequence and format it as LLVM expects pass sequences to be arranged.

This implementation is the core functionality of the pass manager. While being very fast and functional, it has some limitations:

- It considers dependencies no further than the pass type.
- It does not allow for the nesting of pass types; this is a key part of the O3 pass sequence.

The first issue is a relative non-problem as the GA can simply discard invalid sequences without issue. The second issue is resolved by including the large nested structure in -O3 as a singular pass.

These are certainly imperfect solutions, however they allow the GA to function.

4.2.4 Worker Pool

The worker pool implements the core parallelism of our project.

```
func NewWorkerPool(sourceCmd []string, workers, numJobs int, baseline []int)
    pool := new(WorkerPool)

    pool.passSeq = [][]int
    pool.fileNum = int channel
    pool.SourceCmd = sourceCmd
    pool.passMan = passmanager.GetPassManager()
    pool.cache.Init(256)

    for i := 0; i < workers; i++ {
        pool.fileNum <- i
    }

    validO3 := pool.passMan.MakeValid(baseline)
    o3Time, _, err := testharness.GetSetBaselineStats(sourceCmd, validO3,
        func() {
            fileNum := <-pool.fileNum
            defer func() { pool.fileNum <- fileNum }()
            pool.runJob([]int{}, fileNum, true)
        }, workers)
    if err != nil {
        return nil, err
    }

    pool.cache.Set(validO3, valueFromJobStats(0, o3Time, 0, 0))

    ProgressBar.initialise()
    return pool, nil
```

Figure 4.17: Pseudocode outlining worker pool initialisation.

The pseudocode shown in Figure 4.17 initialises the workers and necessary functionality. Crucially the pool initialises the baseline (O3) execution time and adds it to a cache; the cache is implemented as an LRU cache, discussed in subsection 4.2.5. The outline of the noise generating function as seen in Figure 4.11 is shown here. This acts to run the compiler with a completely empty pass sequence, providing a consistency of noise for the baseline so it does not perform better than expected solely due to running in isolation.

Additionally Figure 4.17 shows our initialisation of the progress bar, that guides the user with

an interactive wheel showing execution time; progress bar capabilities are further defined in subsection 4.3.2

```
func StartPool()
    var wg WaitGroup
    p.Results = make(Result channel, len(p.passSeq))

    for seq in p.passSeq {
        fileNum := <-p.fileNum
        wg.Go(func() { p.runJob(seq, fileNum, false) })
    }

    p.manageCPULoad(&wg)
    wg.Wait()

    close(p.Results)
    return ArrayFromChannel(p.Results)

func manageCPULoad(wg *WaitGroup)
    while len(p.Results) != cap(p.Results) {
        fileNum := <-p.fileNum
        wg.Go(func() {
            defer func() { p.fileNum <- fileNum }()
            p.runJob([]int{}, fileNum, true)
        })
    }
```

Figure 4.18: Pseudocode for worker pool running.

Figure 4.18 showcases parallelism alongside its usage of channels for safe data communication. The initialisation of the worker pool with a fixed number of elements in fileNum allows

```
fileNum := <-p.fileNum
```

to act as a blocking line. This key behaviour is what allows our parallel processing to function correctly. Provided all thread creations require to checkout a file number before any heavy processing we minimise noise to only the expected level. This behaviour closely mirrors that of a *semaphore* synchronisation primitive.

Similarly to Figure 4.17 we utilise a noise function. Here it is implemented as a separate function *manageCPULoad*. This begins running only once all jobs are scheduled and ensures a consistent amount of noise for the remaining jobs until the results are filled.

In Figure 4.19 is outlined the core individual job our worker pool is designed to run. To note is the use of the cache to reduce runtime by avoiding re-evaluation of recently seen sequences. Additionally the cache is avoided when running a *dummy* job ran to maintain noise levels. This ensures dummy jobs provide a realistic simulation of parallel compilation and execution tasks without early stopping due to cache presence. Crucially, by writing to the cache prior to error handling the function avoids failure to cache error results.

```
func runJob(seq []int, diff int, dummy bool)
    if !dummy {
        defer ProgressBar.Increment()
        defer func() { p.fileNum <- diff }()
    }

    validSeq := p.passMan.MakeValid(seq)
    if score, ok := p.cache.Get(validSeq); ok and not dummy {
        p.Results <- Result{score, seq}
        return
    }

    compilation_t, execution_t, binary_size, maxrss, err := (
        testharness.CompileAndExecute(validSeq, diff, false, p.SourceCmd)
    )
    if dummy { return }
    finalScore := valueFromJobStats(compilation_t, time.Duration(execution_t), binary_size, maxrss)
    defer p.cache.Set(validSeq, finalScore)
    if err != nil { p.queueErrorResult(seq) }
    else {
        p.Results <- Result{
            Score:   finalScore,
            PassSeq: seq,
        }
    }
```

Figure 4.19: Pseudocode for parallel running of one compilation execution cycle.

The function *queueErrorResult* adds a result representative of an error. In the GAs case this is a *MAX_FLOAT* value, representing a maximum possible execution time. These results are checked for in 4.5, preventing invalid sequences from polluting the gene pool of the population.

4.2.5 Utilities

The utilities package provides a wide range of functionality necessitated by the rest of the project. The interface includes: *ArrayFromChannel, Median, IQR, ForEach*, and various functionality to implement necessary wrappers for the pass types. It provides the structures (types): *Cache* and *BidirectionalMap*.

To avoid the issue of system thermal throttling effecting the outcome of our GA, we implement an LRU cache. Specifically a variant of an LRU that ignores access recency. In practice, this discards the oldest recorded result when full. Over time this allows the original best sequences, recorded when thermal constraints were low, to be adjusted as necessary as host temperature rises.

The Bidirectional map structure is naively implemented as two generic mappings and access functions. This provides functionality to the pass manager, allowing it to first convert individual passes to integer representations and later recall them via these representations.

4.3 Third-party libraries

4.3.1 Shlex

shlex [26] is an open-source go library implemented by google. It lexes an input string exactly the way that the shell would. The project utilises this to lex command inputs without having to maintain this isolated functionality alongside the project. Implemented correctly, we are guaranteed to be able to parse any given clang/++ command (N2). This is necessary in order to execute them later via Go's os.exec function.

```
clang++ -I"./example/hello world/utilities" "./example/hello world/helloworld.cpp"
```

is predictably parsed as: `clang++`, `-I./example/hello world/utilities`, and `./example/hello world/helloworld.cpp`.

4.3.2 Progress Bar

progressbar [59] is an open-source Go library that provides a number of solutions to illustrating to an end-user the ongoing execution of a process. This is crucial for good UX and readability given the command-line environment. Further it allows our user to have an idea of the progress of the autotuner and ensures it is clear that it has not crashed or deadlocked even in the event of delays (U6, S6).

4.3.3 XXH3

xxh3 [57] is the open-source Go implementation of the XXH3 algorithm [8]. This is a high-speed, non-cryptographic hash function. Extreme throughput is achieved by leveraging SIMD (single-instruction, multiple data) vectorisation to process large data blocks in parallel. It utilises heavily optimised routines for short inputs. As with other hash functions, the algorithm mixes data using multiplication, XOR, and bit-rotations against a pseudo random array to ensure strong dispersion and minimal collisions.

4.4 Testing

To ensure the reliability and correctness of the system, unit testing was conducted across the codebase. The primary objective of testing was to logically verify the core components of the GA and its supporting data structures, ensuring that the search space was navigated correctly without introducing regressions.

4.4.1 Test methodology

The Go idiomatic table-driven approach was taken to testing [25]. This allows for homogeneity within the codebase relative to code within the community and consequentially for easier maintenance as a result of standardisation.

```
func TestGet(t *testing.T)
    tests := []struct {
        name string
        hit  bool
    }{
        {name: "cache hit", hit: true},
        {name: "cache miss", hit: false},
    }

    for test in tests {
        t.Run(test.name, func(t *testing.T) {
            cache := Cache[int, int]{}
            cache.Init(1)

            if test.hit {
                cache.Set(1, 1)
            }
            value, ok := cache.Get(1)

            if ok != test.hit {
                t.error("expected ok to be %v", test.hit)
            }
            if ok and value != 1 {
                t.error("expected value to be 1, got %d", value)
            }
        })
    }
```

Figure 4.20: Pseudocode for testing cache behaviour with table-driven methodology.

Table-driven testing is a methodology that defines each test case as a data type, in Go's case a struct [25] as shown in Figure 4.20. This allows for testing all cases of a target function with a single corresponding test function. This generalisation allows for the boilerplate testing code to be massively reduced even for complex functionality; the figure illustrates this with how multiple test cases can be tested with a singular code block in a loop. The key benefits include:

- despite an increase to code volume in simple cases, when you have several tests to execute with largely similar logic a decrease in code length is seen as a result of generalised code replacing repeated code.
- Once the boilerplate struct and execution logic are laid out, execution can be scaled to

as many test cases as necessary with simple entries into the struct.

- Developers who read these tests in the future only have to understand the core business logic once. Reading through the test cases then concisely informs them of what has been tested. This contrasts general tests where each test case may be separated into unique functions.

4.4.2 Comprehensive tests

Overall, the project achieved a total code coverage of 83.8%. This high level of coverage was attained by prioritising the algorithmic and data management functionality over the highly volatile orchestration layers between the autotuner and the rest of the computer system.

The most critical mathematical and logical components were prioritised for testing. This was to place the focus heavily on areas where algorithmic faults would directly compromise the validity of the autotuner's output:

- Genetic Operators: The logical backbone of the GA, the mutation and crossover operations, have 100% coverage. This guarantees that the generation of LLVM pass sequences functions deterministically and safely while correctly exploiting the information available about the search space.

- Test Harness and Caching: The test harness and caching mechanisms have 100% coverage. Verifying the LRU cache was vital to ensure the system does not waste computational resources re-evaluating known sequences while ensuring it correctly re-runs sequences after the passage of time may have changed background conditions.

- Data Structures and Statistics: The statistical verification and pass manager alongside its data structure have 92.7% coverage on average. This ensures that the mapping of string-based LLVM passes to integer representations is flawless, and that the error calculations correctly allow for identifying stable execution times.

4.4.3 Uncovered Components

While the mathematical core of the autotuner is thoroughly verified, certain architectural components maintain lower coverage metrics due to the inherent complexities of testing concurrent, system-level execution.

- System Orchestrators: Unit testing the portions of the system managing the heavy concurrency of the system (goroutines and channels) and the direct access to the host operating system to execute Clang/LLVM commands often requires extensive mocking of the OS, file system, and external binaries. This tends to produce brittle tests that verify the mock rather than the system, as well as more boilerplate code to allow for mocking the many intricate components. Verification was instead primarily verified through end-to-end integration runs rather than isolated unit tests.

- Entry Point: The main execution wrapper acts purely as a command-line interface (CLI) parser and entry point. Because it contains no business logic and only delegates user input to the core of the system, writing unit tests for this file was deemed to offer diminishing returns when this functional is deeply verified by the rest of the codebase.

Chapter 5

Results & Evaluation

The following chapter outlines the performance and behaviours of our GA and its results, particularly in comparison to standard O3.
Wherever final results are provided, they were gathered post autotuning via the following standardised command.

```
hyperfine -w 20 -m 80 ./ga.out ./o3.out
```

`ga.out` is the file produced by our genetic algorithm converging or reaching the time limit. `o3.out` is the file produced by running the original command passed into the GA, but including the -O3 compiler flag.

Performance of the autotuner was recorded on a Lenovo V15 G3 ABA with an AMD Ryzen™ 7 5825U CPU @ 2.0GHz with a TDP of 15W. The system was equipped with 16GB of DDR4-3200MHz memory and running Windows. Within this WSL2 (Windows Subsystem for Linux) was utilised via Docker. The version of LLVM used was the most modern available at the time of starting the development of the autotuner, 21.1.2. Each run of the autotuner proceeded to convergence or to a maximum of 4 hours of runtime with a population size of 32 and fixed max pass sequence length of 200.

5.1 Performance Testing

5.1.1 Benchmark choices

Evaluating the performance of our genetic algorithm requires as diverse a set of programs as possible for comprehensive testing. The PolyBench suite was used with strict program selection.

Each benchmark represents unique computations and was chosen specifically for the stress they place on the compiler's optimisation capabilities:

- **Medley (`nussinov`):** An RNA secondary structure prediction algorithm utilising dynamic programming. It acts as a stress test for compiler optimisation due to its complex, non-uniform data dependencies and unusual affine loop bounds. Unlike the standard rectangular loops found in linear algebra, affine loop bounds are notoriously difficult to automatically parallelise or tile [6]. This is thereby useful for proving the system's ability to handle these irregular loop structures.

- **Stencils (`heat-3d`):** A 3D heat equation solver. Stencil computations are inherently memory-bound, meaning performance is dictated by memory timings rather than CPU throughput [12]. By extending the stencil access pattern into three dimensions over multiple time steps, `heat-3d` exceeds modern cache capacities. This forces testing of the compiler's ability to perform advanced spatial and temporal cache blocking to yield performance improvements.

- **BLAS (`gemm`):** General Matrix Multiply ($C = \alpha AB + \beta C$). As the core of dense linear algebra, it scales at $\mathcal{O}(N^3)$ computational complexity but only $\mathcal{O}(N^2)$ memory footprint [44]. It serves as a standard for measuring peak floating-point performance and cache blocking efficiency [38].

- **Kernels (`3mm`):** Three sequential matrix multiplications ($E = A \times B$, $F = C \times D$, and $G = E \times F$). By chaining three heavy computations together, this benchmark creates complex data dependencies. It evaluates the compiler's ability to effectively pipeline or fuse loops.

- **Solvers (`ludcmp`):** LU Decomposition and Equation Solve. To solve a linear system $Ax = b$, this benchmark first factors the matrix into lower and upper triangular components, then performs forward and backward substitution. By encompassing the work of multiple stand-alone benchmarks, `ludcmp` evaluates dense matrix operations and theoretical peak computational capability.

Additionally, all benchmarks from the datamining domain, specifically `correlation` and `covariance`, were included.

These selections ensure a diverse baseline across different algorithmic patterns.

5.1.2 Results

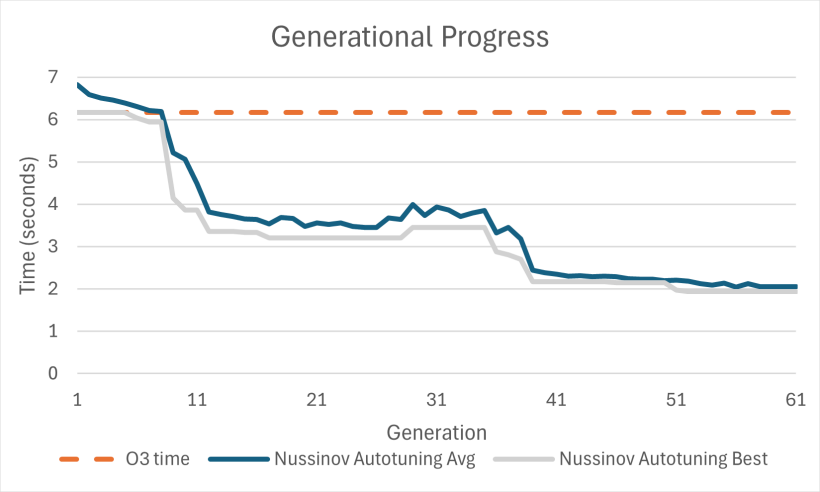


Figure 5.1: Line graph showing the results of the average top 10 nussinov population member per generation alongside the best member per generation.

In Figure 5.1, the progression of the elites from a population of pass sequences is shown until convergence. As seen, the population is initially seeded with the sequence of passes executed in O3. This leads to an initial plateau as the population explores the immediate fitness neighbourhood. Following this, significant improvements are found around generation 8-13 where a sharp drop in execution time is seen.

It is worth noting the minor regression in execution time seen around generation 26-37. This is an expected side effect of the implementation; as a result of caching, results are not re-evaluated every generation. When re-evaluated, environmental noise may have changed leading to a different evaluation. This should have little effect on results as the population's fitness remains the same relative to one another.

A final major optimisation is found around generation 38. The population has at this point settled into a local optimum and finds little further to exploit within the immediate environment via genetic operators. This leads to convergence at generation 61 which can be observed by the difference between the best and average fitness narrowing.

Nussinov sequence analysis

The data representing the pass sequence found by the GA compared against the standard O3 pass sequence is summarised in appendix section A.4. Unlike the generalised O3 sequence, which is heavily structured around the Call Graph Strongly Connected Components (CGSCC) pass manager, the GA sequence is incredibly specialised. It discards much of the standard pipeline's

structure in favour of aggressive loop transformations targeted directly at the benchmark's bottlenecks.

The most prominent deviation from the default O3 configuration is the GA's early invocation of `loop-interchange`. By applying this early in the pipeline, the GA sequence alters the traversal order of the multi-dimensional arrays. This vastly improves spatial cache locality and exposes the inner affine loops to more efficient auto-vectorisation later in the compilation process. Standard O3 typically struggles to restructure these complex dependencies automatically.

The autotuned sequence further introduces passes to mitigate the overhead of irregular loop bounds, notably Inductive Range Check Elimination (`irce`) and `loop-data-prefetch`. Because nussinov features affine loop bounds, `irce` becomes highly effective at stripping away redundant bounds checks. Concurrently, the inclusion of `loop-data-prefetch` helps mask memory latency by proactively loading data into the cache.

Equally, the GA omitted and condensed vast swathes of the O3 pipeline. The nested and repeated applications of `simplifycfg` and `instcombine` alongside inlining strategies are largely bypassed. The GA instead relies on the execution of scalar optimisations such as `sccp`, `loop-reduce`, and `aggressive-instcombine`. This flattened structure likely prevents the compiler from prematurely simplifying or obfuscating the control flow before vectorising and memory optimisation passes can act upon the restructured loop nests.

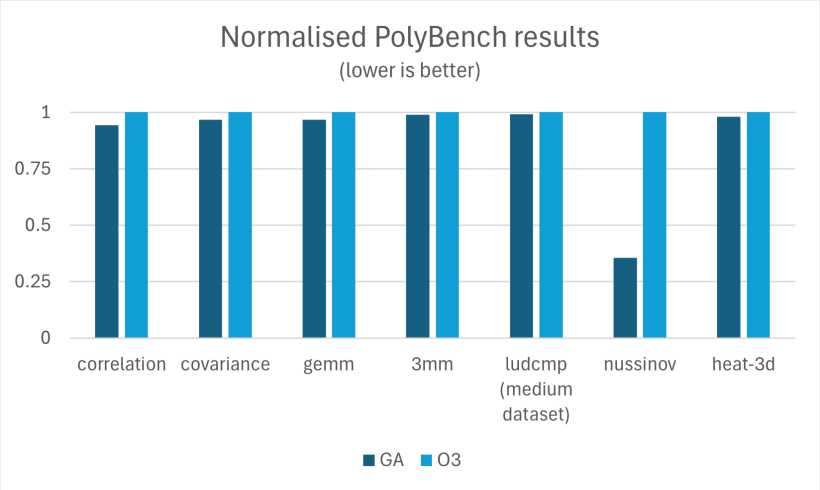


Figure 5.2: Normalised results for execution time of listed PolyBench files.

Table 5.1: Displays the raw results of benchmarking optimised PolyBench binaries. Standard error is written as *stderr*.

File	GA mean	O3 mean	GA stderr	O3 stderr	% improvement
correlation	4.22	4.47	0.030	0.010	5.59
covariance	4.29	4.43	0.012	0.0047	3.16
gemm	0.29	0.30	0.0018	0.0022	3.33
3mm	2.18	2.20	0.0044	0.0045	0.91
ludcmp	0.0527	0.0531	0.00010	0.00010	0.75
nussinov	1.034	2.907	0.0030	0.0072	64.4
heat-3d	2.173	2.214	0.0060	0.0058	1.85
average	n/a	n/a	0.0034	0.0030	3.36

From Table 5.1 and Figure 5.2, trends are visible within the data. Most notably, the GA optimisation consistently outperforms standard O3 across all benchmarked programs. This culminates in an overall average performance increase of approximately 3%; the average was measured using geometric mean. This average contains a large outlier; as noted in 5.1.1, nussinov is challenging for generic optimisation levels due to the affine loops present. As a consequence, the specialised sequence found by the autotuner is able to beat O3 by 64% as a result of exploiting the specific optimisation passes missed by the standard approach.

The standard errors remain low for all measurements, and the GA's standard error was lower or equal to O3 in more than half the cases. This indicates that the GA is capable of discovering stable optimisations rather than just trading consistency for peak performance. Specific outliers, such as the covariance and correlation benchmarks, resulted in a marginally higher overall average standard error for the GA.

5.2 Significance

The significance of our results can be evaluated by utilising *Welch's t-test* [56]. This tests a null hypothesis for mean equality and is known for being more reliable than the more commonly used *Student's t-test* when variance between the two samples is unequal [14].

Table 5.2: Displays the significance statistics calculated by the aforementioned data. Standard error is written as *stderr*.

File	Δstderr	Δmean	T-value	Degrees of Freedom	P-value

correlation	0.0319	0.250	7.85	96.7	5.95E-12
covariance	0.0132	0.140	10.6	102	3.60E-18
gemm	0.00286	0.0100	3.49	151	0.000630
3mm	0.00625	0.0200	3.20	158	0.00165
ludcmp	0.000142	0.000400	2.81	158	0.00557
nussinov	0.00777	1.87	241	106	4.60E-147
heat-3d	0.00838	0.0410	4.89	158	2.45E-06

Table 5.2 summarises our intermediate values while conducting a two-tailed *Welch's t-test*, given to 3 significant figures. The method through which these are calculated is given in A.3. Each test was conducted with a null hypothesis, $H_0 : \mu_1 = \mu_2$, and an alternate hypothesis, $H_1 : \mu_1 \neq \mu_2$. **At a 95% significance level, the dataset demonstrates that all results were statistically significant** since for every binary, $p < 0.05$ (N6).

5.3 Comparisons to past results

Table 5.3: Compares the percent improvement across the PolyBench suite measured by various papers.

File	CompTuner [60]	BOCA [7]	OpenTuner [4, 60]	Proposed (GA)
correlation	4.2	8.2	n/a	5.59
covariance	2.5	7.9	2.5	3.16
gemm	8.8	n/a	n/a	3.33
3mm	6.6	10.1	7.2	0.91
ludcmp	7.2	n/a	8.7	0.75
nussinov	11.6	29.0	14.3	64.4
heat-3d	1.4	2.0	1.4	1.85

Table 5.3 contains the results achieved by applying various autotuning approaches to the tested PolyBench executables. All results are given as percentage improvement over -O3 using LLVM. Results gathered for CompTuner and OpenTuner were gathered by researchers on a system with an "Intel(R) Xeon(R) Gold 5218R CPU @ 2.10 GHz, 377.8G memory, and Ubuntu 16.04.6 LTS operating system [...] on [...] LLVM 15.0.0" [60]. Results gathered for BOCA

were gathered by researchers on a system with "LLVM 2.9 [...and...] a workstation with 16-core Intel(R) Xeon(R) CPU E5-2640 v3, 126G memory, and CentOS 6.10 operating system" [7].

5.3.1 LLVM version

These comparisons are difficult to quantify directly, due to the differing versions of LLVM used. The proposed GA was evaluated against LLVM 21.1.2, whereas BOCA utilised LLVM 2.9 [7] and CompTuner and OpenTuner utilised LLVM 15.0.0 [60].

Compiler optimisation heuristics in standard suites like -O3 have matured significantly over the last decade. This would allow autotuners evaluated on these older LLVM versions to exploit improvements relative to the baseline more easily. This makes the 64.4% improvement in nussinov particularly significant, as it suggests the GA discovered complex loop transformations that the modern LLVM, even with the years of manual tweaking, still fails to optimise effectively.

5.3.2 Architecture

The performance difference between the proposed GA and past research may also reflect hardware differences.

- This study utilised an AMD Ryzen™ 7 5825U, a mobile processor with a strict 15W TDP. In contrast, CompTuner and BOCA used Intel Xeon server-grade processors [7, 60].
- The architectures feature differing cache designs and quantities [2, 36, 35]. Optimisations such as `loop-data-prefetch` and `irce` likely interact differently with the Ryzen's buffers and cache hierarchy than the Xeon's.

The lower percentage improvements seen for the proposed GA's results in benchmarks like 3mm compared to the alternatives may suggest that on modern Zen 3 hardware [2], the default LLVM -O3 pipeline is already highly efficient at managing dense linear algebra kernels, or that the performance is bottlenecked by power limitations rather than instruction scheduling.

5.3.3 Environment

The use of WSL2 and Docker introduces a layer of virtualisation not present in the native Linux environments used in the reference studies. The overhead of system calls and memory management within a containerised environment can introduce minor jitter [16].

Despite these constraints, the GA's ability to consistently outperform -O3 across the suite validates the effectiveness of the pass sequences it generates. The massive gain in nussinov

relative to BOCA and CompTuner suggests that the GA's search space or mutation strategy is particularly well-suited for the irregular data dependencies found in dynamic programming algorithms.

Chapter 6

Conclusion and Future Work

The final chapter concludes the report by assessing what can be drawn from the research results after executing the GA across a range of binaries in section 6.1. Further, the limitations of the work are discussed in section 6.2 before potential future work in section 6.3.

6.1 Conclusion

Originally the report set out to navigate the search space of compiler flags by utilising AI optimisation techniques with the intention of finding improvement.
The research has conclusively shown that genetic approaches can be utilised to navigate the LLVM pass search space to find incremental improvements over the hand-crafted human measure, -O3. Linking together past literature, a number of approaches (LRU cache, robust error approximation A.2, using fewer cores) have been found to be successful for the purpose of decreasing runtime of GA optimised programs. This allows more optimisation to be conducted within a smaller time frame; potentially this allows use where previously the gains would not have been worth the GA runtime.

It was found that benchmark data needs careful consideration to aggregate into a singular metric that allows comparison between binaries. In practice this required use of the most robust metric accessible, median, while relying on a robust approximation of RSE that utilised the IQR. By looping a reasonable minimum of times and waiting for the robust error to fall low enough, we are left with output benchmark times that can be directly compared with lower error than that of a fixed number of execution cycles. This allowed the GA to correctly classify binaries as faster or slower than their competition, and as such more rapidly converge

on optimum sequences.

Preserving the simplicity of use of the GA by allowing any user to call it on any command rapidly via the `-c` flag allows the approach to resolve the largest barrier of entry to using an autotuning system. The constant command line stimulus also improves UX and informs the user on progress, potentially allowing a greater convenience for a user to not have to wait attentively for completion.

Overall, this project highlights the potential of GA driven approaches to compiler autotuning. It proves that by using rigorous, error-resistant evaluation metrics, evolutionary algorithms can reliably and efficiently discover performance gains that general compilation measures like -O3 cannot provide.

6.2 Limitations

6.2.1 Sample issues

There exist a variety of issues present with the testing that the report conclusion is built upon.

Small sample size

With only seven source code samples tested on, extracted from PolyBench, the sample size is extremely small. As a result it is difficult to conclusively state that the GA is capable of navigating the pass search space for truly any LLVM backend.

Sample homogeneity

Exacerbating the small number, all samples evaluated are drawn from the PolyBench benchmark suite. PolyBench is specifically designed to evaluate polyhedral compilation and consisting largely of matrix operations, stencil computations, and dynamic programming algorithms [46]. Consequently, the source code across these samples is highly homogenous. They are dominated by nested `for` loops with static control flow and linear array accesses.

This lack of diversity means the testing suite may omit common constructs seen throughout wider source code. Specifically an absence of the following are noted:

- complex pointer manipulation
- dynamic memory allocation
- unpredictable branching

- recursive functions
- complex data structures

Because the Genetic Algorithm (GA) was only tested on these specific binaries, the final pass sequences likely overfit to a specific subset of passes. Therefore, claiming the GA conclusively navigates the search space better than standard static heuristics is flawed. However, the results do show that within this specific domain the GA's effectiveness is conclusive.

6.2.2 Heuristic guidance

To limit the scope of the research, heuristic guidance was not implemented. This means mutation made wholly uninformed decisions on what passes are better within specific contexts. While this should not effect the optimum eventually reached by the GA, it is likely to increase runtime. There is consequentially a possibility that exploitable parts of the search space are completely missed due to the increased runtime.

6.2.3 Version locked software

The project is limited to a specific version of clang and the LLVM environment by the Docker container it is designed to work within. This is a necessary decision in order to ensure compatibility and a consistent pass registry regardless of updates to the relied upon software. Unfortunately this means that the autotuner cannot take advantage of future optimisations of clang's IR generation or a more effective LLVM -O3 pipeline and passes.

6.3 Future Work

The future work outlined provides solutions to the identified limitations alongside ideas that may benefit the autotuning process.

- *Sample size expansion.* By expanding the number of samples used in evaluation the sample homogeneity and lack of size is overcome. This would require powerful hardware to be conducted efficiently, and would allow for more conclusive evaluation of the GA implementation. This would be a simply carried out improvement by taking advantage of numerous benchmark suites for the types of operations PolyBench specifically lacks. Supplementing this may be utilising cloud compute to run a number of evaluations in parallel, working to combat time constraints.

- *Heuristic approaches to mutation.* Utilising a heuristic to guide GA search is a common approach for autotuning to, "significantly reduce the state and action spaces." [28]. This allows for significant further research, specifically the approach taken into feature extraction and data mining could allow for significant speed-up alongside performance improvements. This could utilise public results for pass sequence and source code pairs, such as that exposed by the CompilerGym leaderboard [11], alongside machine learning techniques for pattern discovery.

- *Generalise across LLVM versions.* Avoiding version locking the LLVM backend used to run optimisation passes would allow the GA's testharness to take advantage of new pass combinations. This could be accomplished by dynamically loading the currently installed LLVM version with the respective passes from its pass registry. Prospectively classifying passes into categories (module, function, cgscc, loop) can be accomplished by relative placement in the registry. Such an approach would be resilient to any version of LLVM utilising the same pass manager, thus allowing for more widespread usage and compatibility.

- *Utilise NLP (natural language processing) or GNNs to categorise source patterns.* NLP libraries such as Code2Vec [1] allow extracting a semantically significant vectorised representation of text. By aggregating these vectors across the source code a cached optimal pass sequence could be recalled that most closely matches the vector's direction. This would allow for search to start with several past optimums seeded in the population; this contrasts the current implementation that only loads randomly from past optima. Such a population would likely discover more significant improvement rapidly compared to the researched GA, due to a healthier (more likely to be functional) and fitter (faster) starting population.

Chapter 7

Legal, Social, Ethical and Professional Issues

This chapter details the specific legal, social, ethical, and professional issues intersecting with the implementation of this project, alongside the research conducted to support it. It ensures that we have adhered to the British Computing Society (BCS) code of conduct [5] throughout.

7.1 Code of Conduct

7.1.1 Legal

When developing software, it is imperative to have due regard for the legitimate rights of Third Parties. This inherently includes respecting intellectual property, patents, and copyright laws. Throughout this project, several open-source third-party libraries were utilised, specifically shlex [26], progressbar [59], and xxh3 [57]. It is a legal necessity to ensure that the use, modification, and distribution of these dependencies comply with their respective licenses.

7.1.2 Social

The BCS Code of Conduct mandates that members should seek to promote equal access to the benefits of IT. The core motivation for this project directly aligns with this principle; it provides an accessible and automated tool for software engineers who do not possess the specialised knowledge necessary for manually tweaking LLVM compiler passes. By abstracting this complexity behind a simple build command, the tool democratises access to highly optimised

software compilation.

Additionally, members are expected to have due regard for the environment. By discovering optimal pass sequences that actively reduce execution time, this project indirectly supports environmental sustainability by reducing the computing resources and energy consumption required for heavy duty applications; further discussed in 7.2.

7.1.3 Ethical

Ethical professional behaviour dictates that developers must not misrepresent the performance of products or systems, nor should they take advantage of the lack of relevant knowledge or inexperience of others. In evaluation of the results of the genetic algorithm, care has been taken to discuss limitations of the testing methodology. Because the tool was evaluated entirely on the PolyBench suite, the report acknowledges that the resulting pass sequences may overfit to this class of problems. Transparently reporting this limitation ensures that the performance claims surrounding the autotuner are honest and not misleading.

Furthermore, the system must be developed to avoid injuring others' property or systems by negligent action. Because the autotuner dynamically compiles and executes code, executing it within a Docker container mitigates the risk of unstable or untested machine code harming the host hardware or operating system.

7.1.4 Professional

As a professional, there is a fundamental duty to uphold the reputation of the profession and avoid any action that could bring it into disrepute. This involves operating strictly within one's professional competence and not claiming levels of expertise that are not possessed. Throughout the development of this autotuner, professional diligence was applied by leveraging rigorous benchmarking practices.

- Using the median to filter out environmental noise.
- Evaluating significance via Welch's t-test to ensure claims were statistically valid.

Finally, to uphold professional responsibility and ensure continuity, the system was fully documented according to mainstream Go best practices, ensuring maintainability and providing a clear duty of care to future users or developers interacting with the codebase.

7.2 Environmental

Software engineering inevitably incurs a cost environmentally as a result of its high energy usage. Not only is there the usage of computer systems throughout development, but the end use of the system also consumes energy. The system developed utilises a genetic algorithm. The requirement to run all population members over the course of many generations can be very costly in terms of energy usage when compared to other forms of search.

However, as the system's main goal is to produce more time and energy efficient programs there should be a point at which the trade-off of development and running cost is balanced by the gains made by the autotuner.

Assuming that an end-user appropriately applies the autotuner on critical portions of a program then the average observed 3% execution time improvement should provide a reasonable speed-up to the parent program as a whole. If a critical section that a program spends 10% of its time executing undergoes a 3% speed-up, then an improvement of 0.3% is granted to the program as a whole in accordance with Amdahl's law [3]. Therefore it is imperative that the autotuner is applied with foresight to allow for improvements to systems where an 0.3% reduction in time spent processing will have a positive impact on the environment, given an appropriately capped execution time.

Bibliography

[1] Alon, U. et al. "code2vec: learning distributed representations of code". *Proceedings of the ACM on Programming Languages* 3.POPL (Jan. 2019), pp. 1–29. DOI: 10.1145/3290353.

[2] *AMD Ryzen™ 7 5825U Specifications*. URL: https://www.amd.com/en/support/downloads/drivers.html (visited on Apr. 21, 2026).

[3] Amdahl, G. M. "Validity of the single processor approach to achieving large scale computing capabilities". *Proceedings of the April 18-20, 1967, spring joint computer conference*. 1967, pp. 483–485.

[4] Ansel, J. et al. "Opentuner: An extensible framework for program autotuning". *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 2014, pp. 303–316.

[5] *BCS Code of Conduct for members - Ethics for IT professionals — BCS*. URL: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ (visited on Mar. 20, 2026).

[6] Bondhugula, U. et al. "A practical automatic polyhedral parallelizer and locality optimizer". *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2008, pp. 101–113.

[7] Chen, J. et al. "Efficient Compiler Autotuning via Bayesian Optimization". *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 2021, pp. 1198–1209. DOI: 10.1109/ICSE43902.2021.00110.

[8] Collet, Y. *Cyan4973/xxHash*. original-date: 2014-04-30T23:32:49Z. Mar. 2026. URL: https://github.com/Cyan4973/xxHash (visited on Mar. 19, 2026).

[9] Cortes, C. and Vapnik, V. "Support-vector networks". *Machine learning* 20.3 (1995), pp. 273–297.

[10] Cummins, C. et al. “Programl: Graph-based deep learning for program optimization and analysis”. *arXiv preprint arXiv:2003.10536* (2020).

[11] Cummins, C. et al. “CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research”. *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. Seoul, Korea, Republic of: IEEE, Apr. 2022, pp. 92–105. DOI: 10.1109/CGO53902.2022.9741258.

[12] Datta, K. et al. “Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures”. *SC'08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing*. IEEE. 2008, pp. 1–12.

[13] De Nijs, R. and Klausen, T. L. *On the expected difference between mean and median*. 2013. DOI: 10.1285/I20705948V6N1P110.

[14] Derrick, B. and White, P. “Why Welch's test is Type I error robust”. *The Quantitative Methods for Psychology* 12.1 (Jan. 2016), pp. 30–38. DOI: 10.20982/tqmp.12.1.p030.

[15] Everitt, B. *The Cambridge dictionary of statistics*. 2. ed., reprinted with corrections. Cambridge: Cambridge Univ. Press, 2003.

[16] Felter, W. et al. “An updated performance comparison of virtual machines and Linux containers”. *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Mar. 2015, pp. 171–172. DOI: 10.1109/ISPASS.2015.7095802.

[17] Fischer, H. *A History of the Central Limit Theorem: From Classical to Modern Probability Theory*. New York, NY: Springer New York, 2011. DOI: 10.1007/978-0-387-87857-7.

[18] Forrest, S. and Mitchell, M. “Relative Building-Block Fitness and the Building-Block Hypothesis”. *Foundations of Genetic Algorithms*. Vol. 2. Elsevier, 1993, pp. 109–126. DOI: 10.1016/B978-0-08-094832-4.50013-1.

[19] *Frequently Asked Questions (FAQ) - The Go Programming Language*. URL: https://go.dev/doc/faq (visited on Mar. 18, 2026).

[20] Garciarena, U. and Santana, R. “Evolutionary Optimization of Compiler Flag Selection by Learning and Exploiting Flags Interactions”. *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*. Denver Colorado USA: ACM, July 2016, pp. 1159–1166. DOI: 10.1145/2908961.2931696.

[21] Garnett, R. *Bayesian Optimization*. Cambridge University Press, 2023, pp. 1–3.

[22] Georges, A., Buytaert, D., and Eeckhout, L. "Statistically rigorous java performance evaluation". *ACM SIGPLAN Notices* 42.10 (2007), pp. 57–76.

[23] *GlobalInterpreterLock*. URL: `https://wiki.python.org/moin/GlobalInterpreterLock` (visited on Mar. 30, 2026).

[24] Glover, F. W. and Kochenberger, G. A. *Handbook of metaheuristics*. Vol. 57. Springer Science & Business Media, 2003, pp. 1–3, 286–320.

[25] *Go Wiki: TableDrivenTests - The Go Programming Language*. URL: `https://go.dev/wiki/TableDrivenTests` (visited on Apr. 19, 2026).

[26] *google/shlex*. original-date: 2015-05-28T14:38:34Z. Dec. 2025. URL: `https://github.com/google/shlex` (visited on Mar. 19, 2026).

[27] Gunther, N. J. "A general theory of computational scalability based on rational functions". *arXiv preprint arXiv:0808.1431* (2008).

[28] Haj-Ali, A. et al. "Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning". *Proceedings of machine learning and systems* 2 (2020), pp. 70–81.

[29] Haj-Ali, A. et al. "NeuroVectorizer: end-to-end vectorization with deep reinforcement learning". *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization*. San Diego CA USA: ACM, Feb. 2020, pp. 242–255. DOI: `10.1145/3368826.3377928`.

[30] Hastie, T. *The elements of statistical learning: data mining, inference, and prediction*. 2009.

[31] Higgins, J. et al. *Cochrane Handbook for Systematic Reviews of Interventions*. 2nd ed. Wiley Cochrane. Newark: John Wiley & Sons, Incorporated, 2019, p. 163.

[32] Hoefler, T. and Belli, R. "Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results". *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Austin Texas: ACM, Nov. 2015, pp. 1–12. DOI: `10.1145/2807591.2807644`.

[33] Holland, J. H. *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. 1rst. MIT press ed., 4th printing. Complex adaptive systems. Cambridge, Mass: MIT Press, 1995.

[34] Hutter, F. et al. “ParamILS: an automatic algorithm configuration framework”. *Journal of artificial intelligence research* 36 (2009), pp. 267–306.

[35] *Intel® Xeon® Gold 5218R Processor*. URL: https://www.intel.com/content/www/us/en/products/sku/199342/intel-xeon-gold-5218r-processor-27-5m-cache-2-10-ghz/specifications.html (visited on Apr. 21, 2026).

[36] *Intel® Xeon® Processor E5-2640 v3*. URL: https://www.intel.com/content/www/us/en/products/sku/83359/intel-xeon-processor-e52640-v3-20m-cache-2-60-ghz/specifications.html (visited on Apr. 21, 2026).

[37] Jannoud, I. et al. “The Role of Genetic Algorithm Selection Operators in Extending WSN Stability Period: A Comparative Study”. *Electronics* 11.1 (Dec. 2021), p. 28. DOI: 10.3390/electronics11010028.

[38] Kågström, B., Ling, P., and Van Loan, C. “GEMM-based level 3 BLAS: high-performance model implementations and performance evaluation benchmark”. *ACM Transactions on Mathematical Software (TOMS)* 24.3 (1998), pp. 268–302.

[39] Kisuki, T., Knijnenburg, P. M., and O’Boyle, M. F. “Combined selection of tile sizes and unroll factors using iterative compilation”. *Proceedings 2000 International Conference on Parallel Architectures and Compilation Techniques (Cat. No. PR00622)*. IEEE. 2000, pp. 237–246.

[40] Klein, L. R. *A textbook of econometrics*. 1st. ed. Row, Peterson, 1953, p. 258.

[41] Lilja, D. J. *Measuring computer performance: a practitioner’s guide*. Cambridge university press, 2005, pp. 25–43.

[42] *LLVM Language Reference Manual — LLVM 23.0.0git documentation*. URL: https://llvm.org/docs/LangRef.html (visited on Apr. 21, 2026).

[43] Lowry, E. S. and Medlock, C. W. “Object code optimization”. *Communications of the ACM* 12.1 (Jan. 1969), pp. 13–22. DOI: 10.1145/362835.362838.

[44] Mouhah, K., Faiz, H., and Bourhnane, S. “Large Matrix Multiplication Algorithms: Analysis and Comparison”. *Proceedings of the 7th International Conference on Algorithms, Computing and Systems*. ICACS ’23. Larissa, Greece: Association for Computing Machinery, 2024, pp. 7–12. DOI: 10.1145/3631908.3631910.

[45] *Overview of the compiler - Rust Compiler Development Guide*. URL: https://rustc-dev-guide.rust-lang.org/overview.html (visited on Mar. 10, 2026).

[46] Reisinger, M. *MatthiasJReisinger/PolyBenchC-4.2.1*. original-date: 2016-06-10T11:45:57Z. Mar. 2026. URL: https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1 (visited on Mar. 20, 2026).

[47] Rocha, M. and Neves, J. "Preventing Premature Convergence to Local Optima in Genetic Algorithms via Random Offspring Generation". *Multiple Approaches to Intelligent Systems*. Ed. by G. Goos et al. Vol. 1611. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 127–136. DOI: 10.1007/978-3-540-48765-4_16.

[48] Sanchez, R. N. et al. "Using machines to learn method-specific compilation strategies". *Proceedings of the 9th Annual IEEE/ACM International Symposium on Code Generation and Optimization*. CGO '11. USA: IEEE Computer Society, 2011, pp. 257–266.

[49] Sandran, T., Zakaria, M. N. B., and Pal, A. J. "A Genetic Algorithm approach towards compiler flag selection based on compilation and execution duration". *2012 International Conference on Computer & Information Science (ICCIS)*. Vol. 1. 2012, pp. 270–274. DOI: 10.1109/ICCISci.2012.6297252.

[50] Shore, J. and Warden, S. *The art of agile development*. " O'Reilly Media, Inc.", 2021, p. 144.

[51] Sutton, R. S. and Barto, A. *Reinforcement learning: an introduction*. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts London, England: The MIT Press, 2020, pp. 1–4.

[52] Tagtekin, B. et al. "FOGA: Flag Optimization with Genetic Algorithm". *2021 International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*. Kocaeli, Turkey: IEEE, Aug. 2021, pp. 1–6. DOI: 10.1109/INISTA52262.2021.9548573.

[53] Tang, P. "Multi-core parallel programming in go". *Proceedings of the First International Conference on Advanced Computing and Communications*. Citeseer, 2010, pp. 64–69.

[54] Touati, S.-A.-A. and Barthou, D. "On the decidability of phase ordering problem in optimizing compilation". *Proceedings of the 3rd conference on Computing frontiers*. Ischia Italy: ACM, May 2006, pp. 147–156. DOI: 10.1145/1128022.1128042.

[55] Weisstein, E. W. *Log Normal Distribution*. URL: https://mathworld.wolfram.com/LogNormalDistribution.html (visited on Mar. 13, 2026).

[56] Welch, B. L. "THE GENERALIZATION OF 'STUDENT'S' PROBLEM WHEN SEVERAL DIFFERENT POPULATION VARLANCES ARE INVOLVED". *Biometrika* 34.1-2 (1947), pp. 28–35. DOI: 10.1093/biomet/34.1-2.28.

[57] Wendling, J. *zeebo/xxh3*. original-date: 2019-03-17T23:26:43Z. Mar. 2026. URL: https://github.com/zeebo/xxh3 (visited on Mar. 19, 2026).

[58] Williams, D. *Weighing the odds: a course in probability and statistics*. Reprinted. Cambridge: Cambridge Univ. Press, 2004, p. 165.

[59] Zack. *schollz/progressbar*. original-date: 2017-10-26T18:28:10Z. Mar. 2026. URL: https://github.com/schollz/progressbar (visited on Mar. 19, 2026).

[60] Zhu, M., Hao, D., and Chen, J. "Compiler Autotuning through Multiple-phase Learning". *ACM Trans. Softw. Eng. Methodol.* 33.4 (Apr. 2024). DOI: 10.1145/3640330.

Appendix A

Extra Information

A.1 Sensitivity analysis of executable repetitions

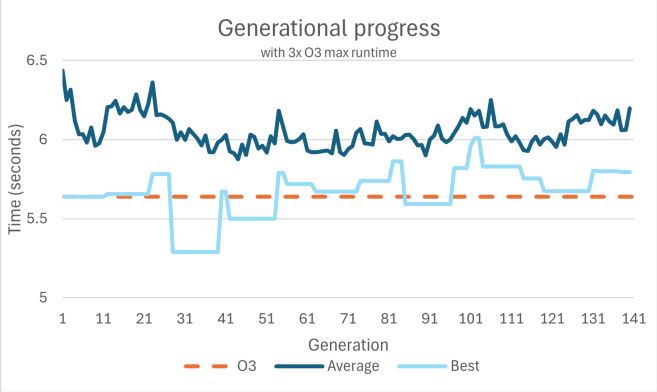


Figure A.1: Line graph showing results for autotuning nussinov with the maximum runtime for each executable being 3 times the optimal baseline.

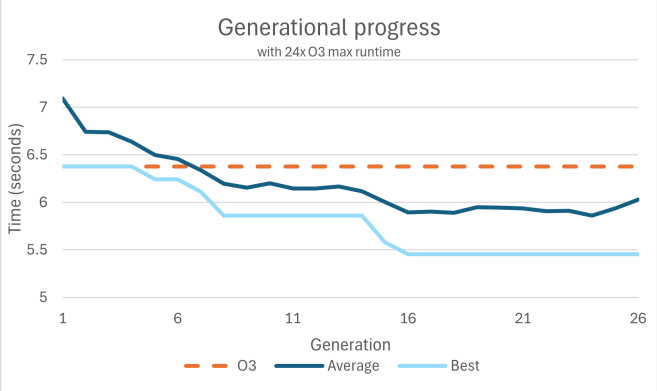


Figure A.2: Line graph showing results for autotuning nussinov with the maximum runtime for each executable being 24 times the optimal baseline.

Both results are seen to be under optimising relative to Figure 5.1.

A.2 Proof for median relative standard error

Supposing that we have made our benchmarks measurements, these would approximate a log-normal distribution [32]. This can be transformed into a normal distribution by applying the natural logarithm (ln) to each member of our dataset [55].

For our now normally distributed dataset, the sample standard deviation, s, can be robustly approximated using the Interquartile Range (IQR). According to the Cochrane Handbook [31], this relationship is:

$$s \approx \frac{\text{IQR}}{1.35} \tag{A.1}$$

The asymptotic variance of the sample median, σ^2, for a sample of size n, is given by Williams [58] as:

$$\sigma^2 = \frac{1}{4nf(m)^2} \tag{A.2}$$

where $f(m)$ is the density function evaluated at the median m. Because the data is normally distributed, the probability density function is $f(x) = \frac{1}{\sqrt{2\pi s^2}} e^{-\frac{(x-\mu)^2}{2s^2}}$. In a normal distribution, the median m is equal to the mean μ, which simplifies the function when evaluated at the median ($e^0 = 1$) to:

$$f(m) = \frac{1}{\sqrt{2\pi s^2}} \tag{A.3}$$

The standard error (SE) of the median is defined as the standard deviation σ of its sampling distribution [15]. Taking the square root of Equation A.2 and substituting $f(m)$ yields:

$$\begin{aligned} \text{SE(median)} &= \sigma = \frac{1}{2\sqrt{n} f(m)} \\ &= \frac{1}{2\sqrt{n}\left(\frac{1}{\sqrt{2\pi s^2}}\right)} \\ &= \frac{\sqrt{2\pi} \cdot s}{2\sqrt{n}} \\ &= \sqrt{\frac{\pi}{2}} \cdot \frac{s}{\sqrt{n}} \end{aligned} \tag{A.4}$$

Substituting the approximation for s from Equation A.1 into this standard error formula, we obtain:

$$\text{SE(median)} = \sqrt{\frac{\pi}{2}} \cdot \frac{\left(\frac{\text{IQR}}{1.35}\right)}{\sqrt{n}}$$

$$= \left(\frac{1}{1.35} \cdot \sqrt{\frac{\pi}{2}}\right) \cdot \frac{\text{IQR}}{\sqrt{n}} \tag{A.5}$$

It is important to note that the initial data transformation applied the natural logarithm, $Y = \ln(X)$, where X is our original dataset and Y our transformed dataset; thereby, the standard error resulting from Equation A.5 exists within the log domain. To transform this back to the original domain of the benchmark measurements, X, we apply the Delta Method [40]. The variance of a function, h, of a single random variable, B, is approximated by:

$$\text{Var}(h) \approx \left(\frac{\partial h}{\partial B}\right)^2 \text{Var}(B) \tag{A.6}$$

The variable B in this case is the log-transformed dataset Y. The inverse function applied is $h = e^Y$, relying on the fact that $e^{ln(x)} = x$. Substituting these specific variables into the Delta Method formula yields:

$$\begin{aligned} \text{Var}(X) &\approx \left(\frac{d}{dY} e^Y\right)^2 \text{Var}(Y) \\ &= (e^Y)^2 \text{Var}(Y) \\ &= X^2 \text{Var}(Y) \end{aligned} \tag{A.7}$$

Taking the square root obtains the standard deviation, and as such the standard error [15]:

$$\text{SE}(X) \approx X \cdot \text{SE}(Y) \tag{A.8}$$

By rearranging we find that the standard error calculated in the log domain, $\text{SE}(Y)$, is equivalent to the ratio of the standard error to the median in the original domain:

$$\text{SE}(Y) \approx \frac{\text{SE}(X)}{X} \tag{A.9}$$

By definition, the ratio of a metric's standard error to the metric is the Relative Standard Error (RSE), also known as the Coefficient of Variation. Therefore, calculating Equation A.5 on the log-transformed dataset directly yields the RSE for the median of the original benchmark measurements.

A.3 Welch's t-test

Student's t-test assumes means being compared for populations obey a normal distribution and have equal variances. Welch's t-test contrastingly does not make any assumptions about population variances, but still assumes a normal distribution of means [56].

Thanks to the Central limit theorem we can make the assumption that the means of benchmarking populations are normally distributed; this is despite our underlying data being log-normally distributed [17].

$$t = \frac{\Delta\overline{X}}{s_{\Delta\overline{X}}} = \frac{\overline{X}_1 - \overline{X}_2}{\sqrt{s^2_{\overline{X}_1} + s^2_{\overline{X}_2}}} \tag{A.10}$$

$$s_{\overline{X}_i} = \frac{s_i}{\sqrt{N_i}} \tag{A.11}$$

$$\nu \approx \frac{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)^2}{\frac{s_1^4}{N_1^2\nu_1} + \frac{s_2^4}{N_2^2\nu_2}} = \frac{s^4_{\Delta\overline{X}}}{\frac{s^4_{\overline{X}_1}}{\nu_1} + \frac{s^4_{\overline{X}_2}}{\nu_2}} \tag{A.12}$$

$$\nu \approx \nu' \frac{s^4_{\Delta\overline{X}}}{s^4_{\overline{X}_1} + s^4_{\overline{X}_2}} \tag{A.13}$$

A.4 LLVM pass sequences

Default O3 sequence for nussinov:

```
annotation2metadata,forceattrs,function<eager-inv>(declare-to-assign),
inferattrs,coro-early,function<eager-inv>(ee-instrument<>,lower-expect,
simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;no-switch-range-
to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;
no-hoist-loads-stores-with-cond-faulting;no-sink-common-insts;
speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,
sroa<modify-cfg>,early-cse<>,callsite-splitting),openmp-opt,ipsccp,
called-value-propagation,globalopt,function<eager-inv>(mem2reg,
instcombine<max-iterations=1;no-verify-fixpoint>,simplifycfg<bonus-inst-
threshold=1;no-forward-switch-cond;switch-range-to-icmp;
no-switch-to-lookup;keep-loops;no-hoist-common-insts;
no-hoist-loads-stores-with-cond-faulting;no-sink-common-insts;speculate-
```

blocks;simplify-cond-branch;no-speculate-unpredictables>),always-inline,require<globals-aa>,function(invalidate<aa>),require<profile-summary>,cgscc(devirt<4>(inline,function-attrs<skip-non-recursive-function-attrs>,argpromotion,openmp-opt-cgscc,function<eager-inv;no-rerun>(sroa<modify-cfg>,early-cse<memssa>,speculative-execution<only-if-divergent-target>,jump-threading,correlated-propagation,simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;no-sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,instcombine<max-iterations=1;no-verify-fixpoint>,aggressive-instcombine,libcalls-shrinkwrap,tailcallelim,simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;no-sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,reassociate,constraint-elimination,loop-mssa(loop-instsimplify,loop-simplifycfg,licm<no-allowspeculation>,loop-rotate<header-duplication;no-prepare-for-lto>,licm<allowspeculation>,simple-loop-unswitch<nontrivial;trivial>),simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;no-sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,instcombine<max-iterations=1;no-verify-fixpoint>,loop(loop-idiom,indvars,extra-simple-loop-unswitch-passes,loop-deletion,loop-unroll-full),sroa<modify-cfg>,vector-combine,mldst-motion<no-split-footer-bb>,gvn<>,sccp,bdce,instcombine<max-iterations=1;no-verify-fixpoint>,jump-threading,correlated-propagation,adce,memcpyopt,dse,move-auto-init,loop-mssa(licm<allowspeculation>),coro-elide,simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,instcombine<max-iterations=1;no-verify-fixpoint>),function-attrs,coro-split,coro-annotation-

```
elide)),deadargelim,coro-cleanup,globalopt,globaldce,elim-avail-extern,rpo-
function-attrs,recompute-globalsaa,function<eager-inv>(float2int,lower-
constant-intrinsics,chr),function<eager-inv>(loop(loop-rotate<header-
duplication;no-prepare-for-lto>,loop-deletion)),function<eager-inv>(loop-
distribute,inject-tli-mappings,loop-vectorize<no-interleave-forced-only;no-
vectorize-forced-only;>,infer-alignment,loop-load-elim,instcombine<max-
iterations=1;no-verify-fixpoint>,simplifycfg<bonus-inst-
threshold=1;forward-switch-cond;switch-range-to-icmp;switch-to-lookup;no-
keep-loops;hoist-common-insts;no-hoist-loads-stores-with-cond-
faulting;sink-common-insts;speculate-blocks;simplify-cond-branch;no-
speculate-unpredictables>,slp-vectorizer,vector-combine,instcombine<max-
iterations=1;no-verify-fixpoint>,loop-unroll<O3>,transform-
warning,sroa<preserve-cfg>,infer-alignment,instcombine<max-iterations=1;no-
verify-fixpoint>,loop-mssa(licm<allowspeculation>),alignment-from-
assumptions,loop-sink,instsimplify,div-rem-pairs,tailcallelim,
simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;
switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-
insts;hoist-loads-stores-with-cond-faulting;no-sink-common-insts;
speculate-blocks;simplify-cond-branch;speculate-unpredictables>),
globaldce,constmerge,cg-profile,rel-lookup-table-converter,
function(annotation-remarks)
```

Genetic algorithm discovered sequence for nussinov:

```
annotation2metadata,function<eager-inv>(declare-to-assign),function<eager-
inv>(loop(loop-interchange)),coro-early,function<eager-inv>(ee-
instrument<>),cross-dso-cfi,function<eager-inv>(early-cse<>),called-value-
propagation,attributor,function<eager-inv>(loop-data-prefetch,lower-
widenable-condition),globalopt,function<eager-inv>(irce,mem2reg,wasm-eh-
prepare,instcombine<max-iterations=1;no-verify-fixpoint>),always-
inline,function<eager-inv>(loop(loop-idiom)),require<globals-
aa>,function<eager-inv>(loop(loop-unroll-full)),require<profile-
summary>,coro-cleanup,function<eager-inv>(loop(extra-simple-loop-unswitch-
passes)),elim-avail-extern,recompute-globalsaa,function<eager-
inv>(chr),function<eager-inv>(loop(loop-rotate<header-duplication;no-
```

prepare-for-lto>)),attributor,function<eager-inv>(assume-builder),function<eager-inv>(loop(loop-instsimplify)),function<eager-inv>(callsite-splitting),function<eager-inv>(loop(indvars)),function<eager-inv>(loop-distribute,consthoist,inject-tli-mappings,loop-vectorize<no-interleave-forced-only;no-vectorize-forced-only;>,loop-mssa(licm<allowspeculation>),loop-load-elim,instcombine<max-iterations=1;no-verify-fixpoint>,simplifycfg<bonus-inst-threshold=1;forward-switch-cond;switch-range-to-icmp;switch-to-lookup;no-keep-loops;hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,vector-combine,sccp,simplifycfg<bonus-inst-threshold=1;forward-switch-cond;switch-range-to-icmp;switch-to-lookup;no-keep-loops;hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,instcombine<max-iterations=1;no-verify-fixpoint>,transform-warning,infer-alignment,instcombine<max-iterations=1;no-verify-fixpoint>),function<eager-inv>(loop(loop-reduce)),ipsccp,function<eager-inv>(loop-sink,instsimplify,div-rem-pairs,aggressive-instcombine),function<eager-inv>(loop(loop-instsimplify,loop-rotate<header-duplication;no-prepare-for-lto>)),coro-cleanup,globaldce,function<eager-inv>(loop-distribute,consthoist,inject-tli-mappings),canonicalize-aliases,canonicalize-aliases,function<eager-inv>(sccp,make-guards-explicit,loop-vectorize<no-interleave-forced-only;no-vectorize-forced-only;>,loop-mssa(licm<allowspeculation>),loop-load-elim,instcombine<max-iterations=1;no-verify-fixpoint>,simplifycfg<bonus-inst-threshold=1;forward-switch-cond;switch-range-to-icmp;switch-to-lookup;no-keep-loops;hoist-common-insts;no-hoist-loads-stores-with-cond-faulting;sink-common-insts;speculate-blocks;simplify-cond-branch;no-speculate-unpredictables>,vector-combine,instcombine<max-iterations=1;no-verify-fixpoint>,loop-unroll<O3>,transform-warning,sroa<preserve-cfg>,infer-alignment,instcombine<max-iterations=1;no-verify-fixpoint>,vector-combine,instsimplify),rpo-function-attrs,function<eager-inv>(simplifycfg<bonus-inst-threshold=1;no-forward-switch-cond;switch-

range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;hoist-loads-stores-with-cond-faulting;no-sink-common-insts;speculate-blocks;simplify-cond-branch;speculate-unpredictables>),function(annotation-remarks),constmerge,cg-profile,rel-lookup-table-converter,assign-guid,function(annotation-remarks)

Appendix B

User Guide

B.1 Instructions

Running on Windows

- Install Docker
- `docker build -t <container_name> .`
- `docker run -it --rm -v ${pwd}:/project -w /project <container_name>`
- IN the container: go run .
- From here you can test running on example projects
- e.g. `go run .  -c "clang++ -I./example/string_concat/include ./example/string_concat/src/TestStringCons.cpp" -t <n>h`

  where n is your available time or alternatively build via

  `go build -o <output_filename> .`

  and then run via

  `<output_filename> -c "clang++ -I./example/string_concat/include ./example/string_concat/src/TestStringCons.cpp" -t <n>h`
- To review the command line arguments to the executable, use the -help argument.

  e.g. `go run .  -help`

  or

  `<output_filename> -help`

  This will produce an output that looks like:

```
Usage of <output_filename>:
    -c string
        command
    -l int
        max sequence length (default 200)
    -p int
        population size (default 30)
    -t duration
maximum run time (e.g., 10s, 1m) (default 2h0m0s)
```

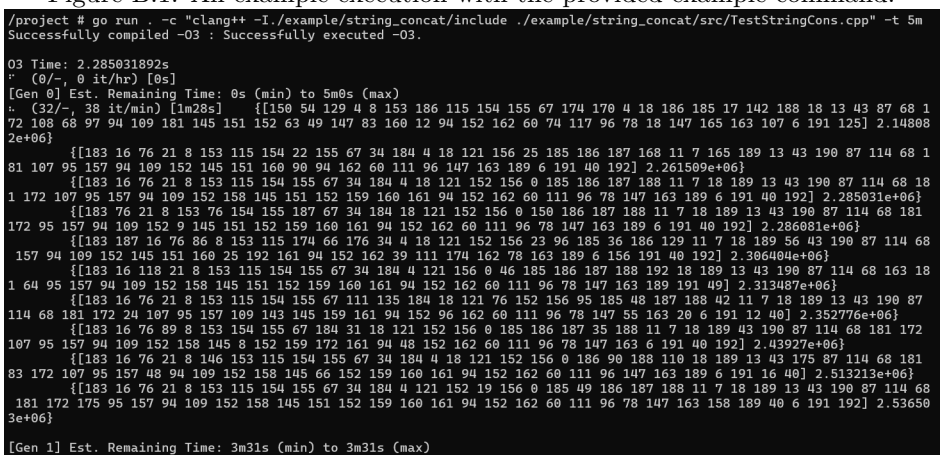
Running on MacOS/linux

Same as Windows except:

- {pwd} -> (pwd)
- e.g. `docker build -t foo .  && docker run -it --rm -v $(pwd):/project -w /project foo`

Once the run command is executed, the population will be initialised. The genetic algorithm then begins compiling and evaluating the fitness of all the population members. The typical genetic algorithm will then run, choosing the best of the members, performing crossover, and performing mutation. This will repeat until the GA converges or the timeout is reached. The terminal output represent the GAs interrim results, where each line represents a member of the population's passes encoded as integers and fitness score.

Figure B.1: An example execution with the provided example command.



Appendix C

Source Code

I verify that I am the sole author of the programs contained in this folder, except where explicitly stated to the contrary

Thomas Holden

21/04/2026

C.1 Dockerfile

```
FROM alpine:3.23.2
COPY --from=golang:1.25.4-alpine /usr/local/go/ /usr/local/go/

ENV PATH="/usr/local/go/bin:${PATH}"
RUN apk update && apk add clang21 llvm
CMD ["/bin/sh", "-c", "go mod tidy && exec /bin/sh"]
```

C.2 go.mod

```
module 6CCS3PRJ/thomasholden

go 1.25.3

require (
        github.com/google/shlex v0.0.0-20191202100458-e7afc7fbc510
```

```
        github.com/schollz/progressbar/v3 v3.18.0
        github.com/zeebo/xxh3 v1.1.0
)

require (
        github.com/klauspost/cpuid/v2 v2.2.10
    // indirect
        github.com/mitchellh/colorstring v0.0.0-20190213212951-
    d06e56a500db // indirect
        github.com/rivo/uniseg v0.4.7 // indirect
        golang.org/x/sys v0.38.0 // indirect
        golang.org/x/term v0.37.0 // indirect
        golang.org/x/tools v0.5.1-0.20230111220935-a7f7db3f17fc
    // indirect
        golang.org/x/tools/cmd/cover v0.1.0-deprecated // indirect
)
```

C.3 go.sum

```
github.com/chengxilo/virtualterm v1.0.4
h1:Z6IpERbRVlfB8WkOmtbHiDbBANU7cimRIof7mk9/PwM=
github.com/chengxilo/virtualterm v1.0.4/go.mod h1:DyxxBZz/
x1iqJjFxTFcr6/x+jSpqN0iwWCOK1q10rlY=
github.com/davecgh/go-spew v1.1.1 h1:vj9j/
u1bqnvCEfJOwUhtlOARqs3+rkHYY13jYWTU97c=
github.com/davecgh/go-spew v1.1.1/go.mod h1:J7Y8YcW2NihsgmVo/
mv3lAwl/skON4iLHjSsI+c5H38=
github.com/google/shlex v0.0.0-20191202100458-e7afc7fbc510
h1:El6M4kTTCOh6aBiKaUGG7oYTSPP8MxqL4YI3kZKwcP4=
github.com/google/shlex v0.0.0-20191202100458-e7afc7fbc510/
go.mod h1:pupxD2MaaD3pAXIBCelhxNneeOaAeabZDe5s4K6zSpQ=
github.com/klauspost/cpuid/v2 v2.2.10
h1:tBs3QSyvjDyFTq3uoc/9xFpCuOsJQFNPiAhYdw2skhE=
```

github.com/klauspost/cpuid/v2 v2.2.10/go.mod

h1:hqwkgyIinND0mEev00jJYCxPNVRVXFQeu1XKlok6oO0=

github.com/mattn/go-runewidth v0.0.16

h1:E5ScNMtiwvlvB5paMFdw9p4kSQzbXFikJ5SQO6TULQc=

github.com/mattn/go-runewidth v0.0.16/go.mod

h1:Jdepj2loyihRzMpdS35Xk/zdY8IAYHsh153qUoGf23w=

github.com/mitchellh/colorstring v0.0.0-20190213212951-

d06e56a500db h1:62I3jR2EmQ4l5rM/4FEfDWcRD+abF5XlKShorW5LRoQ=

github.com/mitchellh/colorstring v0.0.0-20190213212951-

d06e56a500db/go.mod h1:l0dey0ia/

Uv7NcFFVbCLtqEBQbrT4OCwCSKTEv6enCw=

github.com/pmezard/go-difflib v1.0.0

h1:4DBwDE0NGyQoBHbLQYPwSUPoCMWR5BEzIk/f1lZbAQM=

github.com/pmezard/go-difflib v1.0.0/go.mod

h1:iKH77koFhYxTK1pcRnkKkqfTogsbg7gZNVY4sRDYZ/4=

github.com/rivo/uniseg v0.4.7

h1:WUdvkW8uEhrYfLC4ZzdpI2ztxP1I582+49Oc5Mq64VQ=

github.com/rivo/uniseg v0.4.7/go.mod

h1:FN3SvrM+Zdj16jyLfmOkMNblXMcoc8DfTHruCPUcx88=

github.com/schollz/progressbar/v3 v3.18.0

h1:uXdoHABRFmNIjUfte/Ex7WtuyVslrw2wVPQmCN62HpA=

github.com/schollz/progressbar/v3 v3.18.0/go.mod

h1:IsO3lpbaGuzh8zIMzgY3+J8l4C8GjO0Y9S69eFvNsec=

github.com/stretchr/testify v1.9.0

h1:HtqpIVDClZ4nwg75+f6Lvsy/wHu+3BoSGCbBAcpTsTg=

github.com/stretchr/testify v1.9.0/go.mod

h1:r2ic/lqez/lEtzL7wO/rwa5dbSLXVDPFyf8C91i36aY=

github.com/zeebo/assert v1.3.0

h1:g7C04CbJuIDKNPFHmsk4hwZDO5O+kntRxzaUoNXj+IQ=

github.com/zeebo/assert v1.3.0/go.mod

h1:Pq9JiuJQpG8JLJdtkwrJESF0Foym2/D9XMU5ciN/wJ0=

github.com/zeebo/xxh3 v1.1.0

h1:s7DLGDK45Dyfg7++yxI0khrfwq9661w9EN78eP/UZVs=

```
github.com/zeebo/xxh3 v1.1.0/go.mod
h1:IisAie1LELR4xhVinxWS5+zf1lA4p0MW4T+w+W07F5s=
golang.org/x/sys v0.38.0
h1:3yZWxaJjBmCWXqhN1qh02AkOnCQ1poK6oF+a7xWL6Gc=
golang.org/x/sys v0.38.0/go.mod
h1:OgkHotnGiDImocRcuBABYBEXf8A9a87e/uXjp9XT3ks=
golang.org/x/term v0.37.0
h1:8EGAD0qCmHYZg6J17DvsMy9/wJ7/D/4pV/wfnld5lTU=
golang.org/x/term v0.37.0/go.mod
h1:5pB4lxRNYYVZuTLmy8oR2BH8dflOR+IbTYFD8fi3254=
golang.org/x/tools v0.5.1-0.20230111220935-a7f7db3f17fc
h1:zRn9MzwG18RZhyanShCfUwJTcobvqw8fOjjROFN9jtM=
golang.org/x/tools v0.5.1-0.20230111220935-a7f7db3f17fc/go.mod
h1:N+Kgy78s5I24c24dU8OfWNEotWjutIs8SnJvn5IDq+k=
golang.org/x/tools/cmd/cover v0.1.0-deprecated
h1:Rwy+mWYz6loAF+LnG1jHG/JWMHRMMC2/1XX3Ejkx9lA=
golang.org/x/tools/cmd/cover v0.1.0-deprecated/go.mod
h1:hMDiIvlpN1NoVgmjLjUJE9tMHyxHjFX7RuQ+rW12mSA=
gopkg.in/yaml.v3 v3.0.1
h1:fxVm/GzAzEWqLHuvctI91KS9hhNmmWOoWu0XTYJS7CA=
gopkg.in/yaml.v3 v3.0.1/go.mod
h1:K4uyk7z7BCEPqu6E+C64Yfv1cQ7kz7rIZviUmN+EgEM=
```

C.4 main.go

```
package main

import (
        "6CCS3PRJ/thomasholden/geneticalgorithm"
        "flag"
        "time"

        "bufio"
```

```
        "fmt"
        "os"
)

// command line args:
// -p population size
// -l max sequence length
// -t maximum run time e.g. 1s 1m 1h
// -c command
func main() {
        popSize := flag.Int("p", 30, "population size")
        maxLength := flag.Int("l", 200, "max sequence length")
        maxTime := flag.Duration("t", 2*time.Hour, "maximum run
    time (e.g., 10s, 1m)")
        command := flag.String("c", "", "command")

        flag.Parse()

        commandText := *command
        if commandText == "" {
                fmt.Print("Enter command: ")
                reader := bufio.NewReader(os.Stdin)
                // e.g. clang++ -I./example/string_concat/include ./
        example/string_concat/src/TestStringCons.cpp
                newCommand, err := reader.ReadString('\n')
                if err != nil {
                        fmt.Printf("text input error: %v\n", err)
                        return
                }
                commandText = newCommand
        }

        gaPopulation, err :=
```

```
    geneticalgorithm.CreatePopulation(*popSize, *maxLength,
    commandText)
        if err != nil {
                fmt.Printf("ga error: %v\n", err)
                return
        }
        generations :=
    gaPopulation.RunGenerationsForDuration(*maxTime)

        seq, err := gaPopulation.Best()
        if err != nil {
                fmt.Printf("ga error: %v\n", err)
                return
        }
        fmt.Printf("%s\n", seq)
        fmt.Printf("Converged in %d generations.\n", generations)
}
```

C.5 README.md

```
# 6CCS3PRJ
Optimising for LLVM compiler passes using GA approaches.

## Running on Windows
 - Install Docker

```bash
docker build -t <container_name> .
```

```bash
docker run -it --rm -v ${pwd}:/project -w /project <container_name>
```
```

```
 - IN the container: go run .
 - From here you can test running on example projects
 - e.g., (where n is your available time)

```bash
go run . -c "clang++ -I./example/string_concat/include ./example/string_concat/src/Te
```

 - Alternatively, build the project via:

```bash
go build -o <output_filename> .
```

 - And then run via:

```bash
./<output_filename> -c "clang++ -I./example/string_concat/include ./example/string_con
```

 - To review the command line arguments to the executable, use the -help argument:

Running on MacOS/linux
 - Same as Windows except:
 - {pwd} -> (pwd)
 - e.g.

```bash
docker build -t foo . && docker run -it --rm -v $(pwd):/project -w /project foo
```

Execution
```

Once the run command is executed, the population will be initialised. The genetic algo

C.6 example

C.6.1 hello_world

hello_world.cpp

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";

    return 0;
}
```

C.6.2 string_concat

include

StringConstruction.h

```
#ifndef STRINGCONSTRUCTION_H
#define STRINGCONSTRUCTION_H

#include <string>
using std::string;
```

```
#include <unordered_map>

const int INF = 2147483647;
```

```
int minCloneLength;

int min(int a, int b) {
        if (a < b) {
                return a;
        }
        else {
                return b;
        }
}

int findMaxRepeatedLength(size_t index, const std::string&
target) {
    size_t n = target.length();

    size_t maxPossibleLength = std::min(index, n - index);
    size_t maxLen = 0;

    for (size_t left = 0; left < index; ++left) {
                if ((index - left) <= maxLen) {
            break;
        }

        if (target[left] == target[index]) {
            size_t currentLen = 1;

            while (currentLen < maxPossibleLength &&
                   left + currentLen < index &&
                   target[left + currentLen] == target[index +
                   currentLen]) {
                currentLen++;
            }
```

```
            if (currentLen > maxLen) {
                maxLen = currentLen;

                if (maxLen == maxPossibleLength) return maxLen;
            }
        }
    }

    return (maxLen >= minCloneLength) ? maxLen : 0;
}

int findCost(size_t index, int cloneCost, int appendCost,
std::string& target, std::unordered_map<int, int>& minMap) {
        if (target.size() == index) {
                return 0;
        }

        if (minMap.find(index) != minMap.end()) {
                return minMap[index];
        }

        int cloneVal = INF;
        int appendVal;
        int minVal;

        int len = findMaxRepeatedLength(index, target);

        if (len != 0) {
                cloneVal = cloneCost + findCost(index + len, cloneCost,
        appendCost, target, minMap);
        }

        appendVal = appendCost + findCost(index + 1, cloneCost,
```

```
    appendCost, target, minMap);


        minVal = min(cloneVal, appendVal);

        minMap[index] = minVal;

        return minVal;
}

int stringConstruction(std::string target, int cloneCost, int appendCost) {
        std::unordered_map<int, int> minMap;
        minCloneLength = (cloneCost + appendCost - 1) / appendCost;

        return findCost(0, cloneCost, appendCost, target, minMap);
}



#endif
```

src

TestStringCons.cpp

```
#include "StringConstruction.h"

#include <iostream>
using std::cout;
using std::endl;
using std::ostream;

#include <vector>
using std::vector;
```

```
struct Testcase {
    string target;
    int appendCost;
    int cloneCost;
    int bestCost;

    string notes;
};

ostream & operator<<(ostream & o, const Testcase & t) {
    o << "Target string: " << t.target << "\n";
    o << "Cost of appending an arbitrary character: " <<
    t.appendCost << "\n";
    o << "Cost of cloning a substring, and appending: " <<
    t.cloneCost << "\n";
    o << "Cheapest way to make the string should be: " <<
    t.bestCost << "\n";

    return o;
}

int toLoop() {
    int retval = 0;

    vector<Testcase> testcases{

        {"aa",1,2,2,"    Append 'a' (cost 1), append 'a' (cost
        1)"},
        {"aaaa",2,3,7,"    Append 'a' (cost 2), append 'a'
        (cost 2), clone 'aa' (cost 3)"},
        {"xzxpzxzxpq", 10, 11, 71, "    Append 'x' (cost 10),
        Append 'z' (cost 10), Append 'x' (cost 10), Append 'p'
```

```
    (cost 10), Append 'z' (cost 10),\n    Clone 'xzxp'
    (cost 11), Append 'q' (cost 10)"}

};


for (size_t i = 0; i < testcases.size(); ++i) {

    cout << "Testcase " << i << "\n";
    cout << testcases[i];
    cout << "\nRunning your function\n";

    int costWas = stringConstruction(testcases[i].target,
    testcases[i].cloneCost, testcases[i].appendCost);

    if (costWas == testcases[i].bestCost) {
        cout << " -- Test passed, correct cost was
        returned\n\n\n";
    } else {
        cout << " -- Test failed, incorrect cost of " <<
        costWas << " was returned\n";
        cout << "    To help your debugging, the cheapest
        way to make the string should be:\n";
        cout << testcases[i].notes << "\n\n\n";

        ++retval;
    }



}

string bigString =
```

”abcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaip
mbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgblgmbaiabcblgmbc
ntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgb
lgmbaipcbcntdblgblgmbaipiblgblgmbaipmbcntdblgblgmbaipfdmbcnt
dblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgblgm
baiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmba
ipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntdblgblgmbaipfdmb
cntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgb
lgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblg
mbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmbaipmbcntdblg
blgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbai
pobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfd
mbcntdblgblgmbaipmbcntdblgblgmbaiprtifpblgmbtmblgmbaipfdmbcn
tdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgblg
mbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmb
aipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmbaipmbcntdblgbl
gmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipo
bacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmb
cntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntdbl
gblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmba
ipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipf
dmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgm
baipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgmbaipfdmbcn
tdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgblg
mbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmb
aipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntdblgblgmbaipfdm
bcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblg
blgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgbl
gmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmbaipmbcntdbl
gblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmba
ipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipf
dmbcntdblgblgmbaipmbcntdblgblgmbaiprtifpblgmbtmblgmbaipfdmbc
ntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgbl

gmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcqaipfdmbcn
tdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntdblgb
lgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaip
obacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdm
bcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmba
ipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcnt
dblgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdbl
gblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifpblgmbtmblg
mbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipob
acjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbc
ntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmbaip
mbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdb
lgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgb
lgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgmb
aipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcn
tdblgblgmbaipobacjoirmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbc
ntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipobacjodblgbl
gmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgm
baipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmbaipmbcntdblgb
lgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaip
obacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfdm
bcntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntdb
lgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmb
aipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaip
fdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblg
mbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcb
cntdblgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcnt
dblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifpblgmbtm
blgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbai
pobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfd
mbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgblgmb
aipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcn
tdblgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdb

```
lgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgbl
gmbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipc
bcntdblgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcn
tdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgbl
gmbaipiblgblgmbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntd
blgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbai
pobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipfd
mbcntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgmbaipmbcntd
blgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgm
baipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbai
pfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmbaipiblgbl
gmbaipmbcntdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipc
bcntdblgblgmbaipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcn
tdblgblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifpblgmbt
mblgmbaipfdmbcntdblgblgmbaipmbcntdblgblgmbaipcbcntdblgblgmba
ipobacjodblgblgmbaiabcblgmbcntdblgblgmbaipmbcntdblgblgmbaipf
dmbcqaipfdmbcntdblgblgmbaipmbcntdblgblgmbaiprtifbcntdblgblgm
baipmbcntdblgblgmbaip";
string doubleIt = bigString + bigString;

cout << "Big target string testcase, using a " <<
doubleIt.size() << " character test string.  An excellent
solution will complete in around a hundredth of a second.
\n";
cout << "(Note -- if the program freezes at this point,
Ctrl+C will stop it from running)\n";

int result = stringConstruction(doubleIt, 1235, 1234);

if (result == 59249) {
    cout << "-- Correct cost of 59249 returned\n";
} else {
    cout << "-- Incorrect cost of " << result << "
```

```
        returned, rather than 59249\n";
    }

    return retval;
}

int main() {
    for (int i = 0; i < 50; i++) {
        toLoop();
    }
    return 0;
}
```

C.7 geneticalgorithm

C.7.1 crossover.go

```
package geneticalgorithm

import (
        "math/rand"
        "slices"
)

// reproduce enacts crossover between two population members
(pass sequences).
func reproduce(a, b []int, maxVal, maxLen int) ([]int, []int) {
        var newA, newB []int
        randNum := rand.Int() % 5
        fallback := func(a, b []int) ([]int, []int) { return
    pairMutate(a, b, maxVal, maxLen) }

        if len(a) == 0 || len(b) == 0 { // avoid splitting at 0
```

```
                newA, newB = pairMutate(a, b, maxVal, maxLen)
        } else if randNum < 3 && !slices.Equal(a, b) {
                newA, newB = lcsCrossover(a, b, fallback)
        } else if randNum == 3 || slices.Equal(a, b) {
                newA, newB = twoPointCrossover(a, b, fallback)
        } else {
                newA, newB = fallback(a, b)
        }

        if len(newA) > maxLen {
                newA = slices.Delete(newA, maxLen, len(newA))
        }
        if len(newB) > maxLen {
                newB = slices.Delete(newB, maxLen, len(newB))
        }

        return newA, newB
}

// pairMutate returns mutated versions of the input slices.
func pairMutate(a, b []int, maxVal, maxLen int) ([]int, []int) {
        newA := slices.Clone(a)
        newB := slices.Clone(b)
        newA = mutate(newA, maxVal, maxLen)
        newB = mutate(newB, maxVal, maxLen)

        return newA, newB
}

// onePointCrossover enacts one point crossover between two
population members.
func onePointCrossover(a, b []int, fallback func([]int, []int)
([]int, []int)) ([]int, []int) {
```

```
        if len(a) <= 1 || len(b) <= 1 { // Avoid panic
                return fallback(a, b)
        }

        var newA, newB []int

        aPtr := rand.Intn(len(a)-1) + 1
        bPtr := rand.Intn(len(b)-1) + 1

        newA = append(newA, a[:aPtr]...)
        newA = append(newA, b[bPtr:]...)

        newB = append(newB, b[:bPtr]...)
        newB = append(newB, a[aPtr:]...)

        return newA, newB
}

// twoPointCrossover enacts two point crossover between two
population members.
func twoPointCrossover(a, b []int, fallback func([]int, []int)
([]int, []int)) ([]int, []int) {
        if len(a) <= 2 || len(b) <= 2 { // Avoid panic
                return fallback(a, b)
        }

        var newA, newB []int

        aPtrStart := rand.Intn(len(a)-2) + 1
        aPtrEnd := rand.Intn(len(a)-aPtrStart-1) + 1 + aPtrStart

        bPtrStart := rand.Intn(len(b)-2) + 1
        bPtrEnd := rand.Intn(len(b)-bPtrStart-1) + 1 + bPtrStart
```

```
        newA = append(newA, a[:aPtrStart]...)
        newA = append(newA, b[bPtrStart:bPtrEnd]...)
        newA = append(newA, a[aPtrEnd:]...)

        newB = append(newB, b[:bPtrStart]...)
        newB = append(newB, a[aPtrStart:aPtrEnd]...)
        newB = append(newB, b[bPtrEnd:]...)

        return newA, newB
}

// lcsCrossover takes the longest common subsequence of two
population members.
// Following this, it swaps the non-matching subsequences
randomly to generate
// two new members with the common subsequence preserved.
//
// https://en.wikipedia.org/wiki/Longest_common_subsequence
func lcsCrossover(a, b []int, fallback func([]int, []int)
([]int, []int)) ([]int, []int) {
        if len(a) == 0 || len(b) == 0 {
                return fallback(a, b)
        }

        matchedIdxsA, matchedIdxsB := lcsPairs(a, b)
        if len(matchedIdxsA) == 0 || len(matchedIdxsB) == 0 {
                return twoPointCrossover(a, b, fallback)
        }
        var newA, newB []int

        var prevA, prevB, currA, currB int
        // One extra loop to handle the tail without code
```

```
    duplication
        for k := range len(matchedIdxsA) + 1 {
                currA, currB = len(a), len(b)
                if k < len(matchedIdxsA) {
                        currA, currB = matchedIdxsA[k], matchedIdxsB[k]
                }

                segA, segB := a[prevA:currA], b[prevB:currB]

                if rand.Int()%2 != 0 {
                        segA, segB = segB, segA // Swaps slice headers
                }
                newA = append(newA, segA...)
                newB = append(newB, segB...)

                // Append the match itself
                if k < len(matchedIdxsA) {
                        val := a[currA]
                        newA = append(newA, val)
                        newB = append(newB, val)
                        prevA, prevB = currA+1, currB+1
                }
        }

        if slices.Equal(newA, a) || slices.Equal(newA, b) {
                return fallback(a, b)
        }

        return newA, newB
}

// lcsPairs returns two slices listing the LCS idxs of the
first sequence and
```

```
// the matching idxs of the second sequence respectively.
func lcsPairs[T comparable](seqA, seqB []T) ([]int, []int) {
        n, m := len(seqA), len(seqB)
        dpTable := make([][]int, n+1)
        for i := range dpTable {
                dpTable[i] = make([]int, m+1)
        }

        // Constructs the dp table of LCS lengths for every prefix
    of the seqs
        for i := 1; i <= n; i++ {
                for j := 1; j <= m; j++ {
                        if seqA[i-1] == seqB[j-1] {
                                dpTable[i][j] = dpTable[i-1][j-1] + 1
                        } else {
                                dpTable[i][j] = max(dpTable[i-1][j], dpTable[i]
                [j-1])
                        }
                }
        }

        var matchedIdxsA []int
        var matchedIdxsB []int

        // Backtracks the dp table to find the matches of the
    longest lengths
        for n > 0 && m > 0 {
                if seqA[n-1] == seqB[m-1] {
                        matchedIdxsA = append(matchedIdxsA, n-1)
                        matchedIdxsB = append(matchedIdxsB, m-1)
                        n--
                        m--
                } else if dpTable[n-1][m] > dpTable[n][m-1] {
```

```
                        n--
                } else {
                        m--
                }
        }

        slices.Reverse(matchedIdxsA)
        slices.Reverse(matchedIdxsB)

        return matchedIdxsA, matchedIdxsB
}
```

C.7.2 crossover_test.go

```
package geneticalgorithm

import (
        "slices"
        "testing"
)

// slice1 is the first parent slice
var slice1 []int = []int{0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20}

// slice2 is the second parent slice
var slice2 []int = []int{1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21}

func fallbackBuilder(maxVal, maxLen int) func(a, b []int)
([]int, []int) {
        return func(a, b []int) ([]int, []int) {
                return pairMutate(a, b, maxVal, maxLen)
        }
}
```

```
// crossoverTeardown verifies that the slices remain unmodified
func crossoverTeardown(t *testing.T) {
        var slice1Original []int = []int{0, 2, 4, 6, 8, 10, 12,
    14,
    16, 18, 20}
        var slice2Original []int = []int{1, 3, 5, 7, 9, 11, 13,
    15,
    17, 19, 21}
        if !slices.Equal(slice1, slice1Original) {
                t.Logf("Crossover should not affect original slice, %v
        : %v", slice1, slice1Original)
                slice1 = slice1Original
                t.FailNow()
        }
        if !slices.Equal(slice2, slice2Original) {
                t.Logf("Crossover should not affect original slice, %v
        : %v", slice2, slice2Original)
                slice2 = slice2Original
                t.FailNow()
        }
}

// TestReproduce tests the high level reproduce function
//
// Performs many iterations to ensure the randomness of the
function
// doesn't allow for lucky passes.
func TestReproduce(t *testing.T) {
        defer crossoverTeardown(t)
        for range 300 {
                slice3, slice4 := reproduce(slice1, slice2, 21, 15)
```

```
                if len(slice3) > 22 {
                        t.Fatalf("Length should not exceed maxLen, %v :
            %d", slice3, len(slice3))
                }
                if len(slice4) > 22 {
                        t.Fatalf("Length should not exceed maxLen, %v :
            %d", slice4, len(slice4))
                }
                if slices.Equal(slice1, slice3) ||
        slices.Equal(slice2,
        slice3) {
                        t.Fatalf("Child slice should never be a clone of
            parent: %v", slice3)
                }
                if slices.Equal(slice1, slice4) ||
        slices.Equal(slice2,
        slice4) {
                        t.Fatalf("Child slice should never be a clone of
            parent: %v", slice4)
                }
        }
}

// TestTwoPoint tests two point crossover
func TestTwoPoint(t *testing.T) {
        defer crossoverTeardown(t)
        fallback := fallbackBuilder(21, 15)

        for range 100 {
                slice3, slice4 := twoPointCrossover(slice1, slice2,
        fallback)

                if slices.Equal(slice1, slice3) ||
```

```
        slices.Equal(slice2,
        slice3) {
                t.Fatalf("Two Point child slice should never be a
            clone of parent: %v", slice3)
            }
            if slices.Equal(slice1, slice4) ||
        slices.Equal(slice2,
        slice4) {
                t.Fatalf("Two Point child slice should never be a
            clone of parent: %v", slice4)
            }
        }
}

// TestLCS tests longest common subsequence crossover
func TestLCS(t *testing.T) {
        defer crossoverTeardown(t)
        fallback := fallbackBuilder(10, 10)

        tests := []struct {
                name string
                a    []int
                b    []int
        }{
                {
                        name: "with matches",
                        a:    []int{1, 4, 2, 3, 5, 8, 9},
                        b:    []int{1, 2, 3, 6, 8, 9, 10},
                },
                {
                        name: "without matches",
                        a:    slice1,
                        b:    slice2,
```

```
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        for range 100 {
                                slice3, slice4 := lcsCrossover(test.a, test.b,
                fallback)

                                if slices.Equal(test.a, slice3) ||
                slices.Equal(test.b, slice3) {
                                        t.Fatalf("LCS child slice should never be
                    a
                    clone of parent: %v", slice3)
                                }
                                if slices.Equal(test.a, slice4) ||
                slices.Equal(test.b, slice4) {
                                        t.Fatalf("LCS child slice should never be
                    a
                    clone of parent: %v", slice4)
                                }
                        }
                })
        }
}

// TestOnePoint tests one point crossover
func TestOnePoint(t *testing.T) {
        defer crossoverTeardown(t)
        fallback := fallbackBuilder(21, 15)

        for range 100 {
                slice3, slice4 := onePointCrossover(slice1, slice2,
```

```
        fallback)

                if slices.Equal(slice1, slice3) ||
        slices.Equal(slice2,
        slice3) {
                        t.Fatalf("One Point child slice should never be a
            clone of parent: %v", slice3)
                }
                if slices.Equal(slice1, slice4) ||
        slices.Equal(slice2,
        slice4) {
                        t.Fatalf("One Point child slice should never be a
            clone of parent: %v", slice4)
                }
        }
}

// TestEmpty tests empty slice handling for all crossovers
func TestEmpty(t *testing.T) {
        a, b := reproduce([]int{}, []int{}, 10, 10)
        fallback := fallbackBuilder(10, 10)

        if len(a) <= 0 || len(b) <= 0 {
                t.Fatalf("Reproduce should ensure offspring are not of
        0 length")
        }

        c, d := onePointCrossover([]int{}, []int{}, fallback)
        e, f := twoPointCrossover([]int{}, []int{}, fallback)
        g, h := lcsCrossover([]int{}, []int{}, fallback)
        if len(c) != 1 || len(d) != 1 ||
                len(e) != 1 || len(f) != 1 ||
                len(g) != 1 || len(h) != 1 {
```

```
                t.Fatalf("Crossover should fallback to mutate for len
        input == 0")
        }
}
```

C.7.3 genetic.go

```
package geneticalgorithm

import (
        "6CCS3PRJ/thomasholden/passmanager"
        "6CCS3PRJ/thomasholden/testharness"
        "6CCS3PRJ/thomasholden/util"
        "6CCS3PRJ/thomasholden/workerpool"

        "errors"
        "fmt"
        "math/rand"
        "runtime"
        "slices"
        "time"

        "github.com/google/shlex" // Parsing logic should be
    exported as not part of the genetic algorithm
)

// Population manages the genomes of the genetic algorithm.
type Population struct {
        members     [][]int                 // slices of int
    slices, where each int slice is a pass sequence.
        passMan     *passmanager.PassManager // pointer to a
    PassManager.
        maxLength   int                     // the maximum length
```

```
    of any genome at any given time.
        workers     *workerpool.WorkerPool   // pointer to a pool
    of parallelised workers.
        lastSafePop []int                    // copy of the last
    safe population member for comparison.
        convCount   int                      // counter to track
    steps without improvement.
}

const (
        LOADED_POP     float64 = 0.25 // Proportion of population
    to be loaded from file if possible
        SAFE_POP       float64 = 0.25 // Proportion of population
    to apply reproduction & selection.
        MUTATION_STEPS float64 = 1.2  // Number of times to run
    mutation (* population size).

        POP_FILENAME string = "optimal_seqs.jsonl"
)

// CreatePopulation initialized a Population struct and its
variables.
func CreatePopulation(minPopSize, maxLength int, command
string) (*Population, error) {
        workers := runtime.NumCPU() / 2
        runtime.GOMAXPROCS(workers)
        size := ((minPopSize / workers) + 1) * workers
        pop := &Population{}
        pop.members = make([][]int, size)
        pop.passMan = passmanager.GetPassManager()
        pop.maxLength = maxLength
        pop.lastSafePop = []int{}
```

```
        pop.members[0] = pop.passMan.GetO3Int()
        pop.members[0] = pop.members[0][:min(len(pop.members[0]),
    maxLength)]

        loadedMembers, _ := util.LoadSequences(
                POP_FILENAME,
                int(float64(size)*LOADED_POP),
        )

        for i := range len(pop.members) - 1 {
                if len(loadedMembers) > 0 {
                        pop.members[i+1] = loadedMembers[0]
                        loadedMembers = loadedMembers[1:]
                } else {
                        pop.members[i+1] = pop.generateRandomIntSlice()
                }

        }

        commandSplit, err := shlex.Split(command)
        if err != nil {
                return nil, errors.New("command lexing failed: " +
        err.Error())
        }

        pop.workers, err = workerpool.NewWorkerPool(commandSplit,
    workers, size, pop.members[0])
        if err != nil {
                return nil, err
        }

        return pop, nil
}
```

```
// generateRandomIntSlice returns a slice of ints within the
pass range of length [1-p.maxLength].
func (p *Population) generateRandomIntSlice() []int {
        returnSlice := p.passMan.GetO3Int()
        returnSlice = returnSlice[:min(len(returnSlice),
    p.maxLength)]
        if p.maxLength <= 0 {
                return returnSlice
        }

        for range rand.Intn(len(returnSlice)) + 1 {
                returnSlice = mutate(returnSlice, p.passMan.MaxPass,
        p.maxLength)
        }

        return slices.Clip(returnSlice)
}

// nextGeneration executes a single generation of the genetic
algorithm.
//
// This is composed of execution -> elitism -> reproduction ->
mutation.
func (p *Population) nextGeneration() bool {
        for _, member := range p.members {
                p.workers.AddPassSequence(member)
        }
        results := p.workers.StartPool()
        p.sortPop(results)
        for i, result := range results {
                if i < 10 {
                        fmt.Printf("\t%v\n", result)
```

```
                }
        }
        p.members = make([][]int, 0, len(p.members))

        safeMarker := int(SAFE_POP * float64(len(results)))
        reproMarker := p.elitism(results, safeMarker)

        p.reproduction(reproMarker)

        p.mutation(safeMarker)

        return p.testConvergence()
}

// elitism selects the top k population members depending on
results.
func (p *Population) elitism(results []workerpool.Result,
marker int) int {
        reproMarker := -1
        for i := range marker {
                if results[i].Score < util.MAX_FLOAT64 {
                        p.members = append(p.members,
            slices.Clone(results[i].PassSeq))
                        reproMarker = i
                }
        }
        if reproMarker < 0 {
                panic("error in GA: all sequences failing")
        }
        return reproMarker
}

// reproduction fills the remaining population via
```

```
reproduction among elites.
func (p *Population) reproduction(marker int) {
        for len(p.members) < cap(p.members) {
                passPtr1 := rand.Int() % (marker + 1)
                passPtr2 := rand.Int() % (marker + 1)

                newSeq1, newSeq2 := reproduce(
                        p.members[passPtr1],
                        p.members[passPtr2],
                        p.passMan.MaxPass,
                        p.maxLength,
                )
                p.members = append(p.members, newSeq1)
                if len(p.members) < cap(p.members) {
                        p.members = append(p.members, newSeq2)
                }
        }
}

// mutation applies a random mutation to the reproduced
population.
func (p *Population) mutation(marker int) {
        steps := int(float64(len(p.members)) * MUTATION_STEPS)
        for range steps {
                popPtr := rand.Intn(len(p.members)-marker-1) + marker
        + 1
                p.members[popPtr] = mutate(p.members[popPtr],
        p.passMan.MaxPass, p.maxLength)
        }
}

// RunNGenerations executes nextGeneration n times.
func (p *Population) RunNGenerations(n int) int {
```

```
        for i := range n {
                converged := p.nextGeneration()

                if converged && i > 10 {
                        return i + 1
                }
        }
        return n
}

// RunGenerationsForDuration executes nextGeneration until the
duration is exceeded.
func (p *Population) RunGenerationsForDuration(d
time.Duration) int {
        startTime := time.Now()
        i := 0

        for ; time.Since(startTime) < d; i++ {
                minETA, maxETA := util.GetMinMaxTimes(startTime, d, i,
        p.convCount)
                fmt.Printf(
                        "\n[Gen %d] Est. Remaining Time: %v (min) to %v
            (max)\n", i,
                        minETA.Round(time.Second),
            maxETA.Round(time.Second),
                )

                converged := p.nextGeneration()

                if converged && i > 10 {
                        return i + 1
                }
        }
```

```
        return i
}

// sortPop sorts the population depending on their achieved
results.
func (p *Population) sortPop(results []workerpool.Result) {
        slices.SortFunc(results, func(a, b workerpool.Result) int {
                if a.Score < b.Score {
                        return -1
                } else if a.Score > b.Score {
                        return 1
                }
                return 0
        })
}

// testConvergence evaluates whether the population has
converged yet.
//
// This is decided based on whether the population elites have
improved over 10 generations.
func (p *Population) testConvergence() bool {
        safeMarker := int(SAFE_POP * float64(len(p.members)))
        safeMarker = max(safeMarker-1, 0)
        currentSafePop := p.members[safeMarker]

        if slices.Equal(p.lastSafePop, currentSafePop) {
                p.convCount += 1
        } else {
                p.convCount = 0
        }
        p.lastSafePop = append([]int{}, currentSafePop...)
```

```
        return p.convCount >= 10
}

// Best returns the best performing population member's passes
and execution time.
func (p *Population) Best() (string, error) {
        validOptimal := p.passMan.MakeValid(p.members[0])
        _, _, _, _, err :=
    testharness.CompileAndExecute(validOptimal, 0, true,
    p.workers.SourceCmd)

        err = errors.Join(util.SaveSequence(POP_FILENAME,
    p.members[0]))

        return validOptimal, err
}
```

C.7.4 genetic_test.go

```
package geneticalgorithm

import (
        "os"
        "reflect"
        "slices"
        "testing"

        "6CCS3PRJ/thomasholden/passmanager"
        "6CCS3PRJ/thomasholden/util"
        "6CCS3PRJ/thomasholden/workerpool"
)

// TestestreatePopulation tests the creation of a population.
```

```
func TestCreatePopulation(t *testing.T) {
        tempDir := t.TempDir()
        dummyFile := tempDir + "/hello_world.c"
        err := os.WriteFile(dummyFile, []byte("int main() { return
    0; }"), 0644)
        if err != nil {
                t.Skip("Failed to create dummy file for testing: " +
        err.Error())
        }

        tests := []struct {
                name        string
                minPopSize int
                maxLength   int
                command     string
                expectErr   bool
        }{
                {
                        name:        "valid command and sizes",
                        minPopSize: 10,
                        maxLength:   50,
                        command:     "clang " + dummyFile,
                        expectErr:   false,
                },
                {
                        name:        "invalid shell command",
                        minPopSize: 10,
                        maxLength:   50,
                        command:     "clang 'hello_world.c",
                        expectErr:   true,
                },
                {
                        name:        "invalid file path",
```

```
                minPopSize: 10,
                maxLength:  50,
                command:    "clang hello_world.c",
                expectErr:  true,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                pop, err := CreatePopulation(test.minPopSize,
    test.maxLength, test.command)

                if test.expectErr {
                        if err == nil {
                                t.Fatalf("expected error for command %q,
            got nil", test.command)
                        }
                } else {
                        if err == nil && pop != nil {
                                if pop.maxLength != test.maxLength {
                                        t.Errorf("expected maxLength %d, got
                %d", test.maxLength, pop.maxLength)
                                }
                                if len(pop.members) == 0 {
                                        t.Errorf("population members should be
                initialized, got length 0")
                                }
                        } else if err != nil {
                                t.Fatalf("unexpected error: %s", err)
                        }
                }
        })
}
```

```
}

// TestGenerateRandomIntSlice tests the generation of a random
int slice.
func TestGenerateRandomIntSlice(t *testing.T) {
        tests := []struct {
                name      string
                passMan   *passmanager.PassManager
                maxLength int
        }{
                {
                        name:      "zero length",
                        passMan:   passmanager.GetPassManager(),
                        maxLength: 0,
                },
                {
                        name:      "small length",
                        passMan:   passmanager.GetPassManager(),
                        maxLength: 5,
                },
                {
                        name:      "~O3 length",
                        passMan:   passmanager.GetPassManager(),
                        maxLength: 75,
                },
                {
                        name:      "big length",
                        passMan:   passmanager.GetPassManager(),
                        maxLength: 150,
                },
        }

        for _, test := range tests {
```

```
                t.Run(test.name, func(t *testing.T) {
                        pop := &Population{
                                passMan:   test.passMan,
                                maxLength: test.maxLength,
                        }

                        returnSlice := pop.generateRandomIntSlice()
                        if len(returnSlice) > pop.maxLength {
                                t.Errorf("expected max length %d, got %d",
                pop.maxLength, len(returnSlice))
                        }
                })
        }
}

// TestElitism tests the elitism function.
func TestElitism(t *testing.T) {
        tests := []struct {
                name        string
                results     []workerpool.Result
                marker      int
                expectPanic bool
                expectedLen int
        }{
                {
                        name: "success with valid results",
                        results: []workerpool.Result{
                                {Score: 1.0, PassSeq: []int{1, 2}},
                                {Score: 2.5, PassSeq: []int{3, 4}},
                        },
                        marker:      2,
                        expectPanic: false,
                        expectedLen: 2,
```

```
        },
        {
                name: "all results failing (panic)",
                results: []workerpool.Result{
                        {Score: util.MAX_FLOAT64, PassSeq: []int{1}},
                },
                marker:      1,
                expectPanic: true,
                expectedLen: 0,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                p := &Population{
                        members: make([][]int, 0),
                }

                if test.expectPanic {
                        defer func() {
                                if r := recover(); r == nil {
                                        t.Errorf("expected elitism to panic
                when all sequences fail, but it did
                not")
                                }
                        }()
                }

                reproMarker := p.elitism(test.results, test.marker)

                if !test.expectPanic {
                        if len(p.members) != test.expectedLen {
                                t.Errorf("expected members length %d, got
```

```
                    %d", test.expectedLen, len(p.members))
                            }
                            if reproMarker < 0 {
                                    t.Errorf("expected valid reproMarker, got
                    %d", reproMarker)
                            }
                        }
                })
        }
}

// TestReproduction tests the reproduction function.
func TestReproduction(t *testing.T) {
        p := &Population{
                members:    make([][]int, 2, 5),
                passMan:    &passmanager.PassManager{MaxPass: 10},
                maxLength: 10,
        }
        p.members[0] = []int{1, 2, 3}
        p.members[1] = []int{4, 5, 6}

        t.Run("fills capacity", func(t *testing.T) {
                p.reproduction(1)
                if len(p.members) != 5 {
                        t.Errorf("expected members length 5, got %d",
            len(p.members))
                }
        })
}

// TestMutation tests the mutation function.
func TestMutation(t *testing.T) {
        p := &Population{
```

```
            members:    make([][]int, 5),
            passMan:    &passmanager.PassManager{MaxPass: 20},
            maxLength: 10,
        }
        p.members[0] = []int{1, 2, 3}
        p.members[1] = []int{4, 5, 6}
        p.members[2] = []int{1, 2, 3}
        p.members[3] = []int{4, 5, 6}
        p.members[4] = []int{1, 2, 3}

        p.mutation(1)

        t.Run("modifies correct slice range", func(t *testing.T) {
                if len(p.members) != 5 {
                        t.Errorf("expected members length 5, got %d",
            len(p.members))
                }
                count := 0
                for i, seq := range p.members[2:] {
                        if slices.Equal(seq, p.members[i%2]) {
                                count++
                        }
                }
                if count == 3 {
                        t.Errorf("expected at least 1 mutation, got None")
                }
        })

        t.Run("doesn't modify incorrect slice range", func(t
    *testing.T) {
                if !slices.Equal(p.members[0], []int{1, 2, 3}) {
                        t.Errorf("should not modify elites, but modified
            at 0")
```

```
                }
                if !slices.Equal(p.members[1], []int{4, 5, 6}) {
                        t.Errorf("should not modify elites, but modified
            at 1")
                }
        })
}

// TestSortPopulation tests the sortPopulation function.
func TestSortPopulation(t *testing.T) {
        tests := []struct {
                name     string
                input    []workerpool.Result
                expected []workerpool.Result
        }{
                {
                        name:     "already sorted",
                        input:    []workerpool.Result{{Score: 1.0},
            {Score: 2.0}, {Score: 3.0}},
                        expected: []workerpool.Result{{Score: 1.0},
            {Score: 2.0}, {Score: 3.0}},
                },
                {
                        name:     "reverse sorted",
            input:    []workerpool.Result{{Score: 3.0},
            {Score: 2.0}, {Score: 1.0}},
                        expected: []workerpool.Result{{Score: 1.0},
            {Score: 2.0}, {Score: 3.0}},
                },
                {
                        name:     "with equals",
                        input:    []workerpool.Result{{Score: 2.0},
            {Score: 1.0}, {Score: 2.0}},
```

```
                        expected: []workerpool.Result{{Score: 1.0},
            {Score: 2.0}, {Score: 2.0}},
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        p := &Population{}
                        p.sortPop(test.input)

                        if !reflect.DeepEqual(test.input, test.expected) {
                                t.Errorf("expected %v, got %v", test.expected,
                test.input)
                        }
                })
        }
}

// TestConvergenceTest tests convergence is calculated
correctly.
func TestConvergenceTest(t *testing.T) {
        tests := []struct {
                name           string
                members        [][]int
                lastSafePop    []int
                initialConv    int
                expectedConv   int
                expectedResult bool
        }{
                {
                        name:           "diverging population",
                        members:        [][]int{{1, 2}, {3, 4}, {5, 6},
            {7, 8}},
```

```
                lastSafePop:    []int{9, 9},
                initialConv:    3,
                expectedConv:   0,
                expectedResult: false,
        },
        {
                name:           "converging population",
                members:        [][]int{{1, 2}, {3, 4}, {5, 6},
    {7, 8}},
                lastSafePop:    []int{1, 2},
                initialConv:    2,
                expectedConv:   3,
                expectedResult: false,
        },
        {
                name:           "reached convergence threshold",
                members:        [][]int{{1, 2}, {3, 4}, {5, 6},
    {7, 8}},
                lastSafePop:    []int{1, 2},
                initialConv:    9,
                expectedConv:   10,
                expectedResult: true,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                p := &Population{
                        members:     test.members,
                        lastSafePop: test.lastSafePop,
                        convCount:   test.initialConv,
                }
```

```
                result := p.testConvergence()

                if p.convCount != test.expectedConv {
                        t.Errorf("expected convCount %d, got %d",
            test.expectedConv, p.convCount)
                }
                if result != test.expectedResult {
                        t.Errorf("expected result %v, got %v",
            test.expectedResult, result)
                }

                safeMarker := int(SAFE_POP *
        float64(len(p.members)))
                safeMarker = max(safeMarker-1, 0)
                if !slices.Equal(p.lastSafePop,
        p.members[safeMarker]) {
                        t.Errorf("expected lastSafePop to be updated
            to %v, got %v", p.members[safeMarker],
            p.lastSafePop)
                }
            })
        }
}
```

C.7.5 mutation.go

```
package geneticalgorithm

import (
        "math/rand"
        "slices"
)
```

```
// mutate applies a random mutation to a single population
member.
func mutate(a []int, maxVal, maxLen int) []int {
        if len(a) == 0 {
                newPass := rand.Int() % maxVal
                a = append(a, newPass)
                return a[:min(len(a), maxLen)]
        }
        point := rand.Int() % len(a)

        randf := rand.Float64()
        if randf < 0.33 { // Replace pass
                newPass := a[point]
                for newPass == a[point] {
                        newPass = rand.Int() % maxVal
                }
                a[point] = newPass
        } else if randf > 0.66 && len(a) < maxLen { // Add pass
                newPass := rand.Int() % maxVal
                a = slices.Insert(a, point, newPass)

        } else { // Remove pass
                a = append(a[:point], a[point+1:]...)
        }

        return a
}
```

C.7.6 mutation_test.go

```
package geneticalgorithm

import (
```

```
        "slices"
        "testing"
)

// TestMutate tests the mutation function.
func TestMutate(t *testing.T) {
        tests := []struct {
                name   string
                seq    []int
                maxVal int
                maxLen int
        }{
                {
                        name:   "empty input and length",
                        seq:    []int{},
                        maxVal: 10,
                        maxLen: 0},
                {
                        name:   "empty input but not length",
                        seq:    []int{},
                        maxVal: 10,
                        maxLen: 5},
                {
                        name:   "standard input",
                        seq:    []int{1, 2, 3, 4, 5},
                        maxVal: 10,
                        maxLen: 5},
        }

        for range 10 { // Repeat to avoid lucky runs due to rand
                for _, test := range tests {
                        t.Run(test.name, func(t *testing.T) {
                                seq := mutate(slices.Clone(test.seq),
```

```
                test.maxVal, test.maxLen)
                        if len(seq) > test.maxLen {
                                t.Errorf("Length should not exceed maxLen,
                    %v : %d", seq, len(seq))
                        }
                        if slices.Equal(seq, test.seq) &&
                (len(test.seq) > 0 || len(seq) > 0) {
                                t.Errorf("Child slice should never be a
                    clone of parent: %v", seq)
                        }
                        if len(seq) > 0 && slices.Max(seq) >
                test.maxVal {
                                t.Errorf("Max value in sequence, %v,
                    should not exceed maxVal, %d", seq,
                    test.maxVal)
                        }
                    })
                }
        }
}
```

C.8 passmanager

C.8.1 passes.go

```
package passmanager

import (
        "6CCS3PRJ/thomasholden/util"
        "bytes"
)

// PassManager initializes and contains pass data.
```

```
type PassManager struct {
        util.BidirectionalMap[string, int]
        MaxPass int // MaxPass is the highest index/size of the
    mapping in PassManager

        moduleRange   *util.Range[int]
        cgsccRange    *util.Range[int]
        functionRange *util.Range[int]
        loopRange     *util.Range[int]
}

var passManager *PassManager = nil

// GetPassManager returns a PassManager singleton.
func GetPassManager() *PassManager {
        if passManager == nil {
                passManager = newPassManager()
        }
        return passManager
}

// NewPassManager initializes and returns a PassManager
pointer.
func newPassManager() *PassManager {
        manager := &PassManager{}
        manager.loadPasses()

        return manager
}

// loadPasses adds all passes to the mapping
func (pm *PassManager) loadPasses() {
        passList := getPasses()
```

```
        passList = append(passList, getO3String()...)

        assignments := 0
        for _, pass := range passList {
                if _, ok := pm.Get(pass); !ok { // If the pass is not
        already in the map
                        pm.Put(pass, assignments)
                        assignments++
                }
        }
        pm.MaxPass = assignments

        pm.initRanges()
}

// initRanges initialises the pass ranges
func (pm *PassManager) initRanges() {
        alwaysinline, ok1 := pm.Get("always-inline")
        wholeprogramdevirt, ok2 := pm.Get("wholeprogramdevirt")
        argpromotion, ok3 := pm.Get("argpromotion")
        coroAnnotationElide, ok4 := pm.Get("coro-annotation-elide")
        adce, ok5 := pm.Get("adce")
        wasmEhPrepare, ok6 := pm.Get("wasm-eh-prepare")
        loopFlatten, ok7 := pm.Get("loop-flatten")
        loopVersioningLicm, ok8 := pm.Get("loop-versioning-licm")

        if !ok1 || !ok2 || !ok3 || !ok4 || !ok5 || !ok6 || !ok7 ||
    !ok8 {
                panic("Could not find pass ranges")
        }

        pm.moduleRange = util.NewRange(alwaysinline,
    wholeprogramdevirt)
```

```
        pm.cgsccRange = util.NewRange(argpromotion,
    coroAnnotationElide)
        pm.functionRange = util.NewRange(adce, wasmEhPrepare)
        pm.loopRange = util.NewRange(loopFlatten,
    loopVersioningLicm)
}

// Passes extracted from
// https://github.com/llvm/llvm-project/blob/
d162c91c01a66e4af0af190044961e60db0eeb3d/llvm/lib/Passes/
PassRegistry.def
//
// All passes including keywords analysis, print, view, dot,
verify, callback, eval, no-op, prof,
// pgo, trigger, and invalidate are removed.
// Other miscellaneous and fragile passes removed e.g.
helloworld
func getPasses() []string {
        return []string{
                "always-inline", // start module passes - <pass>
                "assign-guid",
                "attributor",
                "attributor-light",
                "called-value-propagation",
                "canonicalize-aliases",
                // "cg-profile", // requires profile (probably)
                "constmerge",
                "coro-cleanup",
                "coro-early",
                "cross-dso-cfi",
                // "ctx-instr-gen", // not beneficial for performance
        + fragile
                // "ctx-prof-flatten",
```

```
        // "ctx-prof-flatten-prethinlink", // both require
profile
        "noinline-nonprevailing",
        "deadargelim",
        // "declare-runtime-libcalls", // unknown module pass
        // "dfsan", // requires libraries
        "dxil-upgrade",
        "elim-avail-extern",
        "extract-blocks",
        "expand-variadics",
        "forceattrs",
        //"function-import", requires file
        "global-merge-func",
        "globalopt",
        "globalsplit",
        /*"hipstdpar-interpose-alloc",
        "hipstdpar-math-fixup",
        "hipstdpar-select-accelerator-code",*/ // All platform
specific
        "hotcoldsplit",
        "inferattrs",
        //"inliner-ml-advisor-release", // too fragile
        "inliner-wrapper",
        "inliner-wrapper-no-mandatory-first",
        // "insert-gcov-profiling", // requires profile
(probably)
        // "instrprof", // requires profile (probably)
        // "ctx-instr-lower", // not beneficial for
performance + fragile
        "iroutliner",
        "jmc-instrumenter",
        "lower-emutls",
        "lower-global-dtors",
```

```
        "lower-ifunc",
        "simplify-type-tests",
        "lowertypetests",
        "fatlto-cleanup",
        // "pgo-force-function-attrs", // requires profile
        // "memprof-context-disambiguation", // requires
profile
        // "memprof-remove-attributes", // unknown module pass
        //"memprof-module", // requires memprof data
        "mergefunc",
        "module-inline",
        // "norecurse-lto-inference", // unknown module pass
        // "nsan", // requires libraries
        "openmp-opt",
        "openmp-opt-postlink",
        "partial-inliner",
        // "pgo-icall-prom", // requires profile
        // "pgo-instr-gen", // requires profile
        //"pgo-instr-use", // requires profile
        "pre-isel-intrinsic-lowering",
        "pseudo-probe",
        "pseudo-probe-update",
        "rel-lookup-table-converter",
        "rewrite-statepoints-for-gc",
        "rewrite-symbols",
        "rpo-function-attrs",
        // "rtsan", // requires libraries
        //"sample-profile", // requires profile
        "sancov-module",
        "sanmd-module",
        "scc-oz-module-inliner",
        "shadow-stack-gc-lowering",
        // "tsan-module", // requires libraries
```

```
        // "tysan", // requires libraries
        "wholeprogramdevirt", // end module passes
        "argpromotion",       // start cgscc passes -
cgscc(<pass>)
        "attributor-cgscc",
        "attributor-light-cgscc",
        "openmp-opt-cgscc",
        // start O3 cgscc passes
        "inline",
        "function-attrs<skip-non-recursive-function-attrs>",
        "function-attrs",
        // end O3 cgscc passes
        "coro-annotation-elide", // end cgscc passes
        "adce",                  // start function passes
function(<pass>)
        "add-discriminators",
        "aggressive-instcombine",
        "alignment-from-assumptions",
        "assume-builder",
        "assume-simplify",
        "atomic-expand",
        "bdce",
        "break-crit-edges",
        "callbr-prepare",
        "callsite-splitting",
        "chr",
        "codegenprepare",
        "complex-deinterleaving",
        "consthoist",
        "constraint-elimination",
        "coro-elide",
        "correlated-propagation",
        "dce",
```

```
        "declare-to-assign", // Seems to be a module pass but
is declared as a function pass
        // Transformation is the same whether wrapped or not
        "dfa-jump-threading",
        "div-rem-pairs",
        "dse",
        "dwarf-eh-prepare",
        "expand-large-div-rem",
        "expand-memcmp",
        "expand-reductions",
        "extra-vector-passes",
        "fix-irreducible",
        "flatten-cfg",
        "float2int",
        "gc-lowering",
        "guard-widening",
        "gvn-hoist",
        "gvn-sink",
        "indirectbr-expand",
        "infer-address-spaces",
        "infer-alignment",
        "inject-tli-mappings",
        "instsimplify",
        "interleaved-access",
        "interleaved-load-combine",
        "irce",
        "jump-threading",
        "jump-table-to-switch",
        "kcfi",
        "lcssa",
        "libcalls-shrinkwrap",
        "load-store-vectorizer",
        "loop-data-prefetch",
```

```
"loop-distribute",
"loop-fusion",
"loop-load-elim",
"loop-simplify",
"loop-sink",
"loop-versioning",
"lower-atomic",
"lower-constant-intrinsics",
"lower-expect",
"lower-guard-intrinsic",
"lower-invoke",
"lower-switch",
"lower-widenable-condition",
"make-guards-explicit",
"mem2reg",
"memcpyopt",
// "memprof", // requires profile
"mergeicmps",
"mergereturn",
"move-auto-init",
"nary-reassociate",
"newgvn",
"objc-arc",
"objc-arc-contract",
"objc-arc-expand",
"partially-inline-libcalls",
// "pgo-memop-opt", // requires profile
"place-safepoints",
"reassociate",
"redundant-dbg-inst-elim",
"replace-with-veclib",
"reg2mem",
"sandbox-vectorizer",
```

```
    "scalarize-masked-mem-intrin",
    "sccp",
    "select-optimize",
    "separate-const-offset-from-gep",
    "sink",
    "simplifycfg",
    "sjlj-eh-prepare",
    "slp-vectorizer",
    "slsr",
    "tailcallelim",
    // "tsan", // requires libraries
    "typepromotion",
    "unify-loop-exits",
    "unreachableblockelim",
    "vector-combine",
    // start O3 specific functional passes
    "instcombine<max-iterations=1;no-verify-fixpoint>",
    "ee-instrument<>",
    "simplifycfg<bonus-inst-threshold=1;" +
            "no-forward-switch-cond;no-switch-range-to-icmp;no-
switch-to-lookup;keep-loops;" +
            "no-hoist-common-insts;no-hoist-loads-stores-with-
cond-faulting;no-sink-common-insts;" +
            "speculate-blocks;simplify-cond-branch;no-
speculate-unpredictables>,sroa<modify-cfg>",
    "early-cse<>",
    "simplifycfg<bonus-inst-threshold=1;" +
            "no-forward-switch-cond;switch-range-to-icmp;no-
switch-to-lookup;keep-loops;" +
            "no-hoist-common-insts;no-hoist-loads-stores-with-
cond-faulting;no-sink-common-insts;" +
            "speculate-blocks;simplify-cond-branch;no-
speculate-unpredictables>",
```

```
        "loop-vectorize<no-interleave-forced-only;no-vectorize-
forced-only;>",
        "simplifycfg<bonus-inst-threshold=1;" +
                "forward-switch-cond;switch-range-to-icmp;switch-
    to-lookup;no-keep-loops;" +
                "hoist-common-insts;no-hoist-loads-stores-with-
    cond-faulting;sink-common-insts;" +
                "speculate-blocks;simplify-cond-branch;no-
    speculate-unpredictables>",
        "vector-combine",
        "loop-unroll<O3>",
        "transform-warning",
        "sroa<preserve-cfg>",
        "loop-mssa(licm<allowspeculation>)",
        "simplifycfg<bonus-inst-threshold=1;" +
                "no-forward-switch-cond;switch-range-to-icmp;no-
    switch-to-lookup;keep-loops;" +
                "no-hoist-common-insts;hoist-loads-stores-with-
    cond-faulting;no-sink-common-insts;" +
                "speculate-blocks;simplify-cond-branch;speculate-
    unpredictables>",
        // end O3 specific functional passes
        "wasm-eh-prepare", // end function passes
        "loop-flatten",    // start loopnest passes -
function(loop(<pass>))
        "loop-interchange",
        "loop-unroll-and-jam", // end loopnest passes
        "canon-freeze",        // start loop passes -
function(loop(<pass>))
        "guard-widening",
        "extra-simple-loop-unswitch-passes",
        "indvars",
        "loop-bound-split",
```

```
                "loop-deletion",
                "loop-idiom",
                "loop-idiom-vectorize",
                "loop-instsimplify",
                // "loop-mssa(licm<allowspeculation>)", // Is a pass
        manager
                "loop-predication",
                "loop-reduce",
                "loop-term-fold",
                "loop-simplifycfg",
                "loop-unroll-full",
                // start O3 loop passes
                "loop-rotate<header-duplication;no-prepare-for-lto>",
                // end O3 loop passes
                "loop-versioning-licm", // end loop passes
        }
}

// Returns the list of strings making up O3
// Has had minor tweaks to remove passes intended for only
debugging
func getO3String() []string {
        return []string{
                "annotation2metadata",
                "forceattrs",
                "declare-to-assign",
                "inferattrs",
                "coro-early",
                "ee-instrument<>",
                "lower-expect",
                "simplifycfg<bonus-inst-threshold=1;" +
                        "no-forward-switch-cond;no-switch-range-to-icmp;no-
            switch-to-lookup;keep-loops;" +
```

```
                "no-hoist-common-insts;no-hoist-loads-stores-with-
    cond-faulting;no-sink-common-insts;" +
                "speculate-blocks;simplify-cond-branch;no-
    speculate-unpredictables>,sroa<modify-cfg>",
        "early-cse<>",
        "callsite-splitting",
        "openmp-opt",
        "ipsccp",
        "called-value-propagation",
        "globalopt",
        "mem2reg",
        "instcombine<max-iterations=1;no-verify-fixpoint>",
        "simplifycfg<bonus-inst-threshold=1;no-forward-switch-
cond;switch-range-to-icmp;" +
                "no-switch-to-lookup;keep-loops;no-hoist-common-
    insts;" +
                "no-hoist-loads-stores-with-cond-faulting;no-sink-
    common-insts;speculate-blocks;" +
                "simplify-cond-branch;no-speculate-
    unpredictables>",
        "always-inline",
        "require<globals-aa>",
        "function(invalidate<aa>)",
        "require<profile-summary>",
        "cgscc(devirt<4>(inline,function-attrs<skip-non-
recursive-function-attrs>,argpromotion," +
                "openmp-opt-cgscc,function<eager-inv;no-
    rerun>(sroa<modify-cfg>,early-cse<memssa>," +
                "speculative-execution<only-if-divergent-
    target>,jump-threading,correlated-propagation," +
                "simplifycfg<bonus-inst-threshold=1;no-forward-
    switch-cond;switch-range-to-icmp;" +
                "no-switch-to-lookup;keep-loops;no-hoist-common-
```

```
insts;" +
            "no-hoist-loads-stores-with-cond-faulting;no-sink-
common-insts;speculate-blocks;" +
            "simplify-cond-branch;no-speculate-
unpredictables>,instcombine<max-iterations=1;" +
            "no-verify-fixpoint>,aggressive-
instcombine,libcalls-shrinkwrap,tailcallelim," +
            "simplifycfg<bonus-inst-threshold=1;no-forward-
switch-cond;switch-range-to-icmp;" +
            "no-switch-to-lookup;keep-loops;no-hoist-common-
insts;" +
            "no-hoist-loads-stores-with-cond-faulting;no-sink-
common-insts;speculate-blocks;" +
            "simplify-cond-branch;no-speculate-
unpredictables>,reassociate,constraint-
elimination," +
            "loop-mssa(loop-instsimplify,loop-
simplifycfg,licm<no-allowspeculation>," +
            "loop-rotate<header-duplication;no-prepare-for-
lto>,licm<allowspeculation>," +
            "simple-loop-
unswitch<nontrivial;trivial>),simplifycfg<bonus-
inst-threshold=1;" +
            "no-forward-switch-cond;switch-range-to-icmp;no-
switch-to-lookup;keep-loops;" +
            "no-hoist-common-insts;no-hoist-loads-stores-with-
cond-faulting;no-sink-common-insts;" +
            "speculate-blocks;simplify-cond-branch;no-
speculate-unpredictables>," +
            "instcombine<max-iterations=1;no-verify-
fixpoint>,loop(loop-idiom,indvars," +
            "extra-simple-loop-unswitch-passes,loop-
deletion,loop-unroll-full),sroa<modify-cfg>," +
```

```
                "vector-combine,mldst-motion<no-split-footer-
    bb>,gvn<>,sccp,bdce," +
                "instcombine<max-iterations=1;no-verify-
    fixpoint>,jump-threading,correlated-propagation," +
                "adce,memcpyopt,dse,move-auto-init,loop-
    mssa(licm<allowspeculation>),coro-elide," +
                "simplifycfg<bonus-inst-threshold=1;no-forward-
    switch-cond;switch-range-to-icmp;" +
                "no-switch-to-lookup;keep-loops;hoist-common-
    insts;" +
                "no-hoist-loads-stores-with-cond-faulting;sink-
    common-insts;speculate-blocks;" +
                "simplify-cond-branch;no-speculate-
    unpredictables>,instcombine<max-iterations=1;" +
                "no-verify-fixpoint>),function-attrs,coro-
    split,coro-annotation-elide))",
        "deadargelim",
        "coro-cleanup",
        "globalopt",
        "globaldce",
        "elim-avail-extern",
        "rpo-function-attrs",
        "recompute-globalsaa",
        "float2int",
        "lower-constant-intrinsics",
        "chr",
        "loop-rotate<header-duplication;no-prepare-for-lto>",
        "loop-deletion",
        "loop-distribute",
        "inject-tli-mappings",
        "loop-vectorize<no-interleave-forced-only;no-vectorize-
forced-only;>",
        "infer-alignment",
```

```
        "loop-load-elim",
        "instcombine<max-iterations=1;no-verify-fixpoint>",
        "simplifycfg<bonus-inst-threshold=1;forward-switch-
cond;switch-range-to-icmp;" +
                "switch-to-lookup;no-keep-loops;hoist-common-
    insts;no-hoist-loads-stores-with-cond-faulting;" +
                "sink-common-insts;speculate-blocks;simplify-cond-
    branch;no-speculate-unpredictables>",
        "slp-vectorizer",
        "vector-combine",
        "instcombine<max-iterations=1;no-verify-fixpoint>",
        "loop-unroll<O3>",
        "transform-warning",
        "sroa<preserve-cfg>",
        "infer-alignment",
        "instcombine<max-iterations=1;no-verify-fixpoint>",
        "loop-mssa(licm<allowspeculation>)",
        "alignment-from-assumptions",
        "loop-sink",
        "instsimplify",
        "div-rem-pairs",
        "tailcallelim",
        "simplifycfg<bonus-inst-threshold=1;no-forward-switch-
cond;switch-range-to-icmp;" +
                "no-switch-to-lookup;keep-loops;no-hoist-common-
    insts;" +
                "hoist-loads-stores-with-cond-faulting;no-sink-
    common-insts;speculate-blocks;" +
                "simplify-cond-branch;speculate-unpredictables>",
        "globaldce",
        "constmerge",
        "cg-profile",
        "rel-lookup-table-converter",
```

```
                "function(annotation-remarks)",
        }
}

// GetO3Int returns an int slice where each O3 pass is
transformed to its integer representation.
func (pm *PassManager) GetO3Int() []int {
        return util.ForEach(getO3String(), pm.PassToInt)
}

// PassToInt returns the int mapped to by the string in the
PassManager.
func (pm *PassManager) PassToInt(s string) int {
        n, _ := pm.Get(s)
        return n
}

// PassToInt returns the string mapped to by the int in the
PassManager.
func (pm *PassManager) IntToPass(n int) string {
        s, _ := pm.GetInverse(n)
        return s
}

// MakeValid transforms a list of ints, representing passes,
into a valid opt pipeline.
func (pm *PassManager) MakeValid(seq []int) string {
        const (
                INMODULE  byte   = iota
                INFUNC    byte   = iota
                INCGSCC   byte   = iota
                INLOOP    byte   = iota
                FUNCTEXT  string = "function<eager-inv>("
```

```
        CGSCCTEXT string = "cgscc(devirt<4>("
        LOOPTEXT  string = "function<eager-inv>(loop("
    )

    if len(seq) == 0 {
        return ""
    }

    // getClosing returns the closing string for the current
scope
    getClosing := func(scope byte) string {
        if scope == INFUNC {
            return ")"
        }
        return "))"
    }

    var buffer bytes.Buffer
    currentScope := INMODULE

    firstVal := seq[0]
    if pm.cgsccRange.Within(firstVal) {
        buffer.WriteString(CGSCCTEXT)
        currentScope = INCGSCC
    } else if pm.functionRange.Within(firstVal) {
        buffer.WriteString(FUNCTEXT)
        currentScope = INFUNC
    } else if pm.loopRange.Within(firstVal) {
        buffer.WriteString(LOOPTEXT)
        currentScope = INLOOP
    }
    buffer.WriteString(pm.IntToPass(firstVal))
```

```
        var next string
        for _, val := range seq[1:] {
                if pm.cgsccRange.Within(val) {
                        next, currentScope = pm.nextAppend(val,
            currentScope, INMODULE, INCGSCC, CGSCCTEXT,
            getClosing)
                } else if pm.functionRange.Within(val) {
                        next, currentScope = pm.nextAppend(val,
            currentScope, INMODULE, INFUNC, FUNCTEXT,
            getClosing)
                } else if pm.loopRange.Within(val) {
                        next, currentScope = pm.nextAppend(val,
            currentScope, INMODULE, INLOOP, LOOPTEXT,
            getClosing)
                } else {
                        next, currentScope = pm.nextAppend(val,
            currentScope, INMODULE, INMODULE, "", getClosing)
                }
                buffer.WriteString(next)
        }

        if currentScope != INMODULE {
                buffer.WriteString(getClosing(currentScope))
        }

        return buffer.String()
}

// nextAppend manages the logic for appending a pass based
upon the previous pass.
func (pm *PassManager) nextAppend(pass int, currentScope,
defaultScope, desiredScope byte,
        prefix string, getClosing func(byte) string) (string,
```

```
    byte) {

        retString := ""

        switch currentScope {
        // If we are in the same scope as the current pass
        case desiredScope:
                retString += ","
        // If we are in the default scope (module) and need to
    move into the current pass' scope
        case defaultScope:
                retString += "," + prefix
        // If we are in a nested scope, but have to move to a
    different nested scope
        default:
                retString += getClosing(currentScope) + "," + prefix
        }
        retString += pm.IntToPass(pass)
        currentScope = desiredScope

        return retString, currentScope
}
```

C.8.2 passes_test.go

```
package passmanager

import (
        "math/rand/v2"
        "slices"
        "testing"
)
```

```
// basicMutate applies a random mutation to a slice for
testing.
func basicMutate(a []int, maxVal, maxLen int) []int {
        point := rand.Int() % len(a)

        newPass := a[point]
        for newPass == a[point] {
                newPass = rand.Int() % maxVal
        }
        a[point] = newPass

        return a
}

func TestGetPassManager(t *testing.T) {
        if GetPassManager() == nil {
                t.Fatal("GetPassManager returned nil")
        }
        passMan := GetPassManager()
        if passMan != GetPassManager() {
                t.Errorf("GetPassManager returned different instances")
        }
        if passMan.MaxPass <= 0 {
                t.Errorf("Expected MaxPass > 0, got %v",
        passMan.MaxPass)
        }
        if passMan.cgsccRange == nil ||
                passMan.moduleRange == nil ||
                passMan.functionRange == nil ||
                passMan.loopRange == nil {
                t.Errorf("Expected non-nil ranges, got nil")
        }
}
```

```
func TestLoadPasses(t *testing.T) {
        passMan := &PassManager{}
        passMan.loadPasses()

        for i := range passMan.MaxPass {
                passString, ok := passMan.GetInverse(i)
                if !ok {
                        t.Errorf("Expected pass %d to be loaded", i)
                } else if n, ok := passMan.Get(passString); !ok || n !
        = i {
                        t.Errorf("Expected pass value for %s to be
            loaded", passString)
                }
        }
}

func TestInitRanges(t *testing.T) {
        passMan := &PassManager{}
        passMan.loadPasses()
        passMan.initRanges()

        if passMan.moduleRange == nil ||
                passMan.cgsccRange == nil ||
                passMan.functionRange == nil ||
                passMan.loopRange == nil {
                t.Errorf("Expected non-nil ranges, got nil")
        }
        o3String := getO3String()
        for i := range passMan.MaxPass {
                s, ok := passMan.GetInverse(i)
                if !passMan.moduleRange.Within(i) &&
                        !passMan.cgsccRange.Within(i) &&
```

```
                        !passMan.functionRange.Within(i) &&
                        !passMan.loopRange.Within(i) &&
                        ok && !slices.Contains(o3String, s) { // O3 module
            passes are excluded
                        t.Errorf("Expected ranges to cover pass %d", i)
                }
        }
}

func TestGetPasses(t *testing.T) {
        passes := getPasses()
        if len(passes) == 0 {
                t.Errorf("Expected at least one pass, got none")
        }
}

func TestGetO3String(t *testing.T) {
        o3String := getO3String()
        if len(o3String) == 0 {
                t.Errorf("Expected at least one O3 pass, got none")
        }
}

func TestGetO3Int(t *testing.T) {
        passMan := GetPassManager()
        o3Int := passMan.GetO3Int()
        if len(o3Int) == 0 {
                t.Errorf("Expected at least one O3 pass, got none")
        }
}

func TestPassToInt(t *testing.T) {
        passMan := GetPassManager()
```

```
        allPasses := append(getPasses(), getO3String()...)
        if len(allPasses) == 0 {
                t.Errorf("Expected at least one pass, got none")
        }

        for _, pass := range allPasses {
                if passMan.PassToInt(pass) == 0 && pass !=
        allPasses[0] {
                        t.Errorf("Expected pass %s to have an int
            representation", pass)
                }
        }
}

func TestIntToPass(t *testing.T) {
        passMan := GetPassManager()

        for i := range passMan.MaxPass {
                passString := passMan.IntToPass(i)
                if passString == "" {
                        t.Errorf("Expected pass %d to be loaded", i)
                }
        }
}

func TestMakeValid(t *testing.T) {
        passMan := GetPassManager()

        mutatedO3function := passMan.GetO3Int()
        for range 100 {
                basicMutate(mutatedO3function, passMan.MaxPass, 100)
        }
        for i := range passMan.MaxPass {
```

```
        if passMan.functionRange.Within(i) {
                mutatedO3function[0] = i
        }
}

mutatedO3loop := passMan.GetO3Int()
for range 100 {
        basicMutate(mutatedO3loop, passMan.MaxPass, 100)
}
for i := range passMan.MaxPass {
        if passMan.loopRange.Within(i) {
                mutatedO3loop[0] = i
        }
}

tests := []struct {
        name string
        seq  []int
}{
        {
                name: "O3 sequence",
                seq:  passMan.GetO3Int(),
        },
        {
                name: "empty sequence",
                seq:  []int{},
        },
        {
                name: "mutated sequence starting with function
    pass",
                seq:  mutatedO3function,
        },
        {
```

```
                name: "mutated sequence starting with loop pass",
                seq:  mutatedO3loop,
            },
        }

        for _, test := range tests {
            t.Run(test.name, func(t *testing.T) {
                passString := passMan.MakeValid(test.seq)
                if passString == "" && len(test.seq) > 0 {
                    t.Fatalf("Expected a valid string, got none")
                } else if passString != "" && len(test.seq) == 0 {
                    t.Fatalf("Expected an empty string, got %s",
            passString)
                }

                wellFormedCount := 0
                for _, c := range passString {
                    switch c {
                    case '(', '[', '{', '<':
                        wellFormedCount++
                    case ')', ']', '}', '>':
                        wellFormedCount--
                    default:
                        continue
                    }
                }
                if wellFormedCount != 0 {
                    t.Fatalf("Expected string to be well-formed,
            got %s", passString)
                }
            })
        }
    }
```

C.9 testharness

C.9.1 median_confidence.go

```
package testharness

import (
        "6CCS3PRJ/thomasholden/util"
        "math"
)

// MedianConfidence returns true if relative error of the
slice is less than threshold.
func MedianConfidence[T util.Number](list []T, threshold
float64) bool {
        return estimateMedianRSE(list) < threshold
}

// estimateRSE provides a value for the relative standard
error of a slice of benchmark measurements
// w.r.t the median.
//
// As the distribution is log normal, we take ln of each
element.
// This also means our data is inherently relative, without
dividing by the median.
//
[3].
//
// [1] Higgins JPT, Thomas J, Chandler J, Cumpston M, Li T,
Page MJ, Welch VA (editors).
// Cochrane Handbook for Systematic Reviews of Interventions.
2nd Edition. Chichester (UK):
// John Wiley & Sons; 2019.
```

```
// [2] Williams, D. (2001). Weighing the Odds. Cambridge
University Press. p. 165. ISBN 052100618X.
// [3] Everitt, B. S. (2003). The Cambridge Dictionary of
Statistics. Cambridge University Press.
// ISBN 978-0-521-81099-9.
func estimateMedianRSE[T util.Number](list []T) float64 {
        if len(list) == 0 {
                return 0
        }
        const (
                ratio                 float64 = 1 / 1.35
                fmMedianSqrt          float64 = 1.253314137 //
        math.Sqrt(math.Pi / 2.0) can't be const
                ratioDotFmMedianSqrt float64 = ratio * fmMedianSqrt
        )

        logList := util.ForEach(list, func(i T) float64 {
                return math.Log(float64(i))
        })

        iqr := util.IQR(logList) // Cannot be negative
        sqrtN := math.Sqrt(float64(len(logList)))

        return (ratioDotFmMedianSqrt * iqr) / sqrtN
}
```

C.9.2 median_confidence_test.go

```
package testharness

import (
        "math"
        "testing"
```

```
)

// TestEstimateMedianRSE tests the mathematical logic of the
RSE approximation.
func TestEstimateMedianRSE(t *testing.T) {
        tests := []struct {
                name   string
                input  []float64
                isZero bool
                isNan  bool
                approx float64
        }{
                {
                        name:   "Constant values",
                        input:  []float64{10, 10, 10, 10, 10},
                        isZero: true,
                },
                {
                        name:   "All close to 0",
                        input:  []float64{0.0001, 0.0001, 0.0001, 0.0001,
    0.0001},
                        isZero: true,
                },
                {
                        name:   "Non zero IQR",
                        input:  []float64{8, 10, 12},
                        isZero: false,
                        approx: 0.1,
                },
                {
                        name:   "Negative values",
                        input:  []float64{-3, -2, -1},
                        isZero: false,
```

```
                        isNan:  true,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        mRse := estimateMedianRSE(test.input)

                        if test.isZero && mRse != 0.0 {
                                t.Errorf("Expected 0, got %v", mRse)
                        } else if test.isNan && !math.IsNaN(mRse) {
                                t.Errorf("Expected NaN, got %v", mRse)
                        } else if mRse+1 < test.approx || mRse-1 >
            test.approx {
                                t.Errorf("Unexpected relative standard error
                for %v : %f", test.input, mRse)
                        }
                })
        }
}

// TestMedianConfidence tests the boundary conditions of the
threshold comparison.
func TestMedianConfidence(t *testing.T) {
        tests := []struct {
                name      string
                input     []float64
                threshold float64
                pass      bool
        }{
                {
                        name:      "RSE is 0, below threshold",
                        input:     []float64{5, 5, 5, 5, 5},
```

```
            threshold: 0.1,
            pass:      true,
        },
        {
            name:      "RSE is 0, at threshold",
            input:     []float64{5, 5, 5, 5, 5},
            threshold: 0.0,
            pass:      false,
        },
        {
            name:      "No variance should pass even if median
    is small",
            input:     []float64{0.01, 0.01, 0.01, 0.01},
            threshold: 1.0,
            pass:      true,
        },
        {
            name:      "RSE should not explode when median is
    close to 0",
            input:     []float64{0.005, 0.01, 10},
            threshold: 10.0,
            pass:      true,
        },
        {
            name:      "Negative median doesn't yield negative
    RSE",
            input:     []float64{-100, -100, -10, -1, -1},
            threshold: 0.0,
            pass:      false,
        },
}

for _, test := range tests {
```

```
                t.Run(test.name, func(t *testing.T) {
                        pass := MedianConfidence(test.input,
            test.threshold)
                        if pass != test.pass {
                                t.Errorf("MedianConfidence() = %v, pass %v",
                pass, test.pass)
                        }
                })
        }
}
```

C.9.3 testharness.go

```
package testharness

import (
        "6CCS3PRJ/thomasholden/util"
        "fmt"
        "os"
        "time"
)

var approxTime time.Duration = time.Hour * 24
var approxMem int64

// GetSetBaselineStats returns the execution time and memory
usage for the baseline while setting the
// ground truth hash value.
func GetSetBaselineStats(compilationCommand []string, passes
string, noiseFunc func(), workers int) (time.Duration, int64,
error) {
        trueHash, baseMem, err :=
    getExpectedHash(compilationCommand, passes)
```

```
    if err != nil {
            return -time.Second, 0, fmt.Errorf("failed basic
    execution, is the command formatted correctly?:\n%v",
    err)
    }
    setTrueHash(trueHash)
    fmt.Print("Successfully compiled -O3 ")

    done := make(chan byte)
    for range workers - 1 {
            go func() {
                    for {
                            select {
                            case <-done:
                                    return
                            default:
                                    noiseFunc()
                            }
                    }
            }()
    }
    baseTime, err :=
getExpectedExecutionStats(compilationCommand, passes)
    close(done)
    if err != nil {
            return -time.Second, 0, fmt.Errorf("failed basic
    execution, is the command formatted correctly?:\n%v",
    err)
    }
    fmt.Println(": Successfully executed -O3.")

    approxTime, approxMem = baseTime, baseMem
    return baseTime, baseMem, nil
```

```
}

// CompileAndExecute manages interfacing with the test harness
to compile and get a stable
// time for execution of a pass sequence.
//
// Returns the compilation time, execution time, binary size
and memory usage of the sequence.
func CompileAndExecute(passes string, diff int, outLocal bool,
SourceCmd []string) (
        time.Duration, time.Duration, int, int64, error) {
        compilation_t, binary_size, newFile, err :=
    compileFileWithPasses(
                SourceCmd,
                passes,
                diff, outLocal,
        )
        if err != nil {
                return util.MAX_DURATION, util.MAX_DURATION, 0, 0, err
        }
        defer os.Remove(newFile.Name())

        execution_t, maxrss, err := runProgramUntilStable(newFile)
        if err != nil {
                return util.MAX_DURATION, util.MAX_DURATION, 0, 0, err
        }
        newFile.Close()

        return compilation_t, time.Duration(execution_t),
    binary_size, maxrss, nil
}
```

C.9.4 testharness_test.go

```
package testharness

import (
        "6CCS3PRJ/thomasholden/util"
        "fmt"
        "os"
        "testing"
        "time"
)

func initEnv() func() {
        backupRunProgramUntilStable := runProgramUntilStable
        backupRunFile := runFile
        backupCompileFileWithPasses := compileFileWithPasses
        backupApproxTime := approxTime
        restore := func() {
                runProgramUntilStable = backupRunProgramUntilStable
                runFile = backupRunFile
                compileFileWithPasses = backupCompileFileWithPasses
                approxTime = backupApproxTime
        }

        runProgramUntilStable = func(_ *os.File) (float64, int64,
    error) {
                return 2.0, 1024, nil
        }
        runFile = func(_ *os.File, _ time.Duration)
    (time.Duration, int64, uint64, error) {
                return 2 * time.Microsecond, 1024, 1, nil
        }
        compileFileWithPasses = func(_ []string, _ string, _ int,
    _ bool) (time.Duration, int, *os.File, error) {
                tmpFile, err := os.CreateTemp("", "testharness_*.bin")
```

```
                if err != nil {
                        return 0, 0, nil, err
                }
                return 100 * time.Millisecond, 32768, tmpFile, nil
        }

        return restore
}

func TestGetSetBaselineStats(t *testing.T) {
        tests := []struct {
                name                            string
                runFileOverride                 func()
                compileFileWithPassesOverride func()
                runProgramUntilStableOverride func()
                errorExpected                   bool
                hashExpected                    uint64
                durationExpected                time.Duration
                memoryExpected                  int64
        }{
                {
                        name: "getExpectedHash error path",
                        compileFileWithPassesOverride: func() {
                                compileFileWithPasses = func(_ []string, _
                string, _ int, _ bool) (time.Duration, int,
                *os.File, error) {
                                        return 0, 0, nil, fmt.Errorf("error")
                                }
                        },
                        errorExpected:    true,
                        hashExpected:     0,
                        durationExpected: -time.Second,
                        memoryExpected:   0,
```

```
        },
        {
                name: "getExpectedExecutionStats error path",
                runProgramUntilStableOverride: func() {
                        runProgramUntilStable = func(_ *os.File)
        (float64, int64, error) {
                                return 0, 0, fmt.Errorf("error")
                        }
                },
                errorExpected:    true,
                hashExpected:     1,
                durationExpected: -time.Second,
                memoryExpected:   0,
        },
        {
                name:             "happy path",
                errorExpected:    false,
                hashExpected:     1,
                durationExpected: 2,
                memoryExpected:   1024,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                restore := initEnv()
                defer restore()

                if test.runFileOverride != nil {
                        test.runFileOverride()
                }
                if test.compileFileWithPassesOverride != nil {
                        test.compileFileWithPassesOverride()
```

```
                }
                if test.runProgramUntilStableOverride != nil {
                        test.runProgramUntilStableOverride()
                }
                d, m, err := GetSetBaselineStats([]string{"echo",
            "helloworld"}, "nil", func() {}, 2)
                if d != test.durationExpected || m !=
            test.memoryExpected || (err != nil) !=
            test.errorExpected {
                        t.Errorf("Expected %v, %v, %v, but got %v, %v,
                %v",
                                test.durationExpected,
                                test.memoryExpected,
                                test.errorExpected,
                                d, m, err)
                }
                if trueHash != test.hashExpected {
                        t.Errorf("Expected trueHash to be %v, got %v",
            test.hashExpected, trueHash)
                }
                })
        }
}

func TestCompileAndExecute(t *testing.T) {
        tests := []struct {
                name                            string
                runProgramUntilStableOvveride   func()
                compileFileWithPassesOvveride   func()
                errorExpected                   bool
                compilationTimeExpected         time.Duration
                executionTimeExpected           time.Duration
                binarySizeExpected              int
```

```
        memUsageExpected                int64
}{
        {
                name: "compileFileWithPasses error path",
                compileFileWithPassesOvveride: func() {
                        compileFileWithPasses = func(_ []string, _
        string, _ int, _ bool) (time.Duration, int,
        *os.File, error) {
                                return 0, 0, nil, fmt.Errorf("error")
                        }
                },
                errorExpected:            true,
                compilationTimeExpected: util.MAX_DURATION,
                executionTimeExpected:   util.MAX_DURATION,
                binarySizeExpected:      0,
                memUsageExpected:        0,
        },
        {
                name: "runProgramUntilStable error path",
                runProgramUntilStableOvveride: func() {
                        runProgramUntilStable = func(_ *os.File)
        (float64, int64, error) {
                                return 0, 0, fmt.Errorf("error")
                        }
                },
                errorExpected:            true,
                compilationTimeExpected: util.MAX_DURATION,
                executionTimeExpected:   util.MAX_DURATION,
                binarySizeExpected:      0,
                memUsageExpected:        0,
        },
        {
                name:                    "happy path",
```

```
                errorExpected:            false,
                compilationTimeExpected: 100 * time.Millisecond,
                executionTimeExpected:    2,
                binarySizeExpected:       32768,
                memUsageExpected:         1024,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                restore := initEnv()
                defer restore()

                if test.runProgramUntilStableOvveride != nil {
                        test.runProgramUntilStableOvveride()
                }
                if test.compileFileWithPassesOvveride != nil {
                        test.compileFileWithPassesOvveride()
                }

                compilationTime, executionTime, binarySize,
    memUsage, err := CompileAndExecute(
                        "nil", 0, false, []string{"echo",
        "helloworld"},
                )

                if compilationTime != test.compilationTimeExpected
    ||
                        executionTime != test.executionTimeExpected ||
                        binarySize != test.binarySizeExpected ||
                        memUsage != test.memUsageExpected {
                        t.Errorf("Expected %v, %v, %v, %v, but got %v,
        %v, %v, %v",
```

```
                                        test.compilationTimeExpected,
                                        test.executionTimeExpected,
                                        test.binarySizeExpected,
                                        test.memUsageExpected,
                                        compilationTime, executionTime,
                        binarySize, memUsage)
                            }
                            if (err != nil) != test.errorExpected {
                                    t.Errorf("Expected an error, but got none")
                            }
                    })
            }
}
```

C.9.5 testharnessutils.go

```
package testharness

import (
        "6CCS3PRJ/thomasholden/util"
        "bytes"
        "context"
        "fmt"
        "os"
        "os/exec"
        "regexp"
        "slices"
        "syscall"
        "time"

        "github.com/zeebo/xxh3"
)
```

```
// Extracts C-family filenames from a string.
var cFileRegex *regexp.Regexp = regexp.MustCompile(`\w+?\.c(?:
(?:pp)|(?:xx)|(?:c))?`)

// Used to validate program execution output.
var trueHash uint64

// getFileSize returns the size of a file in bytes.
func getFileSize(file *os.File) (int, error) {
        stat, err := file.Stat()
        if err != nil {
                return -1, fmt.Errorf("failed to stat file '%s': %w",
        file.Name(), err)
        }
        return int(stat.Size()), nil
}

// extractInputFiles takes a string slice parsed by schlex and
returns only the
// parts determined to be filenames for C-family files.
func extractInputFiles(cmd []string) []string {
        if len(cmd) == 0 {
                return nil
        }

        temporarySlice := slices.Clone(cmd)

        returnS := slices.DeleteFunc(temporarySlice, func(s
    string) bool {
                return !cFileRegex.MatchString(s)
        })

        return returnS
```

```
}

// extractFlags extracts all the portions of a build command
that are
// determind not to be filenames for C-family files.
func extractFlags(cmd []string) []string {
        if len(cmd) == 0 {
                return nil
        }

        temporarySlice := slices.Clone(cmd[1:])

        returnS := slices.DeleteFunc(temporarySlice, func(s
    string) bool {
                return cFileRegex.MatchString(s)
        })

        return returnS
}

// mapInputFilesToBC replaces filenames for C-family files
with filenames
// of their LLVM bytecode counterparts.
func mapInputFilesToBC(cmd []string, bcFiles []string)
[]string {
        filePtr := 0
        temporarySlice := slices.Clone(cmd)

        for i, s := range temporarySlice {
                if cFileRegex.MatchString(s) {
                        temporarySlice[i] = bcFiles[filePtr]
                        filePtr++
                }
```

```
        }

        return temporarySlice
}

// Takes a file reference
// Returns execution time, memory usage, hash & error
//
// Defined as a variable function for testing purposes.
var runFile = func(file *os.File, maxDuration time.Duration)
(time.Duration, int64, uint64, error) {
        var stderr bytes.Buffer
        var stdout bytes.Buffer

        ctx := context.Background()
        var cancel context.CancelFunc
        ctx, cancel = context.WithTimeout(context.Background(),
    maxDuration)
        defer cancel()

        file.Chmod(0755)
        cmd := exec.CommandContext(ctx, file.Name())
        cmd.Stderr = &stderr
        cmd.Stdout = &stdout

        if err := cmd.Run(); err != nil {
                return 0, 0, 0, fmt.Errorf("execution failed:
        %w\nStderr: %s", err, stderr.String())
        }

        duration := cmd.ProcessState.UserTime().Microseconds()

        var hash uint64 = calculateHash(
```

```
                stderr.Bytes(), stdout.Bytes(),
        )

        // Will fail within windows
        memUsage := func() int64 {
                if usage, ok := cmd.ProcessState.SysUsage().
        (*syscall.Rusage); ok {
                        return usage.Maxrss
                }
                return util.MAX_INT64
        }()

        totalTime := time.Duration(duration *
    int64(time.Microsecond))
        return totalTime, memUsage, hash, nil
}

// calculateHash returns a uint64 hash for any number of byte
streams.
func calculateHash(streams ...[]byte) uint64 {
        hasher := xxh3.New()
        for _, stream := range streams {
                hasher.Write(stream)
        }

        return hasher.Sum64()
}

// setTrueHash assigns hash to package variable trueHash.
// Used to calculate whether a programs output has been
corrupted.
func setTrueHash(hash uint64) {
        trueHash = hash
```

```
}

// getExpectedHash returns the correct hash for the output of
the program.
//
// As a byproduct also returns memory usage.
func getExpectedHash(compilationCommand []string, passes
string) (uint64, int64, error) {
        _, _, f, err := compileFileWithPasses(compilationCommand,
    passes, 0, false)
        if err != nil {
                return 0, 0, fmt.Errorf("compilation failure: %w", err)
        }

        defer os.Remove(f.Name())
        defer f.Close() // defer is LIFO

        _, mem, hash, err := runFile(f, time.Hour*24)
        if err != nil {
                return 0, 0, fmt.Errorf("execution failure: %w", err)
        }

        return hash, mem, nil
}

// getExpectedExecutionTime returns only the median execution
time of a pass sequence.
func getExpectedExecutionStats(compilationCommand []string,
passes string) (time.Duration, error) {
        _, _, f, err := compileFileWithPasses(compilationCommand,
    passes, 0, false)
        defer os.Remove(f.Name())
        defer f.Close() // defer is LIFO
```

```
        if err != nil {
                return -time.Second, fmt.Errorf("compilation failure:
        %w", err)
        }

        timeTaken, _, err := runProgramUntilStable(f)
        if err != nil {
                return -time.Second, fmt.Errorf("execution failure:
        %w", err)
        }

        return time.Duration(timeTaken), nil
}

// compileFileWithPasses takes a file reference, pass list &
unique id.
//
// Returns compilation time, binary size, file reference &
error.
//
// Caller must resolve returned executable reference (close
and delete).
// Defined as a variable function for testing purposes.
var compileFileWithPasses = func(compilationCommand []string,
passes string, id int, outLocal bool) (time.Duration, int,
*os.File, error) {
        if len(compilationCommand) == 0 {
                return 0, 0, nil, fmt.Errorf("empty command passed")
        }

        binFile, err := os.CreateTemp("", fmt.Sprintf("bin-%d-*",
    id))
```

```
if err != nil {
        return 0, 0, nil, fmt.Errorf("failed to create
temporary file: %w", err)
}
binName := binFile.Name()
binFile.Close()

// Extract input files as a list of strings
filesToOptimise := extractInputFiles(compilationCommand)

var intermediaryFiles []string
for range filesToOptimise {
        bcFile, err := os.CreateTemp("", fmt.Sprintf("opt-%d-
*.bc", id))
        if err != nil {
                return 0, 0, nil, fmt.Errorf("failed to create
    temporary file: %w", err)
        }
        intermediaryFiles = append(intermediaryFiles,
bcFile.Name())
        defer os.Remove(bcFile.Name()) // Deletes IR when we
finish compiling
        bcFile.Close()
}

var stderr bytes.Buffer
// Extract flags as a list of strings
flags := extractFlags(compilationCommand)
for i := range filesToOptimise {
        stderr.Reset()
        inpFile, outFile := filesToOptimise[i],
intermediaryFiles[i]
```

```
        // here we add -O3, despite disabling running passes
in this step
        // this provided essential target library info to the
IR, allowing us to optimise
        // without being at a disadvantage to a simple clang -
O3 command
        command := []string{
                compilationCommand[0], inpFile, "-O3", "-emit-
    llvm", "-Xclang", "-disable-llvm-passes",
                "-Xclang", "-disable-llvm-optzns", "-Xclang", "-
    disable-O0-optnone", "-c", // https://arxiv.org/
    pdf/2109.08267
        }
        command = append(command, flags...)
        command = append(command, "-o", outFile)

        cmd := exec.Command(command[0], command[1:]...)
        cmd.Stderr = &stderr

        if err := cmd.Run(); err != nil {
                os.Remove(binName)
                return 0, 0, nil, fmt.Errorf("compilation failed:
    %w\nCommand: %v\nStderr: %s", err, command,
    stderr.String())
        }
}

var totalDuration int64
for i := range intermediaryFiles {
        stderr.Reset()
        inpFile, outFile := intermediaryFiles[i],
intermediaryFiles[i]
```

```
            command := []string{"opt", "-passes=" + passes,
    inpFile, "-o", outFile}

            cmd := exec.Command(command[0], command[1:]...)
            cmd.Stderr = &stderr

            if err := cmd.Run(); err != nil {
                    os.Remove(binName)
                    return 0, 0, nil, fmt.Errorf("optimisation failed:
        %w\nCommand: %v\nStderr: %s", err, command,
        stderr.String())
            }

            duration := cmd.ProcessState.UserTime().Microseconds()

            totalDuration += duration
    }

    newCommand := mapInputFilesToBC(compilationCommand,
intermediaryFiles)

    newCommand = append(newCommand, "-O3", "-Xclang", "-
disable-llvm-passes",
            "-Xclang", "-disable-llvm-optzns") // applies O3
    optimisations without running passes

    if outLocal {
            localCmd := exec.Command(newCommand[0],
    newCommand[1:]...)
            localCmd.Stderr = &stderr
            if err := localCmd.Run(); err != nil {
                    return 0, 0, nil, fmt.Errorf("final linking
        failed: %w\nStderr: %s", err, stderr.String())
```

```
            }
        }
        newCommand = append(newCommand, "-o", binName)

        cmd := exec.Command(newCommand[0], newCommand[1:]...)
        cmd.Stderr = &stderr

        if err := cmd.Run(); err != nil {
            os.Remove(binName)
            return 0, 0, nil, fmt.Errorf("linking failed:
        %w\nStderr: %s", err, stderr.String())
        }

        binFile, err = os.Open(binName)
        if err != nil {
            os.Remove(binName)
            return 0, 0, nil, fmt.Errorf("failed to open return
        binary: %w", err)
        }

        binSize, err := getFileSize(binFile)
        if err != nil {
            binFile.Close()
            os.Remove(binName)
            return 0, 0, nil, fmt.Errorf("failed to size binary
        '%s': %w", binName, err)
        }

        totalTime := time.Duration(totalDuration *
    int64(time.Microsecond))
        return totalTime, binSize, binFile, nil
}
```

```
// runProgramUntilStable returns the UCB for the median
execution time and maxrss of a
// file execution.
//
// Defined as a variable function for testing purposes.
var runProgramUntilStable = func(toRun *os.File) (float64,
int64, error) {
        var times []int64
        var rsss []int64

        i := 0
        for begin := time.Now(); (i < 5 || // run at least 5 times
                !MedianConfidence(times, 0.03)) && // median is not
        within 3%
                time.Since(begin) < 12*approxTime; { // run for at
        most 12*o3 time

                execution_t, rss, hash, err := runFile(toRun,
        approxTime*3)
                if err != nil {
                        return util.MAX_FLOAT64, util.MAX_INT64,
            fmt.Errorf("execution failed: %v", err)
                } else if hash != trueHash {
                        return util.MAX_FLOAT64, util.MAX_INT64,
            fmt.Errorf(
                                "execution out hash: %v\ntrue out hash: %v",
                hash, trueHash,
                        )
                } else if execution_t == 0 {
                        rsss = append(rsss, rss)
                        continue
                }
```

```
                times = append(times, int64(execution_t))
                rsss = append(rsss, rss)
                i++
        }

        median := util.Median(times)
        confidence := estimateMedianRSE(times)
        errorScaledMedian := median + (median * confidence)
        if errorScaledMedian < float64(approxTime) {
                approxTime = time.Duration(errorScaledMedian)
        }

        return errorScaledMedian, slices.Max(rsss), nil
}
```

C.9.6 testharnessutils_test.go

```
package testharness

import (
        "os"
        "testing"
        "time"

        "github.com/google/shlex"
)

// compilePassFunc is defined to prevent repeat definitions
for overrides.
type compilePassFunc func([]string, string, int, bool)
(time.Duration, int, *os.File, error)

func TestGetFileSize(t *testing.T) {
```

```
zeroFile, err := os.CreateTemp("", "testharness_*.bin")
if err != nil {
        t.Fatalf("failed to create temporary file: %v", err)
}

tmpFile, err := os.CreateTemp("", "testharness_*.bin")
if err != nil {
        t.Fatalf("failed to create temporary file: %v", err)
}
bytesToWrite := make([]byte, 1024) // 1KB of 0 data
tmpFile.Write(bytesToWrite)

deletedFile, _ := os.CreateTemp("", "testharness_*.bin")
deletedFile.Close()
os.Remove(deletedFile.Name())

tests := []struct {
        name           string
        file           *os.File
        sizeExpected   int
        errorExpected bool
}{
        {
                name:           "valid file of 0 size",
                file:           zeroFile,
                sizeExpected:   0,
                errorExpected: false,
        },
        {
                name:           "valid file of non-zero size",
                file:           tmpFile,
                sizeExpected:   1024,
                errorExpected: false,
```

```
                },
                {
                        name:          "error when file does not exist",
                        file:          deletedFile,
                        errorExpected: true,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        size, err := getFileSize(test.file)
                        if err == nil && test.errorExpected {
                                t.Errorf("expected to fail getting file size")
                        } else if err != nil && !test.errorExpected {
                                t.Errorf("failed to get file size
                unexpectedly: %v", err)
                        } else if err == nil && size != test.sizeExpected {
                                t.Errorf("got file size of %d, but expected
                %d", size, test.sizeExpected)
                        }
                })
        }
}

func TestExtractInputFiles(t *testing.T) {
        validCmd, err := shlex.Split(
                "clang -I./example/PolyBenchC-4.2.1/utilities -I./
        example/PolyBenchC-4.2.1/datamining/" +
                        "correlation ./example/PolyBenchC-4.2.1/utilities/
                polybench.c ./example/PolyBenchC" +
                        "-4.2.1/datamining/correlation/correlation.c",
        )
        if err != nil {
```

```
        t.Fatalf("failed to split command: %v", err)
}

tests := []struct {
        name    string
        command []string
        files   []string
}{
        {
                name:    "no files in command",
                command: []string{"echo", "helloworld"},
                files:   []string{},
        },
        {
                name:    "empty command",
                command: []string{},
                files:   []string{},
        },
        {
                name:    "only files in command",
                command: []string{"helloworld.c",
    "hello_world.cpp", "helloworld.cc"},
                files:   []string{"helloworld.c",
    "hello_world.cpp", "helloworld.cc"},
        },
        {
                name:    "valid example command",
                command: validCmd,
                files: []string{
                        "./example/PolyBenchC-4.2.1/utilities/
        polybench.c",
                        "./example/PolyBenchC-4.2.1/datamining/
        correlation/correlation.c",
```

```
                },
            },
        }

        for _, test := range tests {
            t.Run(test.name, func(t *testing.T) {
                files := extractInputFiles(test.command)
                if len(files) != len(test.files) {
                    t.Errorf("got %d files, but expected %d",
            len(files), len(test.files))
                }
                for i, file := range files {
                    if file != test.files[i] {
                        t.Errorf("got file %s, but expected %s",
                file, test.files[i])
                    }
                }
            })
        }
}

func TestExtractFlags(t *testing.T) {
        validCmd, err := shlex.Split(
            "clang -I./example/PolyBenchC-4.2.1/utilities -I./
        example/PolyBenchC-4.2.1/datamining/" +
                "correlation ./example/PolyBenchC-4.2.1/utilities/
            polybench.c ./example/PolyBenchC" +
                "-4.2.1/datamining/correlation/correlation.c",
        )
        if err != nil {
            t.Fatalf("failed to split command: %v", err)
        }
```

```
tests := []struct {
        name    string
        command []string
        flags   []string
}{
        {
                name:    "no files in command",
                command: []string{"run", "echo", "helloworld"},
                flags:   []string{"echo", "helloworld"},
        },
        {
                name:    "empty command",
                command: []string{},
                flags:   []string{},
        },
        {
                name:    "only files in command",
                command: []string{"run", "helloworld.c",
    "hello_world.cpp", "helloworld.cc"},
                flags:   []string{},
        },
        {
                name:    "valid example command",
                command: validCmd,
                flags: []string{
                        "-I./example/PolyBenchC-4.2.1/utilities",
                        "-I./example/PolyBenchC-4.2.1/datamining/
        correlation",
                },
        },
}

for _, test := range tests {
```

```
                t.Run(test.name, func(t *testing.T) {
                        flags := extractFlags(test.command)
                        if len(flags) != len(test.flags) {
                                t.Fatalf("got %d flag(s), but expected %d",
                len(flags), len(test.flags))
                        }
                        for i, flag := range flags {
                                if flag != test.flags[i] {
                                        t.Errorf("got file %s, but expected %s",
                    flag, test.flags[i])
                                }
                        }
                })
        }
}

func TestMapInputFilesToBC(t *testing.T) {
        validCmd, err := shlex.Split(
                "clang -I./example/PolyBenchC-4.2.1/utilities -I./
        example/PolyBenchC-4.2.1/datamining/" +
                "correlation ./example/PolyBenchC-4.2.1/utilities/
            polybench.c ./example/PolyBenchC" +
                "-4.2.1/datamining/correlation/correlation.c",
        )
        if err != nil {
                t.Fatalf("failed to split command: %v", err)
        }

        tests := []struct {
                name    string
                command []string
                files   []string
                outCmd  []string
```

```
}{
        {
                name:    "no files in command",
                command: []string{"echo", "helloworld"},
                outCmd:  []string{"echo", "helloworld"},
        },
        {
                name:    "empty command",
                command: []string{},
                files:   []string{},
                outCmd:  []string{},
        },
        {
                name:    "only files in command",
                command: []string{"helloworld.c",
    "hello_world.cpp", "helloworld.cc"},
                files:   []string{"helloworld1.bc",
    "hello_world2.bc", "helloworld3.bc"},
                outCmd:  []string{"helloworld1.bc",
    "hello_world2.bc", "helloworld3.bc"},
        },
        {
                name:    "valid example command",
                command: validCmd,
                files: []string{
                        "./example/PolyBenchC-4.2.1/utilities/
    polybench.bc",
                        "./example/PolyBenchC-4.2.1/datamining/
    correlation/correlation.bc",
                },
                outCmd: []string{"clang", "-I./example/
    PolyBenchC-4.2.1/utilities", "-I./example/
    PolyBenchC-4.2.1/datamining/" +
```

```
                                "correlation", "./example/PolyBenchC-4.2.1/
                utilities/polybench.bc", "./example/
                PolyBenchC" +
                                "-4.2.1/datamining/correlation/
                correlation.bc"},
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        outCmd := mapInputFilesToBC(test.command,
            test.files)
                        if len(outCmd) != len(test.outCmd) {
                                t.Fatalf("got command of length %d, but
                expected %d", len(outCmd), len(test.outCmd))
                        }
                        for i, param := range outCmd {
                                if param != test.outCmd[i] {
                                        t.Errorf("got command %v, but expected
                    %v", outCmd, test.outCmd)
                                }
                        }
                })
        }
}

func TestRunFile(t *testing.T) {
        deletedFile, err := os.CreateTemp("", "testharness_*.bin")
        if err != nil {
                t.Fatalf("failed to create temp file: %v", err)
        }
        os.Remove(deletedFile.Name())
```

```
validFile, err := createShellScriptWithOutput("success")
if err != nil {
        t.Fatalf("failed to create temp file: %v", err)
}
defer os.Remove(validFile.Name())

tests := []struct {
        name          string
        file          *os.File
        errorExpected bool
}{
        {
                name:          "invalid file",
                file:          deletedFile,
                errorExpected: true,
        },
        {
                name:          "valid file",
                file:          validFile,
                errorExpected: false,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                duration, memUsage, _, err := runFile(test.file,
    2*time.Second)

                if test.errorExpected && err == nil {
                        t.Errorf("expected an error, but got nil")
                } else if !test.errorExpected && err != nil {
                        t.Fatalf("did not expect an error, but got:
        %v", err)
```

```
                } else if duration < 0 {
                        t.Errorf("expected duration to be non-
            negative, got %v", duration)
                } else if memUsage < 0 {
                        t.Errorf("expected memory usage to be non-
            negative, got %d", memUsage)
                }
            })
        }
}

func TestCalculateHash(t *testing.T) {
        stream1 := []byte("hello world")
        stream2 := []byte("")
        stream3 := []byte("helloworld")

        tests := []struct {
                name   string
                first  []byte
                second []byte
                match  bool
        }{
                {
                        name:   "same stream",
                        first:  stream1,
                        second: stream1,
                        match:  true,
                },
                {
                        name:   "two empty streams",
                        first:  stream2,
                        second: stream2,
                        match:  true,
```

```
                },
                {
                        name:   "subtly different streams",
                        first:  stream1,
                        second: stream3,
                        match:  false,
                },
                {
                        name:   "completely different streams",
                        first:  stream1,
                        second: stream2,
                        match:  false,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        hash1 := calculateHash(test.first)
                        hash2 := calculateHash(test.second)
                        if test.match != (hash1 == hash2) {
                                t.Errorf("expected hashes to match, but got %v
                and %v", hash1, hash2)
                        }
                })
        }
}

func TestSetTrueHash(t *testing.T) {
        before := trueHash
        setTrueHash(before + 1)
        if trueHash != before+1 {
                t.Errorf("expected trueHash to be %d, but got %d",
        before+1, trueHash)
```

```
        }
}

func TestGetExpectedHash(t *testing.T) {
        tests := []struct {
                name          string
                first         compilePassFunc
                second        compilePassFunc
                match         bool
                errorExpected bool
        }{
                {
                        name:          "same output",
                        first:
            buildCompileFileWithPassesOut("hello world", nil,
            false),
                        second:
            buildCompileFileWithPassesOut("hello world", nil,
            false),
                        match:         true,
                        errorExpected: false,
                },
                {
                        name:          "two empty outputs",
                        first:         buildCompileFileWithPassesOut("",
            nil, false),
                        second:        buildCompileFileWithPassesOut("",
            nil, false),
                        match:         true,
                        errorExpected: false,
                },
                {
                        name:          "compilation error path",
```

```
                first:
        buildCompileFileWithPassesOut("hello world",
        os.ErrClosed, false),
                second:
        buildCompileFileWithPassesOut("helloworld",
        os.ErrClosed, false),
                match:          true,
                errorExpected: true,
            },
            {
                name:           "run error path",
                first:
        buildCompileFileWithPassesOut("hello world", nil,
        true),
                second:
        buildCompileFileWithPassesOut("hello world", nil,
        true),
                match:          true,
                errorExpected: true,
            },
    }

    backupCompileFileWithPasses := compileFileWithPasses
    defer func() { compileFileWithPasses =
backupCompileFileWithPasses }()

    for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                compileFileWithPasses = test.first
                hash1, mem1, err1 := getExpectedHash([]string{},
        "")

                compileFileWithPasses = test.second
```

```
                hash2, mem2, err2 := getExpectedHash([]string{},
        "")

                if test.match != (hash1 == hash2) {
                        t.Errorf("expected hashes to match, but got %v
            and %v", hash1, hash2)
                } else if err1 != nil && err2 != nil && mem1 > 0
        && mem2 > 0 {
                        t.Errorf("expected memory usage to be above 0,
            got %v and %v", mem1, mem2)
                } else if test.errorExpected && (err1 == nil ||
        err2 == nil) {
                        t.Errorf("expected an error, but got nil: %v
            and %v", err1, err2)
                } else if !test.errorExpected && (err1 != nil ||
        err2 != nil) {
                        t.Fatalf("did not expect an error, but got: %v
            and %v", err1, err2)
                }
            })
        }
}

func TestGetExpectedExecutionStats(t *testing.T) {
        tests := []struct {
                name          string
                override      compilePassFunc
                hash          uint64
                errorExpected bool
        }{
                {
                        name:          "happy path",
                        override:
```

```
            buildCompileFileWithPassesOut("hello world", nil,
            false),
                    hash:           calculateHash([]byte("hello
            world\n")),
                    errorExpected: false,
                },
                {
                    name:           "compilation error path",
                    override:
            buildCompileFileWithPassesOut("hello world",
            os.ErrClosed, false),
                    errorExpected: true,
                },
                {
                    name:           "run error path",
                    override:
            buildCompileFileWithPassesOut("hello world", nil,
            true),
                    errorExpected: true,
                },
        }

        backupCompileFileWithPasses := compileFileWithPasses
        backupApproxTime := approxTime
        defer func() { compileFileWithPasses =
    backupCompileFileWithPasses }()

        for _, test := range tests {
            t.Run(test.name, func(t *testing.T) {
                    defer func() { approxTime = backupApproxTime }()
                    compileFileWithPasses = test.override
                    setTrueHash(test.hash)
                    time, err := getExpectedExecutionStats([]string{},
```

```
            "")

                        if test.errorExpected != (err != nil) {
                                t.Errorf("expected error mismatch, got %v,
                expected? %v", err, test.errorExpected)
                        } else if err == nil && time < 0 {
                                t.Errorf("expected time > 0, got %v", time)
                        }
                })
        }
}

func TestCompileFileWithPasses(t *testing.T) {
        validFile, err := os.CreateTemp("", "testharness_*.c")
        if err != nil {
                t.Fatalf("failed to create temp file: %v", err)
        }
        defer os.Remove(validFile.Name())
        if _, err := validFile.WriteString("int main() { return 0;
    }\n"); err != nil {
                t.Fatalf("failed to write to temp file: %v", err)
        }
        validFile.Sync()
        validFile.Close()

        tests := []struct {
                name                string
                compilationCommand []string
                passes              string
                errorExpected       bool
        }{
                {
                        name:                "empty command",
```

```
                compilationCommand: []string{},
                passes:             "",
                errorExpected:      true,
        },
        {
                name:               "invalid command",
                compilationCommand: []string{"invalidcmd",
    validFile.Name()},
                passes:             "",
                errorExpected:      true,
        },
        {
                name:               "valid compilation",
                compilationCommand: []string{"clang",
    validFile.Name()},
                passes:             "default<O1>",
                errorExpected:      false,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                duration, size, file, err :=
    compileFileWithPasses(test.compilationCommand,
    test.passes, 0, false)

                if file != nil {
                        defer os.Remove(file.Name())
                        defer file.Close()
                }

                if !test.errorExpected && err != nil {
                        t.Fatalf("did not expect an error, but got:
```

```
                %v", err)
                        }

                        if err == nil {
                                if test.errorExpected {
                                        t.Errorf("expected an error, but got nil")
                                }
                                if file == nil {
                                        t.Errorf("expected a valid file reference,
                    got nil")
                                }
                                if size <= 0 {
                                        t.Errorf("expected binary size > 0, got
                    %d", size)
                                }
                                if duration < 0 {
                                        t.Errorf("expected duration >= 0, got %v",
                    duration)
                                }
                        }
                })
        }
}

func TestRunProgramUntilStable(t *testing.T) {
        deletedFile, err := createShellScriptWithOutput("failure")
        if err != nil {
                t.Fatalf("failed to create temp file: %v", err)
        }
        os.Remove(deletedFile.Name())

        validFile, err := createShellScriptWithOutput("success")
        if err != nil {
```

```
        t.Fatalf("failed to create temp file: %v", err)
}
defer os.Remove(validFile.Name())

invalidFile, err := createShellScriptWithOutput("failure")
if err != nil {
        t.Fatalf("failed to create temp file: %v", err)
}
defer os.Remove(validFile.Name())

tests := []struct {
        name          string
        file          *os.File
        hash          uint64
        errorExpected bool
}{
        {
                name:          "happy path",
                file:          validFile,
                hash:          calculateHash([]byte("success\n")),
                errorExpected: false,
        },
        {
                name:          "run error path",
                file:          deletedFile,
                errorExpected: true,
        },
        {
                name:          "invalid hash",
                file:          invalidFile,
                errorExpected: true,
        },
}
```

```
        backupApproxTime := approxTime

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        defer func() { approxTime = backupApproxTime }()

                        setTrueHash(test.hash)
                        execTime, maxRSS, err :=
            runProgramUntilStable(test.file)

                        if test.errorExpected != (err != nil) {
                                t.Errorf("expected error mismatch, got %v,
                expected? %v", err, test.errorExpected)
                        } else if err == nil && execTime < 0 {
                                t.Errorf("expected time > 0, got %v", execTime)
                        } else if err == nil && maxRSS < 0 {
                                t.Errorf("expected maxRSS > 0, got %v", maxRSS)
                        }
                })
        }
}

// createShellScriptWithOutput creates a shell script with the
given output.
//
// Returns the file and an error.
func createShellScriptWithOutput(out string) (*os.File, error)
{
        validFile, err := os.CreateTemp("",
    "testharness_valid_*.sh")
        if err != nil {
                return nil, err
```

```
        }
        validFile.Chmod(0755)

        _, err = validFile.WriteString("#!/bin/sh\necho '" + out +
    "'\n")
        if err != nil {
                return nil, err
        }
        validFile.Sync()
        validFile.Close()

        return validFile, nil
}

// buildCompileFileWithPassesOut builds and returns a
compileFileWithPasses function override.
func buildCompileFileWithPassesOut(out string, err error,
delete bool) compilePassFunc {
        return func(_ []string, _ string, _ int, _ bool)
    (time.Duration, int, *os.File, error) {
                file, _ := createShellScriptWithOutput(out)
                if delete {
                        os.Remove(file.Name())
                }
                return 0, 0, file, err
        }
}
```

C.10 util

C.10.1 bidirectionalmap.go

```
package util
```

```
import (
        "fmt"
        "maps"
        "slices"
)

// BidirectionalMap implements a two way mapping using two
unique maps.
type BidirectionalMap[K, V comparable] struct {
        direct  map[K]V // The map from key to value
        inverse map[V]K // The map from value to key
}

// Initialises the map components.
func (b *BidirectionalMap[K, V]) init() {
        if b.direct == nil {
                b.direct = make(map[K]V)
        }
        if b.inverse == nil {
                b.inverse = make(map[V]K)
        }
}

// Put adds a key value pair to the map.
func (b *BidirectionalMap[K, V]) Put(key K, value V) error {
        if existingValue, ok := b.direct[key]; ok {
                return fmt.Errorf("key %v already bound to value: %v",
        key, existingValue)
        }
        if existingKey, ok := b.inverse[value]; ok {
                return fmt.Errorf("value %v already bound to key: %v",
        value, existingKey)
```

```
        }

        b.init()

        b.direct[key], b.inverse[value] = value, key
        return nil
}

// PutInverse adds a value key pair to the map.
//
// The same as calling Put with reversed args.
func (b *BidirectionalMap[K, V]) PutInverse(key V, value K)
error {
        if existingValue, ok := b.direct[value]; ok {
                return fmt.Errorf("value %v already bound to key: %v",
        key, existingValue)
        }
        if existingKey, ok := b.inverse[key]; ok {
                return fmt.Errorf("key %v already bound to value: %v",
        value, existingKey)
        }

        b.init()

        b.inverse[key], b.direct[value] = value, key
        return nil
}

// Get accesses the map by key to obtain a value.
func (b *BidirectionalMap[K, V]) Get(key K) (V, bool) {
        existingValue, ok := b.direct[key]
        return existingValue, ok
```

```
}

// GetInverse accesses the map by value to obtain a key.
func (b *BidirectionalMap[K, V]) GetInverse(key V) (K, bool) {
        existingValue, ok := b.inverse[key]
        return existingValue, ok
}

// Delete removes a key mapping from a bidirectional map.
func (b *BidirectionalMap[K, V]) Delete(key K) {
        if existingValue, ok := b.direct[key]; ok {
                delete(b.direct, key)
                delete(b.inverse, existingValue)
        }
}

// DeleteInverse removes a value mapping from a bidirectional
map.
func (b *BidirectionalMap[K, V]) DeleteInverse(key V) {
        if existingValue, ok := b.inverse[key]; ok {
                delete(b.direct, existingValue)
                delete(b.inverse, key)
        }
}

// Keys returns all of the keys from the Bidirectional map as
a slice.
func (b *BidirectionalMap[K, V]) Keys() []K {
        return slices.Collect(maps.Keys(b.direct))
}

// Values returns all of the values from the Bidirectional map
as a slice.
```

```
func (b *BidirectionalMap[K, V]) Values() []V {
        return slices.Collect(maps.Keys(b.inverse))
}
```

C.10.2 bidirectionalmap_test.go

```
package util

import (
        "slices"
        "testing"
)

var m BidirectionalMap[int, int] // m is a bidirectional map
for testing purposes

// bidirectionalMapTeardown is used to clean up the
bidirectional map post testing.
func bidirectionalMapTeardown() {
        for _, key := range m.Keys() {
                m.Delete(key)
        }
}

// TestPut tests that the Put function correctly handles valid
and invalid
// key-value pairs.
func TestPut(t *testing.T) {
        defer bidirectionalMapTeardown()

        err := m.Put(0, 1)
        if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
```

```
        }
        err = m.Put(0, 3) // Should error due to key collision
        if err == nil {
                t.Fatalf("Didn't error on invalid put")
        }
}

// TestPutInverse tests that the PutInverse function correctly
handles valid and
// invalid key-value pairs.
func TestPutInverse(t *testing.T) {
        defer bidirectionalMapTeardown()

        err := m.PutInverse(0, 1)
        if err != nil {
                t.Fatalf("Errored on attempted inverse put %v", err)
        }
        err = m.PutInverse(0, 3) // Should error due to key
    collision
        if err == nil {
                t.Fatalf("Didn't error on invalid inverse put")
        }
}

// TestGet verifies that the Get function, returns nil for
missing keys and
// the correct value for existing keys.
func TestBDMapGet(t *testing.T) {
        defer bidirectionalMapTeardown()

        val, ok := m.Get(0)
        if ok {
                t.Fatalf("Got %v, but expected nil", val)
```

```
        }

        err := m.Put(0, 1)
        if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
        }

        val, ok = m.Get(0)
        if !ok || val != 1 {
                t.Fatalf("Got %v, but expected 1 | ok=%v", val, ok)
        }
}

// TestGet verifies that the GetInverse function, returns nil
for missing values
// and the correct key for existing values.
func TestGetInverse(t *testing.T) {
        defer bidirectionalMapTeardown()

        val, ok := m.GetInverse(1)
        if ok {
                t.Fatalf("Got %v, but expected nil", val)
        }

        err := m.Put(0, 1)
        if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
        }

        val, ok = m.GetInverse(1)
        if !ok || val != 0 {
                t.Fatalf("Got %v, but expected 0 | ok=%v", val, ok)
        }
```

```
}

// TestDelete tests that the Delete function correctly deletes
key-value and
// value-key pairs.
func TestDelete(t *testing.T) {
        defer bidirectionalMapTeardown()

        err := m.Put(0, 1)
        if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
        }

        val, ok := m.Get(0)
        if !ok || val != 1 {
                t.Fatalf("Got %v, but expected 1 | ok=%v", val, ok)
        }

        m.Delete(0)

        val, ok = m.Get(0)
        if ok {
                t.Fatalf("Got %v, but expected nil", val)
        }
}

// TestDeleteInverse tests that the DeleteInverse function
correctly deletes
// value-key and key-value pairs.
func TestDeleteInverse(t *testing.T) {
        defer bidirectionalMapTeardown()

        err := m.Put(0, 1)
```

```
        if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
        }

        val, ok := m.Get(0)
        if !ok || val != 1 {
                t.Fatalf("Got %v, but expected 1", val)
        }

        m.DeleteInverse(1)

        val, ok = m.Get(0)
        if ok {
                t.Fatalf("Got %v, but expected nil", val)
        }
}

// TestKeys tests that the Keys function returns the expected
keys.
func TestKeys(t *testing.T) {
        defer bidirectionalMapTeardown()

        keys := []int{
                0, 1, 2, 3, 4, 5,
        }

        values := []int{
                0, 2, 4, 6, 8, 10,
        }

        for i := range keys {
                err := m.Put(keys[i], values[i])
                if err != nil {
```

```
                t.Fatalf("Errored on attempted put %v", err)
            }
        }

        mapKeys := m.Keys()
        slices.Sort(mapKeys)
        if !slices.Equal(mapKeys, keys) {
            t.Fatalf("Expected %v, got %v", keys, m.Keys())
        }
}

// TestValues tests that the Values function returns the
expected values.
func TestValues(t *testing.T) {
        defer bidirectionalMapTeardown()

        keys := []int{
            0, 1, 2, 3, 4, 5,
        }

        values := []int{
            0, 2, 4, 6, 8, 10,
        }

        for i := range keys {
            err := m.Put(keys[i], values[i])
            if err != nil {
                t.Fatalf("Errored on attempted put %v", err)
            }
        }

        mapValues := m.Values()
        slices.Sort(mapValues)
```

```
        if !slices.Equal(mapValues, values) {
                t.Fatalf("Expected %v, got %v", values, m.Values())
        }
}
```

C.10.3 cache.go

```
package util

import "sync"

// Cache is a basic map designed for concurrent use.
type Cache[K, V comparable] struct {
        cache         map[K]V    // The underlying map for the
    cache struct
        queue         []K        // The queue of keys
        queueLastPtr  int        // The pointer to the last
    element in the queue
        queueFirstPtr int        // The pointer to the first
    element in the queue
        lock          sync.Mutex // The lock for concurrent access
    to the cache
}

// Initialises the map components.
func (c *Cache[K, V]) Init(size int) {
        c.lock.Lock()
        defer c.lock.Unlock()
        if c.cache == nil {
                c.cache = make(map[K]V)
                c.queue = make([]K, size)
        }
}
```

```
// Get accesses the map by key to obtain a value.
//
// Returns the value and whether it was found.
func (c *Cache[K, V]) Get(key K) (V, bool) {
        c.lock.Lock()
        defer c.lock.Unlock()
        value, ok := c.cache[key]
        return value, ok
}

// Set adds a key value pair to the map.
func (c *Cache[K, V]) Set(key K, value V) {
        c.lock.Lock()
        defer c.lock.Unlock()
        _, inMap := c.cache[key]
        c.cache[key] = value

        if !inMap {
                c.enqueue(key)
        }
}

func (c *Cache[K, V]) enqueue(key K) {
        if len(c.cache) > cap(c.queue) {
                keyToRemove := c.queue[c.queueFirstPtr]
                delete(c.cache, keyToRemove)
                c.queueFirstPtr = (c.queueFirstPtr + 1) % cap(c.queue)
        }
        c.queue[c.queueLastPtr] = key
        c.queueLastPtr = (c.queueLastPtr + 1) % cap(c.queue)
}
```

C.10.4 cache_test.go

```
package util

import (
        "testing"
)

func TestInit(t *testing.T) {
        tests := []struct {
                name string
                size int
        }{
                {
                        name: "0 size cache",
                        size: 0,
                },
                {
                        name: "1 size cache",
                        size: 1,
                },
                {
                        name: "100 size cache",
                        size: 100,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        cache := Cache[int, int]{}
                        cache.Init(test.size)

                        if len(cache.queue) != test.size {
                                t.Errorf("expected queue length of %d, got
```

```
                %d", test.size, len(cache.queue))
                        }
                })
        }
}

func TestGet(t *testing.T) {
        tests := []struct {
                name string
                hit  bool
        }{
                {name: "cache hit", hit: true},
                {name: "cache miss", hit: false},
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        cache := Cache[int, int]{}
                        cache.Init(1)

                        if test.hit {
                                cache.Set(1, 1)
                        }

                        value, ok := cache.Get(1)

                        if ok != test.hit {
                                t.Errorf("expected ok to be %v", test.hit)
                        }
                        if ok && value != 1 {
                                t.Errorf("expected value to be 1, got %d",
                value)
                        }
```

```
                })
        }
}

func TestSet(t *testing.T) {
        tests := []struct {
                name      string
                firstKey  int
                firstVal  int
                secondKey int
                secondVal int
                finalLen  int
        }{
                {
                        name:      "two unique keys",
                        firstKey:  1,
                        firstVal:  0,
                        secondKey: 2,
                        secondVal: 1,
                        finalLen:  2,
                },
                {
                        name:      "overwrite key",
                        firstKey:  1,
                        firstVal:  0,
                        secondKey: 1,
                        secondVal: 1,
                        finalLen:  1,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
```

```
                cache := Cache[int, int]{}
                cache.Init(2)

                cache.Set(test.firstKey, test.firstVal)
                cache.Set(test.secondKey, test.secondVal)

                if len(cache.cache) != test.finalLen {
                        t.Errorf(
                                "expected queue length of %d, got %d: %v",
                                test.finalLen, len(cache.cache),
            cache.queue,
                        )
                }
        })
    }
}

func TestEnqueue(t *testing.T) {
    tests := []struct {
            name         string
            size         int
            writes       int
            containFirst bool
    }{
            {
                    name:         "saturated cache",
                    size:         10,
                    writes:       10,
                    containFirst: true,
            },
            {
                    name:         "write to 1 size cache",
                    size:         1,
```

```
                writes:       1,
                containFirst: true,
            },
            {
                name:         "write to saturated cache",
                size:         10,
                writes:       11,
                containFirst: false,
            },
        }

        for _, test := range tests {
            t.Run(test.name, func(t *testing.T) {
                cache := Cache[int, int]{}
                cache.Init(test.size)

                for i := range test.writes {
                    cache.cache[i] = i
                    cache.enqueue(i)
                }

                if test.containFirst && cache.queue[0] != 0 {
                    t.Errorf("expected first value to be 0, got
            %d", cache.queue[0])
                } else if !test.containFirst && cache.queue[0] ==
        0 {
                    t.Errorf("expected first value to not be 0,
            got %d", cache.queue[0])
                }
            })
        }
    }
```

```
func TestCache(t *testing.T) {
        type operation struct {
                op  string
                key int
                val int
                ok  bool
        }

        tests := []struct {
                name string
                size int
                ops  []operation
        }{
                {
                        name: "basic set and get",
                        size: 2,
                        ops: []operation{
                                {op: "get", key: 1, ok: false},
                                {op: "set", key: 1, val: 100},
                                {op: "get", key: 1, val: 100, ok: true},
                        },
                },
                {
                        name: "overwrite existing key",
                        size: 2,
                        ops: []operation{
                                {op: "set", key: 1, val: 10},
                                {op: "set", key: 1, val: 99},
                                {op: "get", key: 1, val: 99, ok: true},
                        },
                },
                {
                        name: "eviction of oldest key on saturation",
```

```
                size: 2,
                ops: []operation{
                        {op: "set", key: 1, val: 10},
                        {op: "set", key: 2, val: 20},
                        {op: "set", key: 3, val: 30},
                        {op: "get", key: 1, ok: false},
                        {op: "get", key: 2, val: 20, ok: true},
                        {op: "get", key: 3, val: 30, ok: true},
                },
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                cache := Cache[int, int]{}
                cache.Init(test.size)

                for i, operation := range test.ops {
                        switch operation.op {
                        case "set":
                                cache.Set(operation.key, operation.val)
                        case "get":
                                val, ok := cache.Get(operation.key)

                                if ok != operation.ok {
                                        t.Errorf("step %d: expected ok to be
                %v for key %d, got %v", i,
                operation.ok, operation.key, ok)
                                } else if ok && val != operation.val {
                                        t.Errorf("step %d: expected value to
                be %d for key %d, got %d", i,
                operation.val, operation.key, val)
                                }
```

```
                                default:
                                        t.Fatalf("unknown operation %q",
                    operation.op)
                                }
                        }
                })
        }
}
```

C.10.5 util.go

```
package util

import (
        "math"
        "slices"
        "time"
)

// Number is an interface that allows generic usage with any
float or int.
type Number interface {
        ~int | ~int8 | ~int16 | ~int32 | ~int64 | ~uint | ~uint8 |
    ~uint16 |
                ~uint32 | ~uint64 | ~uintptr | ~float32 | ~float64
}

// Range stores a pair of numbers, and Within(n) returns true
if start <= n <= end.
type Range[T Number] struct {
        start T
        end   T
}
```

```
const (
        MAX_DURATION time.Duration = time.Hour * 292      // The
    maximum possible duration value.
        MAX_INT64    int64         = 9223372036854775807 // The
    maximum possible int64 value.
        MAX_FLOAT64  float64       = 1.7e+308            // The
    maximum possible float64 value.
)

// ArrayFromChannel takes a channel and returns an array
containing all the elements queued.
func ArrayFromChannel[T any](channel chan T) []T {
        sliceT := make([]T, 0, len(channel))
        for elem := range channel {
                sliceT = append(sliceT, elem)
        }

        return sliceT
}

// linearInterpolate returns an interpolated value across the
list input.
func linearInterpolate[T Number](list []T, t float64) float64 {
        slices.Sort(list)

        length := len(list) - 1
        lower := int(math.Floor(float64(length) * t))
        upper := int(math.Ceil(float64(length) * t))
        if len(list) == 0 {
                return 0.0
        } else if lower == upper {
                return float64(list[lower])
```

```
        } else {
                fraction := (float64(length) * t) -
        math.Floor(float64(length)*t)
                return float64(list[lower]) + float64(list[upper]-
        list[lower])*fraction
        }
}

// Median returns the median of any slice of ints or floats.
//
// Sorts as a side effect.
func Median[T Number](list []T) float64 {
        if len(list) == 0 {
                return MAX_FLOAT64
        }
        val := linearInterpolate(list, 0.5)
        return val
}

// IQR returns the IQR of any slice of ints or floats.
//
// Sorts as a side effect.
func IQR[T Number](list []T) float64 {
        if len(list) == 0 {
                return MAX_FLOAT64
        }
        val := linearInterpolate(list, 0.75) -
    linearInterpolate(list, 0.25)
        return val
}

// ForEach applies a function to every element of a slice.
func ForEach[T, S any](slice []T, f func(T) S) []S {
```

```
        newSlice := make([]S, len(slice))

        for i, v := range slice {
                newSlice[i] = f(v)
        }

        return newSlice
}

// NewRange returns an initialised range from numbers.
func NewRange[T Number](a, b T) *Range[T] {
        newRange := Range[T]{a, b}

        return &newRange
}

// Within tests whether a value falls between the two values
of a range.
func (r *Range[T]) Within(n T) bool {
        return r.start <= n && n <= r.end
}

// getMinMaxTimes returns the minimum and maximum expected
time bounds until
// execution will finish. This is done via calculating average
generation time
// and extrapolating based on convergence progress.
func GetMinMaxTimes(startTime time.Time, maxTime time.Duration,
        genCount, convergence int) (time.Duration, time.Duration) {
        avgTime := time.Duration(0)
        if genCount > 0 {
                avgTime = time.Since(startTime) /
        time.Duration(genCount)
```

```
        }
        minGens := max(11-genCount, 10-convergence)

        minT := time.Duration(minGens) * avgTime
        maxT := maxTime - time.Since(startTime)

        minT = min(minT, maxT) // minETA should not be greater
    than maxETA

        return minT, maxT
}
```

C.10.6 util_test.go

```
package util

import (
        "slices"
        "strconv"
        "testing"
        "time"
)

func TestArrayFromChannel(t *testing.T) {
        tests := []struct {
                name     string
                setup    func() chan int
                expected []int
        }{
                {
                        name: "buffered channel with elements",
                        setup: func() chan int {
                                ch := make(chan int, 3)
```

```
                ch <- 1
                ch <- 2
                ch <- 3
                close(ch)
                return ch
        },
        expected: []int{1, 2, 3},
    },
    {
        name: "empty channel",
        setup: func() chan int {
                ch := make(chan int)
                close(ch)
                return ch
        },
        expected: []int{},
    },
    {
        name: "unbuffered channel populated via goroutine",
        setup: func() chan int {
                ch := make(chan int)
                go func() {
                        for i := 10; i <= 12; i++ {
                                ch <- i
                        }
                        close(ch)
                }()
                return ch
        },
        expected: []int{10, 11, 12},
    },
}
```

```
        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        ch := test.setup()
                        got := ArrayFromChannel(ch)
                        if !slices.Equal(got, test.expected) {
                                t.Errorf("generated array %v, wanted %v", got,
                test.expected)
                        }
                })
        }
}

func TestLinearInterpolate(t *testing.T) {
        tests := []struct {
                name     string
                list     []float64
                tVal     float64
                expected float64
        }{
                {
                        name:     "empty list",
                        list:     []float64{},
                        tVal:     0.5,
                        expected: 0.0,
                },
                {
                        name:     "single element",
                        list:     []float64{42.0},
                        tVal:     0.5,
                        expected: 42.0,
                },
                {
                        name:     "two elements median",
```

```
        list:     []float64{10.0, 20.0},
        tVal:     0.5,
        expected: 15.0,
    },
    {
        name:     "three elements median",
        list:     []float64{10.0, 20.0, 30.0},
        tVal:     0.5,
        expected: 20.0,
    },
    {
        name:     "four elements lower quartile",
        list:     []float64{10.0, 20.0, 30.0, 40.0},
        tVal:     0.25,
        expected: 17.5,
    },
    {
        name:     "four elements upper quartile",
        list:     []float64{10.0, 20.0, 30.0, 40.0},
        tVal:     0.75,
        expected: 32.5,
    },
    {
        name:     "lower bound",
        list:     []float64{10.0, 20.0},
        tVal:     0.0,
        expected: 10.0,
    },
    {
        name:     "upper bound",
        list:     []float64{10.0, 20.0},
        tVal:     1.0,
        expected: 20.0,
```

```
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        got := linearInterpolate(test.list, test.tVal)
                        if got != test.expected {
                                t.Errorf("got interpolated value %v, wanted
                %v", got, test.expected)
                        }
                })
        }
}

func TestMedian(t *testing.T) {
        tests := []struct {
                name     string
                list     []int
                expected float64
        }{
                {
                        name:     "empty list returns MAX_FLOAT64",
                        list:     []int{},
                        expected: MAX_FLOAT64,
                },
                {
                        name:     "odd number of elements",
                        list:     []int{3, 1, 2},
                        expected: 2.0,
                },
                {
                        name:     "even number of elements",
                        list:     []int{1, 4, 2, 3},
```

```
                expected: 2.5,
        },
        {
                name:     "negative values",
                list:     []int{-10, -50, -30},
                expected: -30.0,
        },
    }

    for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
            listCopy := slices.Clone(test.list)
            got := Median(listCopy)
            if got != test.expected {
                t.Errorf("got median of %v, want %v", got,
        test.expected)
            }
        })
    }
}

func TestIQR(t *testing.T) {
    tests := []struct {
        name     string
        list     []float64
        expected float64
    }{
        {
            name:     "empty list returns MAX_FLOAT64",
            list:     []float64{},
            expected: MAX_FLOAT64,
        },
        {
```

```
                name:     "standard sequence",
                list:     []float64{10.0, 20.0, 30.0, 40.0},
                expected: 15.0,
            },
            {
                name:     "single element sequence",
                list:     []float64{10.0},
                expected: 0.0,
            },
        }

        for _, test := range tests {
            t.Run(test.name, func(t *testing.T) {
                listCopy := slices.Clone(test.list)
                got := IQR(listCopy)
                if got != test.expected {
                    t.Errorf("got iqr of %v, wanted %v", got,
            test.expected)
                }
            })
        }
}

func TestForEach(t *testing.T) {
        t.Run("int to int (multiply by 2)", func(t *testing.T) {
            input := []int{1, 2, 3}
            expected := []int{2, 4, 6}
            got := ForEach(input, func(i int) int {
                return i * 2
            })
            if !slices.Equal(got, expected) {
                t.Errorf("applied foreach got %v, wanted %v", got,
            expected)
```

```
                }
        })

        t.Run("int to string format", func(t *testing.T) {
                input := []int{10, 20}
                expected := []string{"10", "20"}
                got := ForEach(input, strconv.Itoa)
                if !slices.Equal(got, expected) {
                        t.Errorf("applied foreach got %v, wanted %v", got,
    expected)
                }
        })

        t.Run("empty slice", func(t *testing.T) {
                input := []int{}
                expected := []int{}
                got := ForEach(input, func(i int) int { return i + 1 })
                if !slices.Equal(got, expected) {
                        t.Errorf("foreach on empty slice got %v, want %v",
    got, expected)
                }
        })
}

func TestRange_Within(t *testing.T) {
        tests := []struct {
                name     string
                rng      *Range[int]
                val      int
                expected bool
        }{
                {
                        name:     "value strictly inside",
```

```
        rng:      NewRange(10, 20),
        val:      15,
        expected: true,
    },
    {
        name:     "value on lower boundary",
        rng:      NewRange(10, 20),
        val:      10,
        expected: true,
    },
    {
        name:     "value on upper boundary",
        rng:      NewRange(10, 20),
        val:      20,
        expected: true,
    },
    {
        name:     "value below lower boundary",
        rng:      NewRange(10, 20),
        val:      9,
        expected: false,
    },
    {
        name:     "value above upper boundary",
        rng:      NewRange(10, 20),
        val:      21,
        expected: false,
    },
    {
        name:     "inverted range bounds (start > end)",
        rng:      NewRange(20, 10),
        val:      15,
        expected: false},
```

```
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        got := test.rng.Within(test.val)
                        if got != test.expected {
                                t.Errorf("Range(%v, %v).Within(%v) = %v, want
                %v",
                                        test.rng.start, test.rng.end, test.val,
                    got, test.expected)
                        }
                })
        }
}

func TestGetMinMaxTimes(t *testing.T) {
        withinTolerance := func(a, b time.Duration) bool {
                const tolerance = 50 * time.Millisecond

                diff := a - b
                if diff < 0 {
                        diff = -diff
                }
                return diff <= tolerance
        }

        tests := []struct {
                name        string
                elapsedTime time.Duration
                maxTime     time.Duration
                genCount    int
                convergence int
                wantMinT    time.Duration
```

```
        wantMaxT     time.Duration
}{
        {
                name:        "Zero generations completed",
                elapsedTime: 10 * time.Second,
                maxTime:     100 * time.Second,
                genCount:    0,
                convergence: 0,
                wantMinT:    0,
                wantMaxT:    90 * time.Second,
        },
        {
                name:        "Early generations, bottlenecked by
    convergence",
                elapsedTime: 10 * time.Second,
                maxTime:     100 * time.Second,
                genCount:    5,
                convergence: 2,
                wantMinT:    16 * time.Second,
                wantMaxT:    90 * time.Second,
        },
        {
                name:        "High convergencce, bottlenecked by
    early generation",
                elapsedTime: 8 * time.Second,
                maxTime:     100 * time.Second,
                genCount:    8,
                convergence: 9,
                wantMinT:    3 * time.Second,
                wantMaxT:    92 * time.Second,
        },
        {
                name:        "Zero minGens expected",
```

```
                elapsedTime: 40 * time.Second,
                maxTime:     100 * time.Second,
                genCount:    20,
                convergence: 10,
                wantMinT:    0,
                wantMaxT:    60 * time.Second,
        },
        {
                name:        "minT caps to maxT",
                elapsedTime: 95 * time.Second,
                maxTime:     100 * time.Second,
                genCount:    10,
                convergence: 0,
                wantMinT:    5 * time.Second,
                wantMaxT:    5 * time.Second,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                startTime := time.Now().Add(-test.elapsedTime)
                minT, maxT := GetMinMaxTimes(startTime,
    test.maxTime, test.genCount, test.convergence)

                if !withinTolerance(minT, test.wantMinT) {
                        t.Errorf("Got min of %v, expected %v", minT,
        test.wantMinT)
                }
                if !withinTolerance(maxT, test.wantMaxT) {
                        t.Errorf("Got max of %v, expected %v", maxT,
        test.wantMaxT)
                }
        })
```

```
        }
}
```

C.10.7 fileio.go

```
package util

import (
        "bytes"
        "encoding/json"
        "math/rand"
        "os"
)

// SaveSequence saves a passed sequence to the file specified,
as a new line.
func SaveSequence(filename string, sequence []int) error {
        f, err := os.OpenFile(filename, os.O_APPEND|os.O_CREATE|
    os.O_WRONLY, 0644)
        if err != nil {
                return err
        }
        defer f.Close()

        if err := json.NewEncoder(f).Encode(sequence); err != nil {
                return err
        }

        return nil
}

// LoadSequences returns up to n unique, random sequences from
a specified file.
```

```
func LoadSequences(filename string, n int) ([][]int, error) {
        data, err := os.ReadFile(filename)
        if err != nil {
                return nil, err
        }

        lines := bytes.Split(bytes.TrimSpace(data), []byte("\n"))

        rand.Shuffle(len(lines), func(i, j int) { lines[i],
    lines[j] = lines[j], lines[i] })

        count := n
        if len(lines) < n {
                count = len(lines)
        }

        var results [][]int
        for i := 0; i < count; i++ {
                if len(lines[i]) == 0 { // blank line
                        continue
                }

                var seq []int
                if err := json.Unmarshal(lines[i], &seq); err != nil {
                        return nil, err
                }
                results = append(results, seq)
        }

        return results, nil
}
```

C.10.8 fileio_test.go

```
package util

import (
        "os"
        "path/filepath"
        "testing"
)

// TestSaveSequence assesses the functionality of
util.SaveSequence
func TestSaveSequence(t *testing.T) {
        tempDir := t.TempDir()
        validFile := filepath.Join(tempDir, "sequences.json")
        invalidFile := filepath.Join(tempDir, "nonexistent",
    "seq.json")

        tests := []struct {
                name     string
                filename string
                sequence []int
                wantErr  bool
        }{
                {
                        name:     "Create new file and save successfully",
                        filename: validFile,
                        sequence: []int{1, 2, 3},
                        wantErr:  false,
                },
                {
                        name:     "Append to existing file successfully",
                        filename: validFile,
                        sequence: []int{4, 5, 6},
                        wantErr:  false,
```

```
                },
                {
                        name:     "Fail to save to an invalid directory
            path",
                        filename: invalidFile,
                        sequence: []int{7, 8},
                        wantErr:  true,
                },
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        err := SaveSequence(test.filename, test.sequence)
                        if (err != nil) != test.wantErr {
                                t.Errorf("SaveSequence() error = %v, wantErr
                %v", err, test.wantErr)
                        }
                })
        }
}

// TestLoadSequence assesses the functionality of
util.LoadSequence
func TestLoadSequences(t *testing.T) {
        tempDir := t.TempDir()

        // Helper function to write mock data to temporary files
        writeMockFile := func(name, content string) string {
                path := filepath.Join(tempDir, name)
                _ = os.WriteFile(path, []byte(content), 0644)
                return path
        }
```

```
    validFile := writeMockFile("valid.json",
"[1,2,3]\n[4,5,6]\n[7,8,9]\n")
    malformedFile := writeMockFile("malformed.json",
"[1,2,3]\n[4,5,\n")
    blankLineFile := writeMockFile("blank.json",
"[1,2]\n\n[3,4]")

    tests := []struct {
            name     string
            filename string
            n        int
            wantLen  int
            wantErr  bool
    }{
            {
                    name:     "Load exact number of available
        sequences",
                    filename: validFile,
                    n:        3,
                    wantLen:  3,
                    wantErr:  false,
            },
            {
                    name:     "Load fewer than available sequences",
                    filename: validFile,
                    n:        2,
                    wantLen:  2,
                    wantErr:  false,
            },
            {
                    name:     "Request more sequences than are
        available",
                    filename: validFile,
```

```
                n:          10,
                wantLen:    3,
                wantErr:    false,
        },
        {
                name:       "Return error when file does not exist",
                filename:   filepath.Join(tempDir, "missing.json"),
                n:          2,
                wantLen:    0,
                wantErr:    true,
        },
        {
                name:       "Return error on malformed JSON data",
                filename:   malformedFile,
                n:          2,
                wantLen:    0,
                wantErr:    true,
        },
        {
                name:       "Skip blank lines properly",
                filename:   blankLineFile,
                n:          5,
                wantLen:    2,
                wantErr:    false,
        },
}

for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
                got, err := LoadSequences(test.filename, test.n)

                if (err != nil) != test.wantErr {
                        t.Errorf("Error loading sequence: %v", err)
```

```
                }

                if !test.wantErr && len(got) != test.wantLen {
                        t.Errorf("LoadSequences returned %d sequences,
        wanted %d", len(got), test.wantLen)
                }
        })
    }
}
```

C.11 workerpool

C.11.1 ga_worker_pool.go

```
package workerpool

import (
    "6CCS3PRJ/thomasholden/passmanager"
    "6CCS3PRJ/thomasholden/testharness"
    "6CCS3PRJ/thomasholden/util"
    "fmt"

    "sync"
    "time"

    "github.com/schollz/progressbar/v3"
)

// Result holds the results for a singular pass sequence's
execution
type Result struct {
    PassSeq []int
    Score   float64
```

```
}

// WorkerPool provides functionality for pooling the execution
of LLVM optimisation executions.
type WorkerPool struct {
        passSeq    [][]int
        SourceCmd []string // the command from which the project
    should be built
        cache      util.Cache[string, float64]
        fileNum    chan int
        Results    chan Result // a channel containing all results
        passMan    *passmanager.PassManager
}

const (
        COMPILATION_WEIGHT  float64 = 0.0 // The proportion of the
    score to be made up of compile time.
        MEMORY_USAGE_WEIGHT float64 = 0.0 // The proportion of the
    score to be made up of execution's max RSS.
        BINARY_SIZE_WEIGHT  float64 = 0.0 // The proportion of the
    score to be made up of compiled binary size.

        // The proportion of the score to be made up of execution
    time.
        EXECUTION_WEIGHT float64 = 1 - COMPILATION_WEIGHT -
    MEMORY_USAGE_WEIGHT - BINARY_SIZE_WEIGHT
)

var PBar *progressbar.ProgressBar

// NewWorkerPool returns a pointer to a worker pool.
func NewWorkerPool(sourceCmd []string, workers, numJobs int,
baseline []int) (*WorkerPool, error) {
```

```
    pool := new(WorkerPool)

    pool.passSeq = make([][]int, 0, numJobs)
    pool.fileNum = make(chan int, workers)

    // Preps the channel that controls how many LLVM sessions
can be active at once
    for i := 0; i < workers; i++ {
            pool.fileNum <- i
    }

    pool.SourceCmd = sourceCmd

    pool.passMan = passmanager.GetPassManager()

    pool.cache.Init(256)

    validO3 := pool.passMan.MakeValid(baseline)
    o3Time, _, err :=
testharness.GetSetBaselineStats(sourceCmd, validO3,
            func() {
                    fileNum := <-pool.fileNum
                    defer func() { pool.fileNum <- fileNum }()
                    pool.runJob([]int{}, fileNum, true)
            },
            workers)
    if err != nil {
            return nil, err
    }

    fmt.Printf("\nO3 Time: %v\n", o3Time)
    pool.cache.Set(validO3, valueFromJobStats(0, o3Time, 0, 0))
```

```
        PBar = progressbar.Default(-1)

        return pool, nil
}

// AddPassSequence appends a sequence to be queued for
execution.
func (p *WorkerPool) AddPassSequence(seq []int) {
        p.passSeq = append(p.passSeq, seq)
}

// StartPool begins worker pool execution, returning the
results.
//
// Handles pooled parallel execution.
func (p *WorkerPool) StartPool() []Result {
        var wg sync.WaitGroup
        p.Results = make(chan Result, len(p.passSeq))

        for _, seq := range p.passSeq {
                fileNum := <-p.fileNum
                wg.Go(func() { p.runJob(seq, fileNum, false) })
        }

        p.manageCPULoad(&wg)

        wg.Wait()

        p.passSeq = make([][]int, 0, len(p.passSeq))
        close(p.Results)
        return util.ArrayFromChannel(p.Results)
}
```

```
// manageCPULoad handles dynamically adding workers to
maintain CPU load towards the
// end of the pool's duration.
func (p *WorkerPool) manageCPULoad(wg *sync.WaitGroup) {
        for len(p.Results) != cap(p.Results) {
                fileNum := <-p.fileNum
                wg.Go(func() {
                        defer func() { p.fileNum <- fileNum }()
                        p.runJob([]int{}, fileNum, true)
                })
        }
}

// runJob executes a single pass sequences optimisation
pipeline to obtain its resulting score.
func (p *WorkerPool) runJob(seq []int, diff int, dummy bool) {
        if !dummy {
                defer PBar.Add(1)
                defer func() { p.fileNum <- diff }()
        }

        validSeq := p.passMan.MakeValid(seq)

        if score, ok := p.cache.Get(validSeq); ok && !dummy {
                p.Results <- Result{
                        Score:   score,
                        PassSeq: seq,
                }
                return
        }

        compilation_t, execution_t, binary_size, maxrss, err :=
    (testharness.CompileAndExecute(validSeq, diff, false,
```

```
    p.SourceCmd))
        if dummy {
                return
        }
        finalScore := valueFromJobStats(compilation_t,
    time.Duration(execution_t), binary_size, maxrss)
        defer p.cache.Set(validSeq, finalScore) // Update our
    mapped cache with the score achieved for the sequence
        if err != nil {
                p.queueErrorResult(seq)
        } else {
                p.Results <- Result{
                        Score:   finalScore,
                        PassSeq: seq,
                }
        }
}

// valueFromJobStats returns our heuristic value based upon
the optimisation pipelines statistics.
func valueFromJobStats(compilation, execution time.Duration,
size int, memory int64) float64 {
        return
    (COMPILATION_WEIGHT*float64(compilation.Microseconds()) +
                EXECUTION_WEIGHT*float64(execution.Microseconds()) +
                BINARY_SIZE_WEIGHT*float64(size) +
                MEMORY_USAGE_WEIGHT*float64(memory))
}

// queueErrorResult adds a result to the Result channel
indicating sequence failure.
func (p *WorkerPool) queueErrorResult(seq []int) {
        p.Results <- Result{
```

```
                Score:    util.MAX_FLOAT64,
                PassSeq: seq,
        }
}
```

C.11.2 ga_worker_pool_test.go

```
package workerpool

import (
        "6CCS3PRJ/thomasholden/passmanager"
        "6CCS3PRJ/thomasholden/util"
        "slices"
        "testing"
        "time"

        "github.com/schollz/progressbar/v3"
)

// pbarInit ensure PBar is initialized for tests.
func pbarInit() {
        if PBar == nil {
                PBar = progressbar.DefaultSilent(-1)
        }
}

// getTestPassManager creates a pass manager for testing
purposes.
func getTestPassManager() *passmanager.PassManager {
        return passmanager.GetPassManager()
}

// TestValueFromJobStats tests the ValueFromJobStats function.
```

```
func TestValueFromJobStats(t *testing.T) {
        pbarInit()

        tests := []struct {
                name        string
                compilation time.Duration
                execution   time.Duration
                size        int
                memory      int64
                want        float64
        }{
                {
                        name:        "Zero values",
                        compilation: 0,
                        execution:   0,
                        size:        0,
                        memory:      0,
                        want:        0.0,
                },
                {
                        name:        "Random values",
                        compilation: time.Second,
                        execution:   time.Second,
                        size:        1024,
                        memory:      2048,
                        want:
            (COMPILATION_WEIGHT*float64(time.Second.Microsecond
            s()) +
                                EXECUTION_WEIGHT*float64(time.Second.Microsecon
            ds()) +
                                BINARY_SIZE_WEIGHT*float64(1024) +
                                MEMORY_USAGE_WEIGHT*float64(2048)),
                },
```

```
        }

        for _, test := range tests {
                t.Run(test.name, func(t *testing.T) {
                        got := valueFromJobStats(test.compilation,
            test.execution, test.size, test.memory)
                        if got != test.want {
                                t.Errorf("valueFromJobStats() = %v, want %v",
                got, test.want)
                        }
                })
        }
}

// TestWorkerPoolAddPassSequence tests the AddPassSequence
function.
func TestWorkerPoolAddPassSequence(t *testing.T) {
        pbarInit()

        pool := &WorkerPool{
                passSeq: make([][]int, 0),
        }

        seq := []int{1, 2, 3}
        pool.AddPassSequence(seq)

        if len(pool.passSeq) != 1 {
                t.Fatalf("Expected passSeq length 1, got %d",
        len(pool.passSeq))
        }

        if !slices.Equal(pool.passSeq[0], seq) {
                t.Errorf("Expected sequence %v, got %v", seq,
```

```
        pool.passSeq[0])
        }
}

// TestWorkerPoolStartPoolCached verifies the worker pool
logic by mocking the cache.
//
// This allows us to test the pool management, threading, and
result aggregation
// without triggering the compiler.
func TestWorkerPoolStartPoolCached(t *testing.T) {
        pbarInit()

        workers := 2
        numJobs := 3

        // Bypass NewWorkerPool's external calls
        pool := &WorkerPool{
                passSeq:   make([][]int, 0, numJobs),
                fileNum:   make(chan int, workers),
                SourceCmd: []string{"clang", "main.c"},
                passMan:   getTestPassManager(),
        }

        for i := 0; i < workers; i++ {
                pool.fileNum <- i
        }

        pool.cache.Init(256)

        inputSequences := [][]int{
                {10, 20},
                {30, 40},
```

```
        {50, 60},
}

expectedScores := map[string]float64{
        pool.passMan.MakeValid(inputSequences[0]): 100.0,
        pool.passMan.MakeValid(inputSequences[1]): 200.0,
        pool.passMan.MakeValid(inputSequences[2]): 300.0,
}

for _, seq := range inputSequences {
        pool.AddPassSequence(seq)
        validSeq := pool.passMan.MakeValid(seq)
        pool.cache.Set(validSeq, expectedScores[validSeq])
}

t.Log("Starting Pool")
results := pool.StartPool()
t.Log("Finishing Pool")

if len(results) != numJobs {
        t.Fatalf("Expected %d results, got %d", numJobs,
len(results))
}

resultMap := make(map[string]float64)
for _, r := range results {
        key := pool.passMan.MakeValid(r.PassSeq)
        resultMap[key] = r.Score
}

for key, wantScore := range expectedScores {
        gotScore, ok := resultMap[key]
        if !ok {
```

```
                        t.Errorf("Result missing for sequence %s", key)
                        continue
                }
                if gotScore != wantScore {
                        t.Errorf("Score mismatch for %s: got %v, want %v",
            key, gotScore, wantScore)
                }
        }

        t.Log("Waiting on channel")
        select {
        case _, ok := <-pool.Results:
                if ok {
                        t.Error("Results channel should be closed")
                }
        default:
        }
}

// TestWorkerPoolQueueErrorResult tests the queueErrorResult
function.
func TestWorkerPoolQueueErrorResult(t *testing.T) {
        pbarInit()

        pool := &WorkerPool{
                Results: make(chan Result, 1),
        }

        seq := []int{999}
        pool.queueErrorResult(seq)

        close(pool.Results)
```

```
        res := <-pool.Results
        if res.Score != util.MAX_FLOAT64 {
                t.Errorf("Expected error score %v, got %v",
        util.MAX_FLOAT64, res.Score)
        }
}

// TestManageCPULoad simulates the scenario where the result
channel is not yet full
func TestManageCPULoad(t *testing.T) {
        pbarInit()

        workers := 2
        pool := &WorkerPool{
                passSeq: [][]int{{1}, {2}}, // 2 jobs
                fileNum: make(chan int, workers),
                Results: make(chan Result, 2),
                passMan: getTestPassManager(),
        }
        for i := 0; i < workers; i++ {
                pool.fileNum <- i
        }
        pool.cache.Init(10)

        pool.cache.Set(pool.passMan.MakeValid([]int{1}), 1.0)
        pool.cache.Set(pool.passMan.MakeValid([]int{2}), 1.0)

        results := pool.StartPool()

        if len(results) != 2 {
                t.Errorf("Expected 2 results, got %d", len(results))
        }
}
```